Smithian Fold Theory: A Zero-Axiom, Zero-Free-Parameter Derivation of the Fine-Structure Constant — Lean 4 Formal Verification

Smithian Fold Theory: the cumulative theory-of-everything manuscript
Maria Smith
Publication edition 0.1 | 29 October 2026
DOI: 10.5281/zenodo.23782217
Author-approved preprint

The full SFT model, formalised in Lean.
Alpha is the opening result.

109 / 2,781 completed entries
6,024 audited theorems | Zero axiom dependencies

Contents

1. Opening result: the fine-structure constant . . . 11

2. A cumulative paper, released at scientific milestones . . . 12

3. Reading and reproducing this edition . . . 12

4. Source and evidence links . . . 12

5. Model and scope . . . 13

6. Operational root . . . 13

7. Structural One . . . 14

8. Exact positive finite count . . . 15

9. Exact positive part coordinate . . . 16

10. Generated common-refinement pairing . . . 16

11. Minimal Count extension in Fold . . . 17

12. Complete finite Fold-word assembly and return . . . 17

13. Formal representation and results . . . 24

14. Correspondence and reproducibility . . . 25

15. Extension of the formalisation . . . 26

16. Source . . . 26

17. Discrete collections and held selections . . . 26

18. Finite collection induction . . . 27

19. Complementary discrete relations . . . 27

20. Composition of discrete maps . . . 27

21. Primitive Fold maps and ordered composition . . . 28

22. Least-size primitive generator . . . 28

23. Ordered exact derivation traces . . . 29

24. Finite combinatorial selections . . . 29

25. Finite arrangements by held choice . . . 30

26. Arrangement completeness and uniqueness . . . 30
27. Disjoint class accounting . . . 30
28. Combinatorial witnesses and complete grammar . . . 31
29. Selection encoding and extension . . . 31
30. Arrangement successor partition and provenance . . . 31
31. Arrangement trace accounting and arithmetic linkage . . . 31
32. Complete encoded selection output . . . 32
33. Discrete collection integration . . . 32
34. Ordered selection provenance . . . 32
35. Raw choices as discrete held selections . . . 33
36. Finite algebraic operation tables . . . 33
37. Complete single-valued operation support . . . 33
38. Exhaustive identity and association . . . 33
39. Held return mates and commutation . . . 34
40. Public return and commutation semantics . . . 34
41. Operation-preserving maps . . . 34
42. Total structure-map semantics . . . 34
43. Original algebra witnesses and complete grammar . . . 35
44. Algebra program correspondence . . . 35
45. Algebraic induction base and carrier extension . . . 35
46. Identity and association under carrier extension . . . 35
47. Return mates under carrier extension . . . 36
48. Operation maps under carrier extension . . . 36
49. Concrete dictionary extension . . . 36
50. Finite order and bound operations . . . 36
51. Exact partial-order admission . . . 37
52. Extremal witnesses and returned bounds . . . 37

53. Conditional meet/join existence and monotonicity . . . 37
54. Complete order grammar and program correspondence . . . 38
55. Order induction base and comparison extension . . . 38
56. Bound lists under carrier extension . . . 38
57. Held-selection updates and extremal retention . . . 38
58. Extremal selection of the added carrier member . . . 39
59. Generated graphs, complete paths and cuts . . . 39
60. Finite reachability search . . . 39
61. Complete path witnesses for successful reachability . . . 40
62. Reachability completeness and equivalence . . . 40
63. Exact finite cycle traversal . . . 40
64. Cycle detection and complete return paths . . . 41
65. Graph witnesses and complete registered grammar . . . 41
66. Graph induction and locality of path updates . . . 42
67. Admitted graph extension, ordered pairs, cuts and balance . . . 42
68. Held-partition updates and exact paired-balance conditions . . . 42
69. Batch source-program correspondence . . . 43
70. Complete Part Equivalence source correspondence . . . 43
71. Finite Form source input correspondence . . . 43
72. Completed mathematics source entries . . . 44
73. Completion of the represented foundation and mathematics entries . . . 44
74. Finite incidence and open-family derivations . . . 44
75. Boundary-depth dimension . . . 45
76. Ordered undirected path search . . . 45
77. General shortest-path and reachability derivations . . . 45
78. Completed finite geometry and topology entry . . . 46
79. Exact events and common-refinement weights . . . 46

80. Pair-cell factorization and exact modal summaries . . . 47
81. Observation partitions and completed finite probability entry . . . 47
82. Exact finite optimization and retained alternatives . . . 47
83. Generated trajectories and predecessor records . . . 48
84. First returns and complete registered perturbation stability . . . 48
85. Finite logic and registered proof traces . . . 49
86. Compositional paths and finite type constructors . . . 49
87. Identity, transport and route comparison . . . 50
88. Symbols and observation-relative distinction . . . 50
89. Observation classes, complete distinction ledgers and extension . . . 50
90. Encoding classes and exact inverse decoding . . . 51
91. Closed encoding distinctions and exact composition . . . 51
92. Encoding source extension and finite closure . . . 51
93. Complete information support and position generation . . . 52
94. Exact information profiles and independent position traces . . . 52
95. Deterministic entropy partitions and observation refinement . . . 53
96. Exact entropy parts and closed-pair monotonicity . . . 53
97. Entropy successor closure and exact finite completion . . . 53
98. Exact compression descriptions and reconstruction provenance . . . 54
99. Prefix parsing and the complete-support shortening boundary . . . 54
100. Finite channel admission and exact confusability . . . 55
101. Complete finite capacity families and relation composition . . . 55
102. Fresh-input closure of exact channel capacity . . . 56
103. Registered action relations and exact correction . . . 56
104. Complete position traces, detection and action succession . . . 56
105. Complete coding support and held codebooks . . . 57
106. Complete single-label error images and exact decoding . . . 57

107. General coding action succession and retained source provenance . . . 58
108. Exact observation incidence and conditional class restriction . . . 59
109. Exact joint fractions and marginal-product comparison . . . 59
110. Conditional fractions, resolution and retained dependence . . . 59
111. Fresh-source joint and conditional succession . . . 60
112. Complete mutual and conditional program correspondence . . . 60
113. Exact conservation and loss of source distinctions . . . 60
114. Composition and downstream retention . . . 61
115. Exact predecessor records and recovery . . . 61
116. Reversibility is equivalent to singleton fibres . . . 61
117. Source succession and conservation of pair accounting . . . 62
118. Complete conservation and transformation program correspondence . . . 62
119. Classical state, observation classes and deterministic realization . . . 62
120. Complete observation-class cardinality . . . 62
121. Exact-whole closure and microstate succession . . . 63
122. Classical and probabilistic operational witnesses . . . 63
123. Complete classical and probabilistic correspondence . . . 63
124. Quantum-information branch support and provenance . . . 63
125. Basis records and joint support . . . 64
126. Independently realized information-unit traces . . . 64
127. Exact support observation and reconstruction . . . 64
128. Canonical position succession . . . 65
129. Quantum support program correspondence . . . 65
130. Canonical Fold basis bijection . . . 65
131. Exact generated transition systems . . . 65
132. Retained execution composition and extension . . . 66
133. Generated words and exact coordinate rewrite . . . 66

134. Finite language derivation certificates . . . 66
135. Finite recognition with retained accepting classes . . . 67
136. Retained held-position rewrite traces . . . 67
137. Registered rewrite boundaries and trace classes . . . 67
138. Structural finite recursion with complete values . . . 68
139. Fold binding and self-application calculus . . . 68
140. Separate FORMX source components . . . 68
141. Finite executable descriptions and halt traces . . . 69
142. Circuit interfaces and complete evaluation traces . . . 69
143. Operational process observations and failure traces . . . 69
144. Process composition and exact decomposition . . . 70
145. Registered step-to-trace simulation . . . 70
146. Finite registered model translation and reconstruction . . . 70
147. Interpretation of frozen generated descriptions . . . 71
148. Recognition, decision and unresolved execution . . . 71
149. Terminal witnesses and the held halting boundary . . . 71
150. Effective finite enumeration and reconstruction . . . 72
151. Explicit reductions and answer reconstruction . . . 72
152. Finite relative computability classes . . . 73
153. Universal-machine execution and recognition . . . 73
154. Generated diagonal contradiction . . . 73
155. Computational proof-system boundary . . . 74
156. Declared relative answer interfaces . . . 74
157. Hypercomputation admissibility boundary . . . 75
158. Exact generated input extent . . . 75
159. Time and retained-state resources . . . 75
160. Circuit structural resources . . . 76

161. Communication and query resources . . . 76
162. Scheduled unresolved branch resources . . . 76
163. Information retained for inverse execution . . . 77
164. Independent and causally layered work . . . 77
165. Finite resource comparison certificates . . . 77
166. Resource-preserving reductions and completeness . . . 78
167. Complete instance-family resource profiles . . . 78
168. Exact approximation and retained objective distinction . . . 78
169. Resource stratification by structural coordinates . . . 79
170. Shortest exact descriptions at declared finite depth . . . 79
171. Exact search on registered carriers . . . 79
172. Exact ordering and complete recursion traces . . . 79
173. Exact arithmetic execution certificates . . . 80
174. Finite sequence matches and position reconstruction . . . 80
175. Finite graph traversal certificates . . . 80
176. Finite algebraic and geometric execution . . . 81
177. Exact subproblem reuse and reconstruction . . . 81
178. Complete optimization traversal . . . 81
179. Scheduled algorithms and exact success support . . . 82
180. Causally independent algorithm layers . . . 82
181. Global replay of retained local process events . . . 82
182. Exact distributed messages and causal reconstruction . . . 82
183. Finite-prefix streaming and retained distinctions . . . 83
184. Exact oriented numerical-error records . . . 83
185. Complete numerical error-ledger reconstruction . . . 83
186. Retained numerical update observations . . . 84
187. Finite numerical boundary certificates . . . 84

188. Exact symbolic transformations within the frozen grammar 84
189. Exact approximation-bound admission and source rows 85
190. Approximation with complete objective and execution ledgers 85
191. Generated program syntax and complete occurrence records 85
192. Complete name occurrences and registered scope 86
193. Binding-extension formation and exact substitution behavior 86
194. Capture preservation at the evaluator substitution boundary 86
195. Canonical replacement in the registered expression grammar 87
196. Complete source evaluation traces 87
197. Operational and compositional meaning correspondence 87
198. Positive semantic-value typing and preservation 88
199. Generated type support and exact interface derivations 88
200. Program equivalence on registered observations 88
201. Generated descent and terminal coverage 89
202. Correctness against the registered specification 89
203. Generated formal specification relations 89
204. Retained program transformation traces 90
205. Semantics-preserving compilation relations 90
206. Machine-checkable program evidence 90
207. Positive causal relation expansion 91
208. Native causal fixed-point execution 91
209. Exact causal operation boundary and output 91
210. Complete concurrency schedules and local observations 91
211. Computational causality and retained provenance 92
212. Structural generator three: exact recurrence derivation 92
213. Native first-return execution 92
214. Complete Structural Generator Three entry 93

215. Spatial Dimension Three 93

216. Boundary Rank Two 93

217. Inverse fine-structure cover construction 94

218. Terminal inverse fine-structure ratio 94

219. Charged-lepton invariant operations 94

220. Complete charged-lepton symmetric carrier 95

221. Terminal charged-lepton self-coupling 95

222. Terminal grammar and exact root brackets 95

223. Exact terminal mass-ratio refinements 96

224. Exact source rows and complete comparison record 96

225. Registered terminal prediction and empirical validation 96

226. Electroweak Fold-fibre mixing 97

Abstract

Smithian Fold Theory (SFT) develops a structural theory of everything from its operational root, with zero added axioms and zero free scientific parameters. This cumulative preprint presents its native Lean formalisation, preserving the definitions, dependency order, derivations, complete candidate grammars, controls and empirical boundaries of the existing V3 model. The opening result is the exact terminal inverse fine-structure ratio, alpha^(-1) = 503846395469/3676744786. Its binary covers, typed structural blocks, finite direction-promotion ladder and exact rational assembly are derived before measurement; the corresponding Lean proofs have no axiom dependencies. The manuscript then develops the broader verified model, from the operational root, structural One, Count, Part and Fold through mathematics, information, computation and the completed physics results. This edition covers 109 completed entries of the 2,781-entry V3 census. The accompanying formalisation contains 6,024 audited theorems; the audit covers 13,133 declarations and reports zero axiom dependencies. Source correspondence, executable comparisons and chronological records accompany the manuscript. Successive versions will incorporate further completed derivations until the full model is formalised.

1. Opening result: the fine-structure constant

The opening result connects the structural foundation of SFT to an exact dimensionless ratio. The complete derivation is carried into native Lean from the pinned V3 model. Each numerical block retains its generated structural role; the terminal order is fixed by exhaustion of the three-direction promotion object.

Let b = 2 be the positive predecessor of the independently derived generator c = 3. The least complete binary covers of the carriers c^c = 27 and c^(c + 1) = 81 have depths 5 and 7. The independent decompositions give 5 = 2 + 3 and 7 = 3 + 4. These generate the tower 2^7 = 128, boundary 3^2 = 9 and complete cover 2 x 5^3 = 250.

Promoting each of the three interchangeable directions once generates the complete finite ladder 125, 175, 245, 343. Each step replaces one down-depth factor by an up-depth factor. After all three directions have been promoted, no successor is admitted. The first rung, 125, supplies the base cover 250 = 2 x 125 and hence the separate leading-order result; the terminal fourth-order assembly adds the reciprocal continuation built from the remaining rungs 175, 245 and 343. The terminal effective cover and inverse fine-structure ratio are:

D = 250 + 1 / (175 + 1 / (245 + 1 / 343))
alpha^(-1) = 128 + 9(D + 1)/D = 503846395469/3676744786
alpha = 3676744786/503846395469

The leading rung yields alpha^(-1) = 34259/250. The terminal fourth order follows from the complete generated ladder. It is not selected by a numerical fit. The source construction reads no measured value of alpha. Its separately registered empirical comparison retains the original post-seal boundary.

Native proofs establish the least-cover property, complete promotion stages, decreasing remaining-direction count, terminal closure and exact rational identities. All 1,024 registered candidate forms are retained, together with the three operational witnesses and four controls. Complete source-program comparisons retain candidate order, decisions, reasons, output fields and hashes. The entry is SFT-PHYS-CONSTANT-INVERSE-FINE-STRUCTURE-001; its nine registered dependencies were completed before it.

In this edition, zero axioms means the audited Lean declarations report an empty axiom-dependency set. Zero free scientific parameters means that the scientific construction uses the generated V3 quantities and original constraints, without fitted coefficients or measurement-fed choices. Host

indices and explicit inputs of general finite operations retain their source roles. The exact expression above and its proof constitute the opening result of the wider manuscript.

2. A cumulative paper, released at scientific milestones

The paper develops SFT as a unified theory and grows with the native verification of the full V3 model. Alpha introduces the first release. Future opening results will feature the unified constants and further major derivations when their source entries are complete. Expanded editions are planned at 500 completed entries and approximately every 250 entries thereafter, with additional releases for substantial scientific milestones. Each edition incorporates all newly completed scientific material and identifies its corresponding source and verification snapshot.

The full scientific exposition follows in the same manuscript. The release includes the chronological timeline, coverage census, source correspondence, native Lean sources and retained verification evidence as separately accessible companion files. Development history belongs to the timeline; scientific definitions, derivations, methods and results belong to the paper.

3. Reading and reproducing this edition

Start with the operational root in Section 6 and follow the registered dependencies through the structural and mathematical constructions. The physics sections include Structural Generator Three, Spatial Dimension Three, Boundary Rank Two, Inverse Fine Structure, Charged-Lepton Cubic, Charged-Lepton Terminal and Electroweak Fold Mixing. The complete alpha construction is developed under "Inverse fine-structure cover construction" and "Terminal inverse fine-structure ratio".

The immutable scientific code checkpoint for this edition is b8f732515dd06e8251c06b1c45e3fdfb858b986d in MettaMazza/SFT-v4-LEAN-VERIFIED. The V3 model is pinned to 76f1b34e9e9682989adcbb088797b4ae4ce4fcf1 in MettaMazza/ernos-labs-sft-platform. Lean is pinned to leanprover/lean4:v4.35.0. COVERAGE.json identifies completed source entries; reference/CORRESPONDENCE.json maps translated sources. The retained TURN_271_ALPHA_COMPLETION.json records the alpha derivation components, and TURN_279_PARITY_REPORT.json records the latest aggregate build, candidate-output comparison and axiom audit. Entry completion and the count of individual theorems are reported separately.

To reproduce the repository checks, install the pinned Lean toolchain and run lake build, followed by python3 verification/check_current_parity.py from the repository root. The latter regenerates native output and performs the configured source-output and axiom checks. Source-specific completion reports retain the additional derivation and complete-program evidence.

4. Source and evidence links

V3 model: https://github.com/MettaMazza/ernos-labs-sft-platform/tree/76f1b34e9e9682989adcbb088797b4ae4ce4fcf1
Verified Lean checkpoint: https://github.com/MettaMazza/SFT-v4-LEAN-VERIFIED/tree/b8f732515dd06e8251c06b1c45e3fdfb858b986d
Alpha completion evidence: https://github.com/MettaMazza/SFT-v4-LEAN-VERIFIED/blob/b8f732515dd06e8251c06b1c45e3fdfb858b986d/verification/TURN_271_ALPHA_COMPLETION.json

5. Model and scope

The source model is Smithian Fold Theory V3, repository MettaMazza/ernos-labs-sft-platform, commit 76f1b34e9e9682989adcbb088797b4ae4ce4fcf1. Its census registers 2,781 model-admitted claims. The complete formalisation follows all 2,781 entries in dependency order, preserving their original definitions, derivations, declared constraints, controls and empirical boundaries. Each translated component is bound to its V3 source, and full-model verification requires both native proofs of the registered formal statements and correspondence of the associated executable procedures. Empirical records retain their original evidential status. The results presented here cover 109 completed entries: fifteen Foundation entries, twelve Mathematics entries, twelve Information Science entries and twelve Formal Computation entries and ten Computability entries and thirteen Computational Complexity entries and fourteen Algorithms entries and twelve Semantics entries and the first two Distributed Computation entries, Fold Concurrency and Computational Causality, plus the Physics Structural Generator Three, Spatial Dimension Three, Boundary Rank Two, Inverse Fine Structure, Charged-Lepton Cubic, Charged-Lepton Terminal and Electroweak Fold Mixing entries. Their finite-generation derivations and exact source operations are presented below. Full-entry verification remains distinct from the translated portions.

SFT distinguishes the structural One, generated positive finite counts and exact positive parts from host implementation indices. Its derivational constraints exclude added axioms, free or fitted scientific parameters, negative proof quantities, irrational or imaginary proof values, floating equality and completed infinity. Formalisation preserves the registered source boundary of each result.

6. Operational root

The registered operational challenge partition has exactly two classes: unpresented-absence and presented-occurrence. An unpresented challenge supplies no counterexample. A presented challenge supplies an occurrence and is therefore not nothing at the operational boundary.

The decision accepts precisely presented-occurrence. Exhaustive enumeration establishes that both registered classes occur in the candidate list. The unique-survivor theorem establishes that a candidate is accepted if and only if it is presented-occurrence. No premise about a particular presentation's internal content is introduced.

Within the registered partition, deleting the presented witness leaves no accepted candidate. The decision is definitionally the presentation predicate. Three additional native proofs establish the false-premise, tampered-artifact and fixed-boundary control outcomes. The source census, decisions, closure evidence and all four control records are emitted by native Lean definitions; every stored field and digest agrees with the pinned V3 artifacts.

The tampered-artifact premise is represented by an acceptance function that accepts both source classes. Its survivor set is exactly the original two-element candidate list. Constructive proofs establish that the two survivors are distinct and that no unique survivor exists. These statements formalise the source rejection condition while retaining the original control record for exact artifact correspondence.

The record-level formalisation retains exactly candidate identity, presentation flag and descriptive form from the source dictionary. Classification is derived from the presentation flag. For every record in this schema, its classified candidate has the same presentation status; classifying a generated representative returns that representative. Two records have the same class if and only if their presentation flags agree. Thus every such record maps into the registered exhaustive partition, independently of its identifier or descriptive text.

The source decision agrees with this classification for every record. Acceptance is equivalent to the presented-occurrence class and to the source operational presentation witness; an accepted record cannot be unpresented. Changing either metadata string changes neither acceptance nor its reason. Deleting presentation rejects the record. These statements formalise the source record decision and operational presentation relation, while preserving all three source fields and the generated evidence values.

The complete registered Root entry is formalised: its source definitions, operational derivation and closure statements, exhaustive candidate partition, four controls and program methods are represented. Fresh native execution reproduces the pinned census, decisions, closure, control records and their hashes; registration fields and refusal of unknown identifiers agree. The source program declares no prior dependencies, axioms or free scientific parameters. Fifteen foundation entries, twelve mathematics entries, twelve information-science entries and sixty-three computation entries and seven Physics entries establish one hundred and nine completed source entries out of 2,781; the 6,024 audited Lean theorems count individual results across the existing translated work.

Source claim: SFT-ROOT-THERE-IS-NO-NOTHING.

7. Structural One

A representation of the root occurrence is classified by its coverage—none, proper or complete—and by whether it adds unforced material. The product gives six classes: omitted-root, replacement-extra, fragmented-root, fragment-plus-extra, exact-self-whole and whole-plus-extra, in source enumeration order.

The representation survives precisely when its coverage is complete and it contains no extra material. Thus exact-self-whole is the unique surviving class. Omission, fragmentation and addition are rejected by the same ordered conditions as in V3. The One is structural self-wholeness, not a numerical magnitude introduced at this stage.

The record-level derivation uses the three source root-coverage relations and two source extra-coverage relations. Their generated product is proved identical, in order, to the six stored candidates. Classification retains both relations and is inverted by the generated representatives. The four source record fields, survival decision and ordered reason all agree with the generated records.

For every record in that relation domain, acceptance is equivalent to complete coverage together with no extra material, and hence to the exact-self-whole class. Replacing coverage by none or proper rejects the record; setting the extra relation to has-extra also rejects it. Acceptance and its reason are unchanged by arbitrary identifier or descriptive-text changes. These universal record statements formalise the source omission, fragmentation, addition and metadata-independence arguments without introducing a numerical magnitude.

The complete One source program preserves its registered dependency, all six ordered candidates, actual identifier-based decisions, closure and generality records, and four original controls. Native lookup correctness and the non-hash control outcomes are proved. Fresh execution matches every corresponding pinned field and digest, including refusal of an unknown identifier. Together with the general derivation above, this completes the registered entry.

Source claim: SFT-FOUNDATION-ONE-001. Registered dependency: the operational root.

8. Exact positive finite count

V3 defines positive finite count through the complete generation trace of a nonempty finite succession of structural-One occurrences. Every generated occurrence is retained once and no ungenerated occurrence is added.

Its representation grammar has ten classes. With no coverage, multiplicity is not applicable and the two extra-material classes remain. With proper or complete coverage, multiplicity is once or duplicated, independently paired with absent or present extra material. Only complete-coverage__once__no-extra survives. The formal classification retains this exact domain, including the absence of a multiplicity choice in the no-coverage case.

The native theorems establish complete enumeration and unique survival within this registered grammar. No replacement numerical count model is required for these results.

The base/successor argument is encoded as finite generation syntax: a structural-One base followed by extensions, each retaining prior occurrence positions and introducing a fresh terminal. Positions denote generated occurrence identities within the trace. They are not interpreted as a numerical magnitude. The complete representation is constructed by retaining the base and appending each newly generated terminal once.

Structural induction proves that every generated position occurs in the complete trace, that no position is repeated, and that the trace is nonempty. The previous complete trace is proper coverage of an extension: all its positions remain, but the fresh terminal is absent. Appending that terminal restores completeness. Repeating any generated position violates exact multiplicity; removing a position loses complete coverage. The two source complete-generation clauses determine a unique trace identity at every finite extension. Finite proposed representations are then classified by their retained generated positions and presence of ungenerated material. The latter is represented by the source outside-trace relation; extra-object content is not a criterion of the source classifier.

Constructive position equality and membership establish the semantic connection to the ten-class grammar. Complete coverage is equivalent to retaining every generated position. The multiplicity predicate is equivalent to absence of repeated retained positions. The extra-material predicate is equivalent to the presence of an ungenerated occurrence. For every finite proposal, survival is therefore equivalent to complete exact-once retention with no ungenerated material. Canonical complete proposals survive; any missing position, repeated retained position or outside addition is rejected. The previous complete prefix of an extension is rejected because it omits the new terminal.

Execution comparisons cover 493 proposals over one-, two- and three-occurrence traces, reaching all ten source classes and matching their decisions and reasons. These bounded comparisons supplement the universal Lean equivalence; they do not replace it.

The complete Count source program preserves its registered dependency, all ten ordered candidates, actual identifier-based decisions, closure and generality records, and four original controls. Native lookup correctness and the non-hash control outcomes are proved. Fresh execution matches every corresponding pinned field and digest, including refusal of an unknown identifier. Together with the general derivation above, this completes the registered entry.

Source claim: SFT-FOUNDATION-COUNT-001. Registered dependency: the structural One.

9. Exact positive part coordinate

The Part derivation registers a finite partition of the structural One and a held selection. Its seven relation domains classify partition coverage, overlap, fibre equality, selection relation, held-count presence, whole-count presence and unforced-extra-coordinate presence. Their source-ordered product contains 192 forms. A native theorem establishes that acceptance is equivalent to all seven requirements: complete, disjoint, equal-fibred, within-partition, both counts present and no extra coordinate. Exactly that form survives. Removing either count, omitting whole coverage, allowing overlap or unequal fibres, using an empty or outside selection, or adding an unforced coordinate is proved to fail.

For a registered whole’s finite generated fibre trace, a held predicate constructs the selected trace. Constructive proofs establish containment and absence of repeated fibres. A witnessed held fibre makes the selection nonempty. Its host-record length cannot exceed the whole trace length; this expresses the finite containment bound without installing a new magnitude in the model. The exact coordinate is the pair of held and whole generation records. Its projections retain those two identities, and the pair is unique from them. Both records are nonempty for a witnessed selection.

The complete Part source program retains the original One and Count dependencies, all 192 candidates, every decision payload, closure and generality certificate, and four controls. Fresh actual identifier-based execution reproduces every pinned field and digest and refuses unknown identifiers. The original boundary control retains the source statement that cross-partition equivalence belongs to a later derivation. Together with the general coordinate proofs, these results complete the registered Part entry.

Source claim: SFT-FOUNDATION-PART-001. Registered dependencies: structural One and Count. Equivalence of coordinates from different partitions is not assumed here; it is the subject of the separately registered common-refinement derivation.

10. Generated common-refinement pairing

The common-refinement construction generates cells by traversing each source A label and then each source B label. The selected A lift retains all B descendants of each held A label; the selected B lift retains all A origins of each held B label. Every generated cell has source origins, and each pair of source origins occurs in the construction. The executable label formatting and order are preserved exactly.

The registered generated-bijection procedure pairs finite host indices in order and checks equal trace lengths and bounds. Native proofs establish that the bounds always hold, that acceptance is equivalent to equal finite trace lengths, and that every index on both accepted sides occurs in the generated pairing. A constructive pairing supplies forward and inverse maps with both inverse identities; injectivity and onto coverage follow. Identity, reversal and chaining establish the registered pairing laws. Uniform refinement retains the shared descendant identity and preserves both inverse identities. For the concrete pair-cell procedure, equal selected-trace lengths remain equal after appending the same descendant trace by nested generation. This is proved structurally without assuming a multiplication law. The source one-of-two/two-of-four example accepts; the unequal source example rejects.

Source claim: SFT-FOUNDATION-PART-EQUIVALENCE-001. Dependencies: One, Count and Part. The exact nine-domain product has 1,024 forms. A constructive product-membership proof establishes exhaustive enumeration. The source identifier-equality decision is proved equivalent to all nine witness requirements; exactly the registered witness survives. Eleven adverse relation cases are proved rejected. Complete selected-origin lifting is proved contained in the common refinement, and

the accepted generated pairing is both one-to-one and onto on the lifted trace positions. The complete original program and its evidence records match the pinned V3 source outputs.

11. Minimal Count extension in Fold

Fold classifies a generated Count stage as identity, first extension or later extension. Structural proofs identify those classes exactly with the base, its immediate successor and a successor of a successor. The generated-stage prefix relation retains each earlier stage under further extension. Every nonidentity finite trace contains the first extension. The first extension is nonidentity and every nonidentity prefix of it is itself; it is therefore minimal. Conversely, any minimal nonidentity stage must equal that first extension. Identity and every later finite extension are consequently excluded as minimal Fold stages.

The first extension retains the base occurrence and the appended occurrence. Its generated-position domain is exhausted by these two distinct positions, and its complete record retains each once. This formalises the source first-extension count argument without introducing signed numerical labels, a fitted quantity or a maximum finite depth. The source equality, disjointness, whole-preservation, distinct-label and return relations are retained in the complete grammar and characterized by the stage acceptance theorem below.

The seven source relation domains generate 288 forms in the original order. The native decision retains the exact source conjunction and ordered explanations. Exhaustive membership, unique survival and acceptance iff all seven requirements are established. Nine adverse criterion values are rejected. A source-record adapter derives the extension field from the actual Count stage and preserves the other six fields. Its acceptance is equivalent to minimal nonidentity of that stage together with complete whole coverage, disjoint and equal fibres, distinct-held labels, present return and absence of extra data. Identity and every later stage are rejected for every choice of the remaining source fields.

The complete Fold program preserves all 288 original candidates and the One, Count and Part dependencies. Fresh native execution matches every census field, actual identifier-based decision, trace/proof digest, closure/generality certificate and original control result. Unknown identifiers are refused. These results accompany the general minimal-stage derivation and source-relation acceptance equivalence, completing the registered Fold entry.

Source claim: SFT-FOUNDATION-FOLD-001. Dependencies: One, Count and Part.

12. Complete finite Fold-word assembly and return

The assembly construction begins with the structural empty One word. Each successor appends held-a and held-b, in that order, to every preceding word. The nonnegative extension index is the source host recursion input; the empty word is a structural form. Negative host extension inputs are rejected.

Appending a held label retains both the predecessor word and the last label. The return relation recovers the predecessor by removing precisely the appended terminal label. Both distinct branches return to the same predecessor, and a successor has only one such predecessor. These properties hold at every finite generated depth. Appending different labels or extending different predecessor words produces distinct successor words; induction establishes uniqueness of every complete support.

A word belongs to the support after n extensions if and only if it has host-record length n and every label belongs to the source pair. This universal equivalence establishes completeness and absence of

additional words. Every retained successor has a return to the preceding support. An outside label or incorrect length excludes a proposed word; repeating a generated word violates uniqueness, and omitting one violates complete coverage. These statements are established by constructive finite-list and generation induction without an imported exponentiation law or a maximum depth.

Source claim: SFT-FOUNDATION-FOLD-ASSEMBLY-001. Dependencies: Count and Fold. The source keeps this recursive assembly claim separate from minimal Fold. Its seven relation domains produce 192 source-ordered forms. Exhaustive enumeration, equivalence of the original identifier decision with the seven criteria, unique survival and rejection of eight adverse values are established. The complete original program and its evidence records match the pinned V3 source outputs.

The canonical assembly retains every stage from the empty source word through the final finite depth. Each transition explicitly retains predecessor, appended label and successor. Its successor uniquely determines that transition. Every generated child at every earlier level has both its transition and return recorded in the full trace. Conversely, each stored stage, transition and return arises from the source construction; each return recovers its recorded predecessor.

A native proof certificate establishes the seven accepted relation requirements on this canonical construction: complete stage coverage, consistent lengths, complete support, word uniqueness, transition retention, return retention and absence of added records. The accepted source witness is realised at every positive finite depth. This result concerns the canonical construction. Proposed word supports additionally admit the executable semantic classification below; source coverage, transition and return relations remain independent inputs.

For an arbitrary finite proposed word support, executable membership and universal finite-list tests determine the source length, support-completeness, uniqueness and extra-word relations. A native theorem establishes acceptance exactly when the coverage relation is complete, every proposed word has the required length, every generated word is retained, no word is repeated, transitions and returns are present, every proposed word is generated and no extra rule is supplied. The coverage, transition and return fields are preserved as independent source relations; their truth is not inferred from a word list alone.

The source finite-set operations are represented constructively. Duplicate removal preserves precisely the original members and produces a duplicate-free list. Equality of the original length and this finite-set cardinality is equivalent to uniqueness. Two word sets compare equal exactly when their membership predicates agree. The original four assembly controls retain their predicates and serialized result hashes; the depth-one-through-five uniqueness/length check follows from the native all-finite theorem. The source literal boundary text check remains distinct from semantic exclusion results.

The finite assembly record representation includes predecessor–label–successor transitions and child–parent returns. Acceptance is equivalent to retention of every required transition and return, exclusion of ungenerated records, complete source coverage, correct word lengths, complete word support, word uniqueness and absence of extra words or rules. Canonical records satisfy these conditions. Missing required transitions, missing required returns and incorrect returns are rejected. Coverage remains the explicit source relation. Registration, census, decisions and closure are represented with the original source fields and ordering; candidate lookup uses the source identifier.

The exact casting operation acts on a positive count ratio n/d by removing one complete denominator while the numerator exceeds it. A structural recursion bound derived from n suffices for every positive d. A native removal-trace theorem proves termination and a unique retained numerator r with $0 < r \leq d$. Admitted numerators remain fixed; adding a complete denominator to a positive numerator does not change its retained value. Every positive whole number of complete turns retains d, representing the One. These results apply to all finite positive count pairs. The numerator-level result supplies the

casting stage of the normalized primitive operations below.

The Fraction admission map preserves the reduced numerator and denominator exactly. An admitted part maps back to the same Fraction, and admission yields that part if and only if the original Fraction is its reduced-pair representation. Zero, signed values and values above the One are rejected; every reduced positive pair through the One is admitted. For admitted a and b, the strict Take guard is the cross-product inequality between their count pairs. Its difference numerator is positive and strictly smaller than the product denominator; equal and reversed Take inputs are rejected. Self-junction followed by cast has a positive numerator no larger than its denominator, and a complete One is Fold-fixed. The normalized operations reduce their output count pairs to the canonical Fraction representation.

Canonical Fraction construction divides the numerator and denominator by their greatest common divisor. Constructive proofs identify the native GCD computation, establish its divisibility and greatest-common-divisor properties, and show that the reduced quotient pair is coprime. Both quotients are positive, reduction preserves exact cross-product value, and an admitted input remains through the One. Normalization is stable on an already reduced part. The integer-pair constructor retains the source rejection messages for signed, zero and beyond-One inputs.

Fold first reduces the self-junction Fraction, then casts complete Ones and reduces the retained part. Guarded Take constructs and reduces its exact positive difference. Both operations return reduced admitted parts. Unison normalizes the quotient and compares it to the One; a native theorem makes this equivalent to equality of the input cross-products. Every part is in unison with itself, and the complete One is Fold-fixed. The proof arguments apply to all finite positive count pairs and use no denominator bound.

The Exact Operations grammar contains the complete 128-form product of domain, full-turn representation, Fold composition, Take guard, unison identity, trace custody and added-data classes. Enumeration is exhaustive; the source identifier decision is equivalent to all seven required criteria, and exactly the registered survivor satisfies them. Each adverse class is rejected. The trace-custody relation remains an explicit source criterion.

The positive Fraction cast adapter rejects zero and signed inputs and returns the normalized cast result for every positive reduced input. The four original controls retain their predicates: reversed Take rejection, changed-source identity, casting two complete Ones to the One, and the four specified hostile carrier constructions. Their full output fields and hashes agree with the pinned source. The eight literal closure fields are preserved.

The first Fold has the two positions already established by minimal Count extension. Every held selection is empty, one of the two singletons, or the whole pair. Its nonempty coordinate classes are precisely one-of-two and One. Every proper nonempty selection therefore normalizes to half-One. Exchanging the two positions is involutive and preserves the selection cardinality and exact coordinate.

For every admitted reduced positive part, normalized self-junction is the One if and only if the part is half-One. The proof uses the full positive numerator/denominator domain: self-junction forces the denominator to be twice the numerator, while reducedness forces the numerator to be one. Half-One complements to itself and Folds to the One; the One is Fold-fixed. At half-One, complement and half-One phase translation have different values. These results formalize the registered complete-partition argument without a sampled rational search.

Source claim: SFT-FOUNDATION-HALF-ONE-001. Dependencies: minimal Fold, Exact Operations and Part Equivalence.

The Half-One candidate grammar is the complete 128-form product of selection class, self-junction, complement, Fold image, One fixedness, antipode terminology and added-data class. Native proofs establish exhaustive membership, equivalence of the source decision to its seven criteria, unique survival and rejection of every adverse class. The source whole-selection control witness is retained in the generated census. The executable self-junction-to-One predicate holds exactly at half-One by the general reduced-part uniqueness theorem.

All four Half-One controls retain the source predicates and serialized fields. The altered-coordinate control compares half-One and one-of-three; the boundary evaluates the actual complement and Fold operations. Current Lean output agrees with the four complete source control records, including hashes, and all nine literal closure fields.

Reduced positive Fractions are equal if and only if their cross-products are equal. Constructive remainder scaling and GCD scaling establish denominator identity, followed by numerator identity. This result connects exact count-pair equations to equality of the native normalized parts. Normalization represents any equal admitted ratio, and casting an admitted reduced part preserves it.

For an arbitrary admitted image y, the lower generated Fold preimage is the normalized ratio y/2. Its doubled numerator does not exceed its denominator, and native Fold of this preimage is exactly y. Phase translation is implemented as normalized addition of half-One followed by cast; the generated preimage pair retains the source lower and translated-upper definitions. The executable uniformity predicate is connected to the arbitrary-count division proofs below.

The exact orbit one-of-three to two-of-three and back to one-of-three is proved using the native Fold operation. Its two values are distinct. Two steps return to the source, while every positive step count below two fails to return. Source claim: SFT-FOUNDATION-FOLD-DYNAMICS-001; dependencies: Exact Operations and Half-One.

Positive scaling of a count-pair representation scales its complete-One removal trace and retained numerator. Normalization commutes with casting, yielding an exact raw count-pair formula for Fold of every normalized part. These identities preserve the original operation while allowing its arbitrary-rational branch equations to be proved.

For a part in the first half-circle, half-One phase translation has the exact normalized coordinate (twice the numerator plus the denominator)/(twice the denominator). Doubling this translated coordinate adds one complete One before cast, so it shares the original Fold image. Applying this result to the generated lower preimage proves that the generated upper preimage also Folds to every admitted image. The upper lies strictly beyond the half-circle boundary, while the lower lies on or below it; hence the two generated preimages are distinct. This establishes validity and distinctness of the generated pair. Every preimage is one of these two forms: a first-half part is recovered as the lower preimage of its Fold image, while a second-half part translates to the first half and is recovered using phase involution. Thus, for every admitted p and y, Fold(p) = y if and only if p is y/2 or cast(y/2 plus half-One). Distinctness and this exhaustive equivalence establish the exact two-preimage law over the entire positive rational circle.

The phase laws hold throughout the positive rational circle, including both half-circle branches and One. Complete-One removal is preserved under the affine half-One shift and under doubling. Consequently Fold(phase(x)) = Fold(x) and phase(phase(x)) = x for every admitted part x. The proofs retain the source normalization and cast order and remove the complete One introduced by two successive half-One translations.

For every positive n, the source uniform division is represented by the ordered normalized parts i/n with indices from one through n. The native enumeration retains every generated index. For every i

from one through n, Fold maps both i/(2n) and (i+n)/(2n) to i/n. Conversely, every index in the even division belongs to one of these branches. Therefore a part occurs among the Fold images of the complete 2n-division if and only if it occurs in the n-division. Distinct indices give distinct normalized coordinates, and the terminal n/n coordinate is One. These arbitrary-count results establish the exact image membership before sorting and gap evaluation.

Exact normalized rational ordering agrees with the original cross-product comparison. Consequently the complete n-division is strictly increasing, has no repeated parts and contains exactly n entries. For each adjacent generated index i,i+1, native Take of i/n from (i+1)/n is exactly the normalized part 1/n. This is established by factoring the reduced numerators and denominators through their GCDs and preserving the exact rational difference. At n=1 the generated list is exactly [One]. Together with terminal One and the even-division image correspondence, these results establish the order, cardinality and gap properties of the uniform image domain.

The sorted distinct Fold image is implemented over the exact reduced-rational carrier. Cross-product identity removes repeated values, and cross-product order determines the ascending output. Native proofs establish input membership preservation and strict output ordering for every finite input list. Two strictly ordered rational lists with the same members are equal. Applying this uniqueness result to the complete even division proves the executable equality folded_distinct(uniform_division(2n)) = uniform_division(n) for every positive n. The resulting list has exactly n entries; no generated input index is omitted before Fold and duplicate-image identification.

The native uniformity predicate preserves signed adjacent differences: positive and negative differences carry their exact normalized magnitudes, and equal neighboring parts give zero. Equality of these gap values is decidable and is proved equivalent to equality of their representations. The singleton-gap-set check holds precisely when a gap occurs and every gap in the list equals it. Empty lists fail; singleton lists pass exactly at One. Longer lists require one distinct signed gap and terminal One, matching the source predicate.

Complete generated-prefix witnesses establish equal positive gaps and the final coordinate for every prefix. Consequently the executable predicate accepts uniform_division(n) for every positive n and accepts folded_distinct(uniform_division(2n)) for every positive n. Separate fresh execution comparisons agree with exact Fraction evaluation on 156 lists covering lengths zero through three and positive, negative, zero and mixed gaps.

The Fold Dynamics candidate grammar retains the complete 128-form product of domain closure, fibre cardinality, phase collision, phase involution, uniform-division preservation, first cycle and extra-data class. The source identifier decision is equivalent to all seven stated criteria, and exactly the registered survivor satisfies them. Native proofs establish complete enumeration and rejection of every adverse axis value.

The Fold Dynamics controls retain the original 35 rational images generated for denominators two through eight and all six even divisions two through twelve. The two-preimage and phase control predicates hold for every admitted image by the arbitrary-domain laws; their finite executions preserve the source enumeration. The uniformity and first-cycle boundary check follows from the arbitrary even-division theorem and exact one-third/two-thirds return. All four serialized control records, including receipt hashes, and all eight closure fields agree with the pinned V3 artifacts and source literals.

The Fold Dynamics program adapter preserves the original registration fields, complete census, candidate lookup, decision proof hashes and depth-independent closure evidence. Fresh execution agrees with the pinned artifacts for all 128 lookup decisions and every census, closure and control field and hash. Candidate lookup depends only on the original identifier, and an unregistered identifier is rejected.

The next foundational construction is the finite Form grammar. Its two productions are the structural One leaf and a Fold node with arbitrary recursively generated left and right subforms, distinct held-a/held-b edges and the explicit return marker. The native tuple encoding preserves these source constructors and is injective. Structural induction applies to every generated finite form; each form has a positive number of One leaves, with leaf count equal to Fold-node count plus one. The five original examples include distinct left-deep and right-deep nonuniform forms. The raw source validator is translated on the finite tuple/string representation described below.

On finite nested tuple/string values, native form validation preserves the source predicate and its evaluation order. The One tuple is accepted; a Fold tuple requires the original tag, labels and return marker before recursive child validation. Every encoded generated Form is accepted by structural induction, including the original nonuniform examples. One-child and same-label nodes are rejected. The source accesses only the first two items of each edge tuple, so trailing edge items remain ignored. Missing indexed items retain their IndexError outcomes, while earlier false conditions suppress later accesses. Fresh execution agrees with the explicit source predicate on 235 tuple/string cases. For every represented finite tuple/string value, successful validation is equivalent to a finite production trace. The forward direction follows structural recursion on the value; the reverse direction follows induction on the trace. Each production retains the original Boolean tag, edge-label and return tests, including arbitrary ignored trailing edge items. Thus this characterization preserves the source acceptance predicate rather than imposing canonical two-item edges. Other Python object kinds remain outside the current representation.

The registered Form Grammar classifies the complete product of two base choices, three child arities, three edge-label relations, two child-provenance choices, two return choices, two termination choices and two extra-constructor choices. All 288 proposals are enumerated in source order. Native proofs establish exhaustive membership and exactly one accepted proposal: the One base with two distinctly held-labelled generated children, return, finite-leaf termination and no additional constructor. Each of the nine adverse field values is rejected. Fresh execution preserves every original candidate identifier, exact form, acceptance and ordered reason. All four original control records and their receipt hashes agree with the pinned artifacts; the seven closure fields agree with the source. The original boundary control retains its literal text predicate; finite production-trace results are separately established by the structural proofs above.

Canonical form enforcement assigns the One leaf its terminal One tuple and each Fold node the ordered tuple Fold, held-a, left canonical trace, held-b, right canonical trace, return. Structural induction proves this map injective on all generated finite Forms. Canonical-trace equivalence is therefore precisely form identity and is reflexive, symmetric and transitive. Equality of Fold traces preserves the two corresponding child traces separately. The source partial trace and asymmetric child swap are rejected. An executable comparison on generated Forms is proved equivalent to canonical-trace equality.

On the represented finite tuple/string domain, the raw canonicalizer validates before constructing the trace, propagates validation errors and preserves ignored edge tails. Every generated input produces its stated canonical trace. More generally, every accepted raw value produces the canonical trace of a generated Form. Fresh execution matches 235 explicit source-predicate trace and error cases. The complete Form Enforcement product contains 192 identity proposals; exhaustive membership, unique acceptance and each adverse-condition rejection are established. All candidate fields and decisions and the four original controls with their receipt hashes match the pinned artifacts. Other Python object kinds and full program integration remain outside the results established here.

Raw form equivalence canonicalizes the left input before the right and propagates the first encountered error. A constructive recursive equality decision on finite tuple/string values is proved equivalent to their exact equality. Consequently, two generated inputs compare successfully as

equivalent exactly when the generated Forms are identical. For arbitrary represented inputs with successful canonicalization, equivalence is precisely equality of the resulting traces. Fresh execution agrees with the source formulas on 2,352 raw input pairs, including malformed forms, ignored trailing edge items and error precedence.

The measured-value boundary preserves the source Boolean protocol: acceptance is equivalent to opening the target after sealing, leaving the law unchanged, retaining all rows and identifying the source. Each violated condition forces rejection regardless of the other flags. All 16 Boolean inputs agree with the source conjunction. The complete seven-axis interface product has 256 proposals, with one surviving outward interface and nine independently rejected adverse field values. Every proposal's order, identifier, exact form, decision and reason agrees with the pinned records. All four original control records and their hashes and all eight closure fields also agree. These results establish the Boolean protocol and finite interface classification; non-Boolean Python truthiness inputs and full phase/capability integration are not included in this result.

Exact Arithmetic represents an atom as a finite string-label path and a trace as a finite sequence of atoms. The trace constructor rejects empty or repeated label inputs. Disjoint junction concatenates nonempty traces only when they have no shared atom. Pair-cell product retains every left/right pair in the source order and flattens the two atom paths by concatenation. Native induction establishes associativity of this flattening at arbitrary finite trace lengths and distribution over concatenation in the left input. Its complete output cardinality is the product of the two input lengths, and positive input traces produce a nonempty result.

Exact pairing returns the source zip precisely for equal input lengths, including its specified empty-input result. Oriented remainder retains unmatched atoms in source order and reports incomparable-held, left-held, right-held, or same-form with the original empty-One representation. No additional distinctness condition is imposed on raw operation inputs. Fresh execution agrees with all five source operations on 81 pairs covering empty, disjoint, overlapping, repeated and nested-path inputs, together with eight trace-constructor cases. Remaining registered arithmetic laws and full generated-law evidence integration are separate from these established operation results.

Exact pairing preserves both complete input sequences: projecting every pair onto its first coordinate recovers the left trace in order, and projecting onto its second recovers the right trace in order. Both output cardinalities equal the input lengths. A returned pairing exists exactly when those lengths agree, and the ordered pairing is unique. These results apply to arbitrary finite source traces, including duplicate atoms and the specified empty pairing, without replacing occurrences by sets.

The Exact Arithmetic LawSpec contributes eight binary representation dimensions. Their complete 256-candidate product is enumerated in source order. Native proofs establish membership for every candidate, equivalence between identifier acceptance and all eight admitted choices, unique acceptance and rejection of each adverse coordinate. Fresh execution matches all candidate identifiers, coordinate-form strings, decisions and ordered rejection reasons from the pinned generated-law artifacts. This finite classification accompanies the operation proofs; full registered-law and evidence integration remain separately tracked.

Right-distribution of pair-cell refinement preserves the complete generated output as a permutation. The proof constructs finite adjacent swaps and block interchanges, retaining every occurrence and its multiplicity even when flattened atom paths coincide. This relation accounts for the source row-major traversal order. Together with ordered left-distribution and flattened associativity, it establishes both distributive directions by cell provenance. The five original arithmetic witnesses—junction associativity, six-cell cardinality, flattened product associativity, exact pairing and held remainder—are evaluated by native operations and proved passing, with their exact source names, statements and results retained.

The complete native Form Grammar program output agrees with the pinned artifacts: registration fields, all 288 actual identifier lookups and their decision proof hashes, every census trace hash, closure, controls and unknown-identifier rejection. This runtime correspondence supplements the general finite Form proofs; full engine admission integration remains separately tracked.

The oriented-remainder output is nonempty for every represented finite trace pair. Its same-form tag holds precisely when both source unmatched filters are empty; the corresponding output is the original empty-One trace. This characterization retains the source predicate on raw inputs, including duplicates, without replacing it by ordered trace equality.

The arithmetic closure retains the registered exact result, laws, operational witnesses, minimality, named uniqueness, induction base and step, and exclusions. The original tampered-artifact control performs both complete enumerations: survivor filtering and distinct identifier counting. Fresh execution yields one survivor and 256 distinct identifiers; all four control records and receipt hashes and all eight closure fields match the pinned source artifacts.

The native Form Enforcement, Measurement Boundary and Exact Arithmetic programs preserve the registered program-method outputs in full. Fresh execution agrees with all 704 candidate lookups across these three programs, every census trace and decision proof hash, all registration fields and provenance classes, closure and generality evidence, and control records. Unknown identifiers retain the source failure: KeyError for the two foundation programs and StopIteration for the generated mathematics program. Arithmetic generality serialization retains the original witness dataclass fields. These correspondences concern the program methods; full engine admission integration remains separately tracked.

13. Formal representation and results

Each Lean candidate constructor denotes exactly one registered V3 candidate record. These constructors are encodings of the existing finite grammar. Field projections retain source text and field values. Decision functions implement the original branch predicates and rejection order.

For each translated grammar, the formalisation proves:

(a) Every candidate belongs to the enumerated domain.
(b) Acceptance is equivalent to equality with the source survivor.
(c) The source survivor is accepted.

The proofs use constructor elimination, reflexivity and explicit list membership. Lean reports no axiom dependencies for all 991 theorems. These results establish the registered finite classifications; they do not stand in for the remaining source closure arguments, control records or admission procedures.

On the annotated registration domain, passing also entails empty axiom and free-parameter lists and no missing dependencies under the source authority lookup. The typed Root registration serializes identically to its source record. Registration diagnostic order is checked on 272 cases.

The admission engine’s forcing gate preserves four ordered requirements: decision identifiers match the generated census in order; exactly one decision survives; every decision provides an explicit reason; every proof identity has canonical SHA-256 syntax. On the source-typed Candidate and CandidateDecision domains, a constructive theorem establishes that an empty violation list implies a survivor list of length one. The original Root decisions have one survivor, while marking both decisions as surviving produces two and is proved to be rejected by this gate. This connects the source tampering premise to the engine’s actual rejection condition.

Closure evidence retains its source scope, exact boundary, minimality and uniqueness flags, proof identity and optional generality identity. Within the source-typed field domains, passing the closure

gate requires a nonblank boundary and successful minimality and uniqueness flags; depth-independent closure also requires a canonical SHA-256 generality identity. The control gate retains the required-kind, duplicate-kind, outcome, behavior-text and receipt-identity checks in source order. Passing implies that every supplied control has passed. Root closure with minimality removed or with its generality identity absent, and Root controls with failed status flags, are constructively proved to be rejected. These are engine-gate properties; they remain distinct from proving the complete operational-boundary semantics.

The candidate census retains the source generation rule, grammar boundary, signed expected cardinality, completeness identity and ordered candidate records. The census gate preserves the eight source checks and their diagnostic order. Passing requires nonblank generation and boundary text, a positive expected cardinality, matching candidate count and no duplicated identifiers. A constructive witness expresses each passing expected cardinality as a natural successor. Root censuses with an incorrect registered count or duplicated candidate are proved rejected. Executable correspondence covers 1,440 census variants, including negative and zero counts, Unicode identifiers, duplicated identifiers, blank forms and malformed identities, together with three direct Root cases.

14. Correspondence and reproducibility

The Lean implementation uses leanprover/lean4:v4.35.0. The build command is lake build. Audit.lean reports the dependencies of each theorem. Parity.lean emits the candidate records and their computed decisions. Each verification invocation rebuilds the current Lean source, generates a declaration audit and executes the current parity program. A read-only artifact comparison checks the complete ordered output against the pinned V3 census and elimination records: 47,206 of 47,206 rows agree on candidate identifier, exact form, decision and reason.

The public formalisation repository is https://github.com/MettaMazza/SFT-v4-LEAN-VERIFIED.

Canonical JSON uses the source ASCII-escaping, compact-separator and sorted-key rules on the Root evidence domain. SHA-256 is implemented constructively with its fixed algorithm constants. All 13,133 audited explicit definitions and theorems have no axiom dependencies. Sixteen byte-message checks and ten JSON checks agree with Python standard-library results; these execution checks are separate from a general proof of algorithm correspondence, which remains outstanding. The fixed Root boundary is ASCII, so its lowercase and substring operations preserve the source predicate exactly. These host operations introduce no free scientific parameters.

The typed forcing-gate implementation is additionally checked on 384 combinations of identifier order and cardinality, survivor flags, reason content and hash syntax. The Unicode whitespace predicate agrees with the host language over all 1,114,112 code points.

Closure/control correspondence is checked on 432 closure variants, 576 control variants and five direct Root cases. The typed control adapter has a native equality proof preserving the serialized source control fields.

The four Fold control outputs, including receipt hashes, and the seven literal closure fields agree with the pinned source. Assembly execution checks cover depths zero through six, four arbitrary prior-word inputs and six signed host indices; the native completeness, uniqueness and return proofs apply to every finite depth. The source literal boundary text predicate is preserved separately from the native exclusion of words containing labels outside the held pair.

Fresh execution comparisons cover 9,240 proposed-support/source-relation combinations and 385 finite-set inputs. The former reach 144 grammar classes; the complete 192-form assembly grammar is verified separately. All four assembly control records, receipt hashes and seven closure fields agree

with the pinned V3 artifacts.

Complete Assembly program outputs agree with the pinned source registration, census, all 192 identifier lookup decisions, closure and four controls, including their hashes. An unregistered identifier is rejected.

The machine-readable correspondence lists the V3 source function and source hash for every scientific declaration. The comparison script does not execute or import the V3 scientific modules and is not part of the Lean model.

15. Extension of the formalisation

The remaining foundation content includes source closure arguments, controls and admission artifacts beyond the finite classifications proved above. Subsequent results follow the existing dependencies into common-refinement equivalence, Fold and the downstream model. The scientific content remains that of V3.

16. Source

Maria Smith. Smithian Fold Theory V3. https://github.com/MettaMazza/ernos-labs-sft-platform/tree/76f1b34e9e9682989adcbb088797b4ae4ce4fcf1

17. Discrete collections and held selections

For finite tuple-of-string forms, generated collections retain their exact input order and reject an empty collection or repeated forms. A successful held selection retains only members of its source collection; the native membership procedure is equivalent to list membership. An empty held selection returns the structural empty-One form, whereas a nonempty selection returns the tuple of held forms. Generated relations retain their accepted pair sequence within the complete Cartesian pair support. Fresh successor construction appends the new form and rejects an already retained member. The total-single-valued predicate counts exactly one outgoing pair for each source member and is true on an empty source. The implementation preserves these source guards, including the distinction between an empty collection and a collection containing an empty form. These results concern the discrete operations; the registered candidate grammar and the remaining general discrete laws are separately tracked.

For arbitrary finite source collections, membership of an ordered pair in complete pair support is equivalent to membership of its two endpoints in their respective collections. Every pair in a successful generated relation therefore has an origin in the left collection and a target in the right. The total-single-valued predicate is equivalent to an exact singleton condition on the retained outgoing pair list for every source member; repeated pairs remain counted as repeated occurrences. Fresh succession preserves collection uniqueness, and appending a fresh form to an accepted generated collection yields another accepted generated collection. These proofs apply at arbitrary finite size by structural induction, without a depth cutoff.

The discrete candidate grammar enumerates the complete product of eight binary questions: collection identity, canonical identity, membership, pair-support relations, map law, generation order, induction closure and additional constructors. All 256 proposals are represented. Exhaustive membership and unique-survivor proofs establish exactly the source all-admitted coordinate, with each rejected coordinate incompatible with acceptance. Every source identifier, exact form, decision and ordered rejection reason agrees under fresh execution. The six original operational witnesses retain their source constructions and predicates. The closure record and all four original control records agree with pinned V3, including one actual enumerated survivor and 256 distinct candidate

identifiers. These results preserve the registered finite grammar and supplement the general collection, relation and succession proofs.

18. Finite collection induction

The accepted domain of the discrete collection operation is exactly characterized by finite construction traces with a singleton base and freshness-preserving append steps. Every such construction yields a nonempty distinct collection accepted by the source operation, and every accepted collection has such a construction. Consequently, a property established for every singleton and preserved by each fresh append holds for every accepted collection, at arbitrary finite length. The converse decomposition recovers both uniqueness of the preceding collection and freshness of the last member. These results concern the exact tuple-of-string operational domain and introduce no additional scientific constructors or free parameters.

19. Complementary discrete relations

For accepted generated source collections, the complete Cartesian pair support contains each pair once. Holding precisely the support pairs absent from a given relation produces its complement within that same support, preserving support order. The complement satisfies the original generated-relation acceptance guard. Its membership is equivalent to source-endpoint membership together with absence from the held relation; it is disjoint from the held relation, and the two cover the complete support. For an accepted relation, taking the complement twice restores its membership. This is a membership result: the source permits arbitrary held-pair order, which need not equal the canonical support order.

20. Composition of discrete maps

Relation composition retains each pair of linked occurrences through their equal intermediate endpoint. Every composed pair has witnesses in both input relations, and every linked pair is retained. For a total single-valued first relation whose targets belong to the second relation’s source, and a total single-valued second relation, the outgoing composed pair list for every original source member is a singleton. Thus composition preserves the original exact one-image predicate. When both input relations satisfy their generated-relation guards on the shared interface, the composed relation also satisfies that guard on the first source and final target: its endpoints are supported and its pairs are unique. These results preserve finite occurrence counts and require no fitted values or additional scientific parameters.

The native discrete program adapter preserves the registered program interface and its complete outputs. Fresh execution agrees with all 256 actual candidate lookups, every candidate trace and decision proof hash, the census completeness certificate, registration fields, closure and generality hashes, and all four control records. Lookup is determined by the candidate identifier, and an unknown identifier retains the source StopIteration failure. Generality serialization retains the original witness fields. These program-method correspondences supplement the native discrete derivations; full engine admission and dependency integration remain separately tracked.

21. Primitive Fold maps and ordered composition

The declared primitive self-maps are identity, square, constant One and Fold on reduced positive exact parts through One. Native evaluation retains each original operation, including exact rational normalization for square, and executes arbitrary finite words from left to right. Concatenating two words is equivalent to executing the first followed by the second. Typed primitive words and their original string names have identical execution; unknown names retain the source ValueError. Identity is injective and fixes every input; constant One collapses every input to the fixed One. Squaring preserves its exact rational value and the admitted part domain. Fold is noninjective because half-One and One share an image, and has a nonstatic recurrent orbit through one-third and two-thirds. These results supply the primitive operational derivations; the complete least-size uniqueness argument remains separately tracked.

The square primitive is injective on the entire admitted exact-part domain. Equality of normalized squared parts implies equality of the original squared cross-products, hence equality of their positive cross-values and of the reduced parts. For every proper part, its square is strictly smaller in exact cross-product order and remains proper. Structural induction then establishes strict contraction at every positive finite iteration count. A positive-time return under square occurs if and only if the input is the fixed One, excluding nonstatic recurrent square orbits throughout the declared domain.

22. Least-size primitive generator

Within the declared four-primitive ordered-word grammar, generation is represented by noninjectivity together with nonstatic finite recurrence. The native behavior proofs exclude identity, square and constant One and establish that Fold generates. Fold is therefore the unique least-size generating word. This result quantifies over every finite word, beyond the executed size-three prefix. Appending a primitive increases construction size by one and preserves the nonminimality of every later word. The complete executed prefix contains all 84 words of sizes one, two and three; each source acceptance decision is equivalent to the derived least-generator predicate. Fresh execution preserves every word, one-based rank, identifier, exact form, decision and reason. The source explicitly restricts uniqueness to the declared normalized grammar and generator predicate. Its local normalization reductions are established on the exact operational domain; unrestricted expression-language uniqueness is not asserted.

The native primitive-map program preserves all 84 actual candidate lookups, complete census and trace hashes, decision proof hashes, registration fields, closure and positive-word-size generality certificate, and original control records. The artifact control checks the full record count, actual survivor count and fourth-record identifier, and its passing result has a zero-axiom proof. Exact preimage and recurrent-orbit controls retain their original computations. Unknown candidate identifiers retain the source KeyError failure. Fresh complete program execution matches the pinned V3 artifacts.

The stated primitive normalization reductions hold for the native exact operations. Multiplication by One on either side preserves every admitted part; self-product is square and casting self-junction is Fold. Any positive Fraction whose numerator is a positive integral multiple of its denominator casts to the One. Raw self-junction is not a total self-map of admitted parts, since its value at One exceeds One. Guarded removal rejects equal operands for every admitted part and therefore is not total. These results retain the source’s declared normalized grammar and operation guards.

23. Ordered exact derivation traces

A derivation row retains its operation name, ordered input identities, exact output and output identity. The native replay function starts at one-of-three and executes the registered Fold or guarded Take operation in source order. Each result must match both the recorded exact output and its canonical SHA256 identity. Unknown operations and mismatched outputs retain the original rejection behavior.

Constructive induction establishes that every successfully constructed finite operation sequence replays to its computed terminal value. If a recorded prefix replays to a given exact part, appending a row constructed from the next accepted operation extends replay to that operation’s exact result. Concatenation preserves replay through the intermediate terminal. These theorems have no finite-length cutoff and add no scientific parameters or axioms. They formalise the base/successor replay component of SFT-FOUNDATION-DERIVATION-TRACE-001. The registered source dependencies and complete executable evidence are verified alongside these general replay results.

The concrete three-move derivation computes Fold(one-of-three), Take(One, two-of-three), and Fold(one-of-three), retaining the exact outputs two-of-three, one-of-three, and two-of-three. Its terminal agrees with the independent direct Fold route. All ordered input identities and exact output identities are retained. Replacing the final operation with Select is rejected as unregistered. Changing the final exact output to one-of-three while preserving its other fields is rejected by replay. Native proofs establish these outcomes and the corresponding boundary control. Fresh native execution agrees with the source artifacts for all four original control records and their receipt hashes, and with independent exact arithmetic and hashing for every row and mutation.

The derivation-record grammar is the complete product of eight source axes: source custody, dependency coverage, step order, intermediate exactness, operation registration, replay equality, terminal identity and added-data status. Its 256 records retain source enumeration order, including no-extra before has-extra on the final axis. Exhaustive membership and cardinality are proved, and acceptance is equivalent to the conjunction of the eight admitted criteria. The unique survivor is the complete source-bound, dependency-complete, ordered, exact, registered, identically replayed, terminal-bound record without extra data. Each rejected axis excludes acceptance.

Fresh native program execution reproduces every candidate, every actual identifier lookup and the complete census, trace, decision, closure and generality digests. Registration, original control receipts and unknown-identifier behavior also agree with pinned V3. These classifier and program-correspondence results accompany the independent native operational replay and finite successor proofs; full engine/dependency admission remains separately tracked.

24. Finite combinatorial selections

Selections are generated by retaining the previous ordered family and appending an ordered copy with the next carrier label held at the end of each trace. A native finite construction relation records the absent and held choices at each successor. Structural induction proves that this relation describes exactly the generated raw traces, that every retained label belongs to the carrier, and that fresh-member absent and held branches are disjoint. For a unique carrier, raw selection traces are unique at every finite length. The output-trace count at a successor is the sum of the two generated branch lengths; no closed-form counting formula is introduced.

The executable output conversion replaces the empty trace with the source singleton empty-One label and preserves trace count. Selection input guards and fresh-extension guards retain their exact domains and errors. The original three-label example and its fresh-extension construction are proved in source order. Fresh Lean comparisons cover 341 selection inputs and 32 recurrence inputs, including the literal empty-One encoding. The uniqueness theorem concerns pre-conversion choice

traces; connection to canonical carrier admission and the remaining combinatorial generators is tracked separately.

25. Finite arrangements by held choice

The native arrangement generator retains the source carrier order: each available label is held in turn, all occurrences of that held label are removed from the recursive carrier, and the held label is prepended to every resulting suffix. Nonempty unique carriers retain the source admission guard. On this domain, each remainder removes exactly one label, remains unique, and is nonempty whenever the parent has at least two members. Recursion is justified by strictly decreasing carrier length, without a depth cutoff or an imported counting formula.

Well-founded induction proves that every generated arrangement is a permutation of the unique source carrier. Thus every output retains all original labels with their multiplicities and introduces none. Fresh Lean execution matches independently generated permutations in exact order for 64 admitted carriers, comprising 748 arrangement rows, within 341 input cases including rejected empty and repeated-label carriers. The unbounded output-permutation theorem is distinct from these finite execution checks. Converse enumeration completeness and uniqueness of the complete output family are established for every nonempty unique finite carrier.

26. Arrangement completeness and uniqueness

Every permutation of a nonempty unique carrier is generated by the original arrangement function. The proof takes the first label of a proposed permutation, establishes its occurrence in the carrier, and identifies the suffix as a permutation of the exact held-label remainder. Well-founded induction then supplies the recursive suffix, which the existing held-choice branch prepends in source order. Together with output preservation, this proves membership in the generated family if and only if a form permutes the carrier.

Each held-label branch is duplicate-free by recursive uniqueness and injectivity of prepending a fixed label. Different branches have distinct first labels and are disjoint. Their source-ordered concatenation is therefore duplicate-free. The combined theorem applies directly to every successful call of the guarded arrangements API: its output contains every carrier permutation exactly once. The result is uniform in finite carrier size and uses no counting formula or depth bound.

27. Disjoint class accounting

For the classifier’s resulting text labels, the class ledger retains keys in first-occurrence order and appends each input occurrence to its matching class. Empty input and repeated items retain their original behavior. Native proofs establish that class insertion and complete ledger construction preserve the input family as a permutation, hence preserve every occurrence and the total count. Class keys are unique.

Lookup of any class equals the stable filter of the input by that class label. This equality preserves both multiplicity and within-class order, and implies that different labels have disjoint classes. These results hold for arbitrary finite inputs and classification functions at the source text-label boundary. Fresh Lean execution matches independent first-occurrence ordering and per-class filtering in 1,026 cases, including identity, constant and the original a/other classification witness.

28. Combinatorial witnesses and complete grammar

The five registered operational witnesses are retained: six unique ordered arrangements of three labels; exact carrier coverage of each arrangement; eight distinct selection forms including the empty One; the original a/other class ledger; and the fresh-member extension count. Each witness is evaluated from the translated operations and has a native proof. The six arrangement rows are derived through the original recursive equations in exact source order.

The combinatorial candidate grammar contains all 256 products of carrier coverage, choice rule, provenance, duplication policy, symmetry identity, class accounting, recurrence closure and extra-formula status. Native proofs establish exhaustive membership, cardinality, acceptance exactly at the all-admitted coordinate, and rejection of each inadmissible coordinate. Fresh native program execution reproduces all candidate lookups, source records, census and trace identities, decision proofs, closure and generality certificates, registration fields and four original control receipts. The general operational derivations remain distinct from these finite grammar and program-correspondence results; encoded canonical-domain and full engine/dependency integration remain separately tracked.

29. Selection encoding and extension

For a raw selection form, source extension commutes with empty-form encoding exactly when that raw form differs from the singleton literal empty-One marker. A native equivalence proves both directions. For distinct admitted carriers without that literal marker, encoding preserves distinct selection forms. With a fresh label different from the marker, the original recurrence operation on encoded selections equals the original selections operation on the carrier extended by that label. These conditions are explicit theorem premises; the executable input guards remain those of v3. The source accepts the literal marker as a label, and its singleton output is proved to contain two identical encoded forms. Connecting the conditional general recurrence result to the canonical operational domain remains pending.

30. Arrangement successor partition and provenance

For every nonempty distinct finite carrier and every fresh label, a form belongs to the original successor arrangement family if and only if it is obtained by inserting the fresh label between a prefix and suffix of a prior generated arrangement. The proof removes the fresh label from the successor carrier, identifies the remaining form through permutation completeness, and reconstructs every inserted form through the same original generator. The prefix and suffix are uniquely determined: freshness prevents the inserted label from being mistaken for any prior occurrence. Thus the successor family is exhausted by disjoint insertion-position classes with retained prior construction provenance. This result is uniform over finite carrier size and leaves the source-ordered enumeration unchanged.

31. Arrangement trace accounting and arithmetic linkage

The count of the arrangement family is obtained directly from its ordered held-choice branches. A singleton carrier generates one arrangement. At each larger carrier, the length of the complete generated trace equals the ordered sum of the generated remainder-family lengths; prepending the held label preserves each branch length. This equation is derived from the enumerated trace and does not replace its generation with a closed-form answer.

For every admitted carrier of at least two labels, the first held-choice branch and the concatenation of the remaining branches are both nonempty and disjoint. Applying the existing exact-arithmetic

disjoint junction to these actual traces succeeds under its original guards and returns the original complete arrangement family. This connects combinatorial branch accounting directly to the translated arithmetic operation while preserving source order and every generated form.

32. Complete encoded selection output

For every successful call to the original selections operation, output membership is equivalent to the existence of a raw absent/held construction whose empty-form encoding equals the returned form. This equivalence covers the full string-label input domain of the source API. Every such output is a positive trace and retains the structural empty One, derived by choosing the absent branch at each carrier extension.

If the carrier omits the literal empty-One label, each encoded result has a unique raw-form preimage and the output is duplicate-free. These uniqueness conclusions retain their explicit premise; the full-domain membership and completeness theorem requires no such exclusion. The preimage result identifies raw forms, without asserting equality of proof terms or erasing the source construction order.

33. Discrete collection integration

Every arrangement family over a nonempty distinct carrier satisfies the original discrete generated-collection operation and has its finite singleton/successor construction trace. Each held-choice branch is a nonempty, duplicate-free subset of the complete family, so the original held-selection operation accepts it and retains its exact tuple representation.

The encoded selection family likewise has a generated-collection construction trace when the carrier is distinct and omits the literal empty-One label. The concrete composition on the singleton literal-marker carrier is also established: selections returns two equal encoded forms, and generated_collection rejects that result through its existing duplicate guard. The translated operations preserve this source behavior. No unconditional collection-admission conclusion is inferred for all encoded selections.

34. Ordered selection provenance

A raw form is generated by the absent/held recurrence if and only if it is an order-preserving subsequence of the carrier. Both directions are established constructively: every recurrence step preserves source order, while every ordered choice can be built by retaining or skipping carrier positions. Encoding this relation gives the exact membership characterization of the returned selection family, including the empty-One conversion.

Every held successor is the existing Fold word-append operation on the prior raw form. Its return operation recovers that exact prior form, and its last-label operation recovers the appended label. This retains the local construction provenance of every held step. The word-operation identity does not extend the fixed binary Fold support to arbitrary string labels.

35. Raw choices as discrete held selections

Each raw selection of a distinct carrier retains distinct labels in carrier order. Representing each label by its singleton form gives a generated discrete carrier collection. Under the same representation, every raw selection is accepted by the original discrete held-selection operation: a nonempty choice returns its exact collection tuple, while an empty choice returns the original structural empty-One form. The proof uses retained membership and distinctness and requires no exclusion of the literal empty-One label. This relation concerns raw choices and explicitly represented member forms; it does not identify the differently nested encoded outputs of the two source functions.

36. Finite algebraic operation tables

The operation-table constructor retains the registered carrier and ordered input/output rows. It first requires a nonempty distinct carrier, then checks that row count equals the complete ordered pair support and that the input sets coincide, and finally checks every result belongs to the carrier. These are the original source checks in their original order. Accepted tables preserve their data, cover exactly the carrier pair support, and have carrier-closed results.

Application filters the retained rows by the requested ordered input pair and returns a result only when exactly one row matches. Empty or multiple matches retain the source error. Fresh execution matches 526 constructor cases and 2630 lookup cases, including complete two-member operations in every row order and malformed inputs. General single-valuedness and total lookup follow from the constructor conditions as established below; the remaining algebraic property derivations are tracked separately.

37. Complete single-valued operation support

The constructor’s input coverage and row cardinality jointly force the projected input rows to be a permutation of the complete ordered pair support. This follows constructively for arbitrary finite size: locate the first support element in the proposed rows, remove that occurrence, and retain coverage and equal length for the remaining support. Since the carrier is distinct, the complete pair support is distinct, hence so is the projected row-input list.

Consequently, for every pair of carrier members the actual matching-row filter contains exactly one row. The translated apply operation returns its result, and the constructor’s closure check proves that result remains in the registered carrier. No uniqueness assumption is added to the source guards; it is derived from their original coverage and cardinality conditions.

38. Exhaustive identity and association

The identity test first checks that the proposed identity belongs to the carrier, then evaluates left and right preservation for each carrier member in source order. Its successful result is equivalent to both applications returning that member for every carrier member. The conjunction preserves source short-circuiting and lookup exceptions.

The associativity test traverses every ordered carrier triple. For each triple it evaluates the left composite before the right composite in the original order. On accepted operation tables, successful exhaustive checking is equivalent to equality of the two actual composed results for every triple. Totality and closure of the constructor supply the intermediate operation results, so no associativity law is assumed. Fresh execution agrees on 156 property calls over 39 table representations, including every two-member binary operation in two orders and malformed diagnostic representations.

39. Held return mates and commutation

The return-mate operation retains the source identity precheck. For each carrier member it exhausts candidate mates in carrier order and tests return to identity on both sides. Exactly one matching candidate is required; zero or multiple matches return None, while lookup exceptions propagate. Successful candidate filtering retains precisely those carrier members satisfying both return equations. Successful mate-row construction preserves the carrier order and provides a singleton candidate search for each returned row.

Commutation is evaluated on every ordered carrier pair, comparing both operation results with the original left-before-right evaluation. Its successful result is equivalent to all pair comparisons passing, and each comparison is equivalent to equality of the actual results. Fresh execution matches 156 return-mate and commutation calls across 39 table representations, covering all two-member operations in two orders and malformed diagnostics.

40. Public return and commutation semantics

The return-mate API produces a successful row list if and only if its identity test succeeds and every carrier member has an exhaustive candidate search containing exactly one mate. Successful output has the same carrier projection in the same order. Every returned mate belongs to the carrier, returns to identity on both sides, and is the only carrier member satisfying both equations for that item.

For accepted operation tables, the commutation test succeeds if and only if the actual operation outputs agree for every ordered pair and its reversal. The constructor’s totality supplies both outputs in the converse direction, so the equivalence does not rely on vacuous comparisons of missing operations.

41. Operation-preserving maps

The mapping relation is converted to the source dictionary semantics: the first occurrence fixes a key’s position, and subsequent occurrences replace its value. Native proofs establish replacement lookup, preservation of other lookups and the last occurrence’s value. The preservation operation retains the source sequence of checks: dictionary cardinality, presence of every source origin with a target-carrier image, then the operation equation for every ordered source pair.

Each equation compares the image of the source operation result with the target operation applied to the two input images, in the source evaluation order. Successful preservation is equivalent to all original checks passing; each successful cell comparison is equivalent to equality of its actual results. Fresh execution agrees on 1555 complete dictionary comparisons and 6220 preservation calls across repeated-key, extra-key and external-image mappings. Dictionary invariants and accepted-table totality are established below.

42. Total structure-map semantics

Dictionary updates preserve distinct keys, and the complete dictionary construction therefore has one entry per retained origin. Lookup succeeds exactly at a stored key. The source image-carrier check is equivalent to successful lookup of an image in the target carrier.

For accepted source and target operation tables, those image checks supply the mapped inputs and the image of every source operation result; constructor closure supplies membership of the intermediate source result. Each preservation comparison consequently terminates with a Boolean result rather than a missing-key or operation error. The public preservation operation succeeds exactly when its original size and image checks hold and the image of every source operation result

equals the corresponding target operation result.

43. Original algebra witnesses and complete grammar

Both original operation tables are retained and accepted by the translated constructor. The six registered witnesses are proved from their actual computations: complete four-row coverage, structural One identity, exhaustive association, exact two-sided return mates, commutation of the symmetric table together with failure of commutation for the lawful left-projection table, and preservation by the canonical self-map. All original names and statements are retained.

The algebra grammar contains all 512 products of its nine source dimensions. Native proofs establish exhaustive membership, exact cardinality, the unique all-preserving survivor, equivalence of acceptance to every original criterion, and rejection of each inadmissible coordinate. Fresh execution matches every candidate record’s order, identifier, survivor decision, exact form and reason against the pinned source artifacts. These finite grammar proofs accompany the general operation derivations; full program and model integration remain separately tracked.

44. Algebra program correspondence

The translated algebra program retains its original registration and dependencies, all 512 candidate records and lookup decisions, completeness certificate, closure and generality payloads, and four original controls. Candidate lookup depends only on the registered identifier, and an unknown identifier retains the source StopIteration outcome. Fresh native execution reproduces the pinned source artifacts, including every census trace identity, decision proof identity, closure and generality certificate, control receipt and registration field. These correspondence results accompany the general operation, return and map derivations above; full engine and dependency integration remains separately tracked.

45. Algebraic induction base and carrier extension

The One carrier has exactly its self pair. Its closed operation returns One, and both identity and association hold. More generally, closure and complete pair coverage force the sole result of any accepted singleton carrier to be its member; identity and association follow for every such accepted table.

Appending a fresh member retains the original nested-loop order of pair generation. Each existing left-member row gains its cell with the fresh right member, followed by the fresh left-member row and the fresh self cell. The enlarged support partitions into old pairs, old-to-fresh pairs, fresh-to-old pairs and the self pair. Exactly the last three classes are new, and a distinct original carrier with a fresh added member generates a distinct complete pair support. These results establish the base and pair-generation component of the original algebra induction; the property checks on an extended operation remain explicit obligations.

46. Identity and association under carrier extension

The exhaustive check on an appended domain evaluates the original prefix first. A false result or exception stops that evaluation; only a successful prefix proceeds to the added domain. Identity on an enlarged carrier requires preservation of each old member and of the added member on both sides. When the old identity holds and all old operation cells are preserved, the enlarged identity holds exactly when its two equations on the added member hold.

Every enlarged-carrier triple either consists entirely of old members or contains the added member in at least one position. For an accepted old table, closure keeps both intermediate old pair results in the old carrier. Preserving old operation cells therefore preserves the full composed old triple calculations. If the old table is associative, the enlarged table is associative exactly when every triple containing the added member passes its original association check. All seven new-coordinate combinations are included; no enumeration or scientific guard is removed.

47. Return mates under carrier extension

The exhaustive mate filter preserves carrier order and propagates lookup errors. With old operation cells retained, each old member’s enlarged search consists of its original candidate search followed by the added candidate’s two-sided return test. An originally unique mate remains the only mate precisely when that last test returns false. A passing added candidate creates a second match, and an error propagates.

Given prior successful return mates and preservation of old cells, successful return mates on the enlarged carrier are equivalent to three obligations: both identity equations hold for the added member; its return test fails for every old member; and its own exhaustive search over the enlarged carrier has exactly one mate. The original identity precheck and singleton requirement remain intact.

48. Operation maps under carrier extension

The original map-admission checks split into dictionary cardinality, images for old members and the added member, and operation preservation for old and newly formed source pairs. All left, right and self pairs involving the added member remain required.

When old source and target operation cells and old dictionary images are retained, source closure ensures that the old operation result also retains its image. Consequently the complete old preservation-cell calculation is unchanged. Old image coverage transfers when the enlarged target retains its old carrier. Given prior map success and these preservation conditions, enlarged map success is equivalent to the original dictionary-size guard, target membership of the added member’s image, and every new source-pair preservation check.

49. Concrete dictionary extension

For a successful operation map from an accepted source table, the generated dictionary keys are a permutation of the source carrier. A fresh source member therefore has no existing dictionary entry. The original update operation appends its key and image at the end, increasing cardinality by one and preserving every old lookup. The fresh lookup returns its assigned image.

These facts discharge the dictionary-size and old-image preservation conditions for the concrete appended mapping. With the old source and target operations retained and the old target carrier included, that mapping preserves the enlarged operation exactly when the fresh image belongs to the enlarged target and every new source pair passes the operation equation. The dictionary’s original first-key order and last-value replacement behavior are retained.

50. Finite order and bound operations

The order kernel retains the original nonempty distinct carrier and distinct relation checks, rejects external endpoints, and exhaustively evaluates reflexivity, antisymmetry and transitivity. Totality requires both partial-order success and comparability of every generated carrier pair. The latter equivalence is proved natively.

Lower and upper bounds retain the nonempty held-selection guard and reject external held members. Successful results contain exactly the carrier members satisfying every held comparison, in original carrier order. Greatest and least witnesses are selected only when the complete matching list has exactly one entry. Meet and join retain the bound-selection errors and conditional extremal result. Monotonicity retains dictionary replacement semantics and checks both endpoint images and every source comparison.

Fresh execution matches 1404 order, bound and extremal records and 13932 monotonicity comparisons, covering all two-member relations together with malformed carriers/relations, repeated and outside selections, and repeated-key mappings. These execution results accompany the stated bound and totality proofs; general partial-order and extremal derivations and complete program integration remain separately tracked.

51. Exact partial-order admission

For arbitrary represented finite carrier and relation lists, the partial-order check succeeds if and only if the carrier is nonempty and distinct, the relation is distinct, every relation endpoint belongs to the carrier, every carrier member has its reflexive cell, every pair of reversed relation cells has equal endpoints, and every pair of composable relation cells has its composite in the relation.

The native equivalence follows each original guard and exhaustive traversal. The transitivity condition covers all pairs of relation cells with matching middle endpoints; nonmatching cells impose no composite requirement. The reverse implication establishes success from precisely these same conditions.

52. Extremal witnesses and returned bounds

Greatest and least operations return a value exactly when the complete filtered list of matching candidates is the singleton containing that value. This characterization retains occurrence multiplicity on unrestricted input lists. Every returned value belongs to the supplied bounds, dominates them in the appropriate direction, and is the only bound with that property. For distinct bounds, those semantic conditions also imply successful return.

Meet succeeds exactly when lower-bound generation succeeds and its complete result has the returned unique greatest witness; join has the corresponding upper-bound and unique least criterion. Every returned meet belongs to the carrier, lies below all held members, and is above every carrier lower bound. Every returned join belongs to the carrier, lies above all held members, and is below every carrier upper bound. Bound-generation errors and absence or multiplicity of matching extrema retain the original outcomes.

53. Conditional meet/join existence and monotonicity

The original bound guard succeeds exactly for a nonempty held selection whose members belong to the carrier. Bound generation then returns its full ordered filter. On a distinct carrier, both bound lists are distinct. For an accepted partial order and a valid held selection, meet returns a given value if and only if it is a carrier lower bound above every other carrier lower bound. Join has the dual equivalence. Antisymmetry supplies extremal uniqueness, so the converses imply actual successful evaluation.

Monotonicity succeeds exactly when both endpoint images exist for every source comparison and their ordered image pair belongs to the target relation. Dictionary replacement semantics are retained; an empty source relation passes its exhaustive check.

The six original witnesses are proved using their actual operations and original names/statements: the diamond is a partial order, its arms remain incomparable, the chain is total while the diamond is not, the arms meet at One and join at whole, and the canonical identity mapping preserves every diamond comparison. Fresh execution reproduces all six witness rows.

54. Complete order grammar and program correspondence

The order/lattice grammar retains all 512 combinations of its nine original dimensions: carrier identity, relation provenance, partial-order witnesses, comparability, bound generation, meet uniqueness, join uniqueness, monotonicity and extra-order status. Native proofs establish complete membership, cardinality, exact acceptance criteria, the sole all-admitted survivor and rejection of each inadmissible coordinate. Every combination remains in the source order.

The native program retains its original registration and dependencies, complete candidate records, identifier lookup behavior, completeness and trace identities, decision proof identities, closure and generality payloads, and four original controls. Fresh execution matches all 512 candidate records and complete program fields against the pinned source artifacts. Unknown identifiers retain StopIteration. These results accompany the general order, bound and monotonicity derivations; source induction and full model integration remain separately tracked.

55. Order induction base and comparison extension

The One carrier with its identity relation is both a partial and total order. Its complete lower and upper bound lists for the held One selection are the singleton One, and both meet and join return One.

For a fresh carrier addition preserving every old comparison cell, the original endpoint and distinct-relation guards are retained. Partial-order success on the enlarged carrier is equivalent to the new reflexive cell, antisymmetry for every reversed pair touching the fresh member, and closure for every composable triple touching it. Old-only obligations follow from the prior order and retained comparisons. Given prior totality and an admitted enlarged partial order, totality of the enlarged order is equivalent to comparison between the fresh member and each old member.

56. Bound lists under carrier extension

Successful bound generation establishes that the held selection is nonempty and contained in the original carrier. When every old comparison cell is retained and that held selection remains fixed, every old candidate has the same bound-check result in the enlarged relation.

The enlarged lower-bound list is exactly the original ordered list followed by the added member when it lies below every held member; otherwise the list is unchanged. The upper-bound list has the dual formula. These are equalities of the actual generated lists, preserving order and occurrence multiplicity, and require no additional distinctness assumption.

57. Held-selection updates and extremal retention

Adding a carrier member to an already valid held selection filters the existing lower-bound list by comparison below that member, and filters the upper-bound list by comparison above it. These are equalities of the generated lists, retaining their order and multiplicity. On an accepted partial order, the old meet remains the meet exactly when it lies below the added held member; the old join remains exactly when it lies above it.

For an admitted carrier extension that retains all old comparisons, an existing meet remains exactly when the added carrier member, if it is a lower bound, lies below that meet. The join condition is dual: an added upper bound must lie above the old join. The conditions preserve the original conditional existence of extrema and include all old carrier candidates through the complete prior bound properties.

58. Extremal selection of the added carrier member

For an accepted carrier extension retaining all original comparisons, the added member is selected as meet exactly when it lies below every held member and above every member of the original lower-bound list. Dually, it is selected as join exactly when it lies above every held member and below every original upper bound. The original bound lists arise from successful nonempty held selections. Every original bound participates in these conditions; reflexivity supplies the added member's self-comparison, and accepted partial-order antisymmetry supplies extremal uniqueness. No existence of an old meet or join is required.

59. Generated graphs, complete paths and cuts

Graph construction checks a nonempty canonical node collection, duplicate-free edge relations and membership of every edge endpoint in the carrier, in that order. Path validation retains the nonempty path, every node membership and every adjacent ordered edge. The recursively generated adjacency list is proved equal to the original path zipped with its tail. The identity path is admitted exactly for a node in the carrier.

At a shared interface node, the adjacency list of the composed path equals the complete first adjacency list followed by the complete second adjacency list. Consequently the composed path is valid if and only if both component paths are valid. This is an equivalence for arbitrary finite prefixes and suffixes, without erasing any transition.

A successful cut returns exactly the original edge list filtered by unequal held-endpoint membership, preserving edge order and multiplicity. Its membership criterion is proved directly. The internal balance operation is equivalent to nonempty ingress, equal ingress and egress trace counts, and separate distinctness of both trace collections. The executable criterion is retained exactly.

Fresh native execution matches 4597 independently reconstructed records for graph construction, paths, adjacency, cuts and balance, including malformed and repeated inputs. Complete engine admission integration remains pending.

60. Finite reachability search

Reachability retains the original identity case: equal origin and target return exactly origin membership in the node carrier. Otherwise the queue starts with the origin and the visited trace is empty. Each iteration removes the queue head, checks target identity before visited membership, skips a previously visited head, or appends a new head to visited and appends every matching unvisited edge target to the queue in original edge order.

Termination follows from finite support and queue length. Initially the support consists of the origin and every edge target. Every queued node and edge target lies in that support or in visited. A new visit removes the held node from the auxiliary unseen support, strictly decreasing its length. A visited skip leaves support unchanged and strictly shortens the queue. Their lexicographic order is well founded. The auxiliary support does not decide search success, change queue order, omit duplicate enqueues or impose a depth bound.

The native search satisfies the exact fixed-point equation and the original empty-queue, target-hit, visited-skip and fresh-expansion equations. Fresh execution matches 8,224 reachability decisions and 10,280 ordered outgoing lists across every directed relation on three nodes, with additional duplicate-edge and external-endpoint stress inputs. Cycle search and its full return-path correspondence are formalised below.

61. Complete path witnesses for successful reachability

For every graph admitted by the original constructor and every origin and target, a true reachability result entails a complete path witness. The witness is a finite node list whose first node is the origin, whose final node is the target, and which satisfies the original path validator. Thus it retains every node membership and every ordered adjacent edge. The theorem requires no separate assumption that the queried origin belongs to the carrier.

Outgoing-list membership is equivalent to a retained edge from the held node to the queried node together with that node's absence from visited. Every property of queued nodes that is preserved by each retained edge is preserved through the exact queue loop. Applying this invariant to complete path witnesses establishes search soundness. The initial origin is retained as an alternative until its first edge supplies carrier membership; the equal-origin/target case obtains membership directly from the original identity check. Successful construction supplies both endpoint memberships for every retained edge.

Path extension appends the new target to the complete prior path and uses the shared-interface composition theorem, preserving its origin and every earlier transition. The resulting proof covers arbitrary finite admitted graphs, rather than only the enumerated execution cases. Search completeness is established by the closed-set argument below.

62. Reachability completeness and equivalence

Every complete valid origin-to-target path is found by the original finite queue search. The proof preserves the invariant that every edge leaving a visited node ends either in visited or in the frontier. Removing an already visited queue head preserves that invariant. Expanding a new head preserves it because every unvisited outgoing target is appended, in the original edge order.

If search terminates unsuccessfully, the resulting exhausted finite node collection contains the initial frontier and visited nodes, excludes the target, and is closed under every retained edge. A complete adjacent path beginning inside such a collection remains inside it at every transition. Consequently a valid path to the target contradicts unsuccessful search. This establishes completeness without bounding path length or discarding repeated queue entries.

Combining completeness with soundness gives the exact equivalence, for every admitted finite graph: reachability returns true if and only if a complete valid path starts at the queried origin and ends at the queried target. Identity reachability holds exactly for carrier nodes. Complete path witnesses compose at their shared interface, yielding the original transitivity law for reachability.

63. Exact finite cycle traversal

The cycle procedure retains the original outer node order and recursively scans every edge in its original order. Edges whose origin differs from the current node are skipped. An edge to a node already in the held path returns true immediately. Otherwise its target is appended to the held path and visited recursively; a false result resumes scanning the remaining edges. No branch or edge is removed.

Finite termination follows from an auxiliary list containing every edge target. Every target remains either in that unseen support or in the held path. A recursive call is made only for a target absent from the held path; that target therefore occurs in unseen support, and removing it strictly decreases the support length. Each local edge scan is structurally recursive on the remaining edge list. Neither argument introduces a depth cutoff.

The native procedure satisfies its exact recursive equation. Empty edge sets and nodes without outgoing edges cannot yield a cycle, and a retained outgoing edge to a held-path member forces a true result. Fresh execution matches 3,084 graph-cycle decisions and 14,392 recursive visits across all directed relations on three nodes and additional malformed/repeated input stresses. General equivalence with complete nontrivial return paths is established below.

64. Cycle detection and complete return paths

For every admitted finite graph, the original cycle procedure returns true if and only if there is a complete valid path with at least one edge whose final node occurs earlier in the path. The starting node belongs to the graph carrier; every node and adjacency is checked by the original path validator. Self-loops satisfy the one-edge case. Longer witnesses may contain repeated nodes, with no extra simplicity restriction.

The proof first establishes exact correspondence between the recursive traversal and a finite return trace recording every successive edge target. A trace either closes along a retained edge to an already held node, or advances along a retained edge to a new node and continues with that node appended to the held path. Every such trace is nonempty, contains only retained adjacent edges, and yields a valid node path on an admitted graph. Its terminal target belongs to the held path together with the preceding trace prefix.

Conversely, a complete path ending at a prior node produces a return trace. Traversal follows its retained edges until the first held-node return; if a repeated node occurs before the supplied terminal return, that earlier return supplies the witness. Thus neither the equivalence nor the executable traversal requires paths to be simple. The proof applies to arbitrary finite admitted graphs and does not depend on the finite execution comparisons.

65. Graph witnesses and complete registered grammar

The six original graph witnesses are retained with their exact names, statements and executable predicates: valid chain path, missing-edge rejection, directed reachability, explicit cycle return, exact crossing cut and paired internal balance. The chain and cycle examples satisfy the original graph constructor. Positive reachability and return results follow from the general path theorems; the negative chain cases follow from the exact traversal equations.

The registered graph grammar exhausts all 512 combinations of its nine binary structural coordinates: nodes, edges, orientation, path, connectivity, cycle, cut, balance and additional rules. Native proofs establish complete enumeration, cardinality, equality of source classification with the preservation criteria, the unique all-preserving survivor, and rejection when any coordinate is replaced by its source alternative. No candidate is omitted.

Fresh native execution of the complete graph program matches the pinned registration, census, all 512 actual candidate lookups and decisions, trace and proof hashes, closure and generality records, four original controls and unknown-identifier behavior. These results accompany the general path/search and cycle/return equivalences. Source graph induction obligations and full engine admission integration remain separately tracked.

66. Graph induction and locality of path updates

The One graph with no edges is admitted by the original constructor, supplies its identity path and identity reachability, and has no directed cycle. Under extensions retaining prior nodes and edges, every existing complete path and return trace remains valid. Consequently prior reachability on an admitted graph and prior detected cycles are retained.

For a carrier extended by a fresh node and an appended edge selection, any newly valid path whose nodes all belong to the old carrier must contain an added edge absent from the old relation. When each added edge has the fresh node as an endpoint and that node is absent from the old carrier, such a path cannot use an added edge. Hence the original and extended path-validation results are exactly equal on every sequence of old nodes, including rejected sequences. This establishes the locality of path updates in the source fresh-node induction step.

These results retain the explicit conditions of the extension. They do not impose new constructor guards or restrict the original graph operations. Ordered pair-support generation, cut and paired-balance extension are established below; held-selection characterization is established below; full source and engine admission integration remain separately tracked.

67. Admitted graph extension, ordered pairs, cuts and balance

The graph constructor succeeds exactly when its node list is nonempty and distinct, its edge list is distinct, and every endpoint belongs to the node list. If a genuinely fresh node is appended, a distinct added-edge selection whose endpoints lie in the enlarged carrier and whose edges touch that fresh node is disjoint from the old relation. The combined graph therefore satisfies precisely the original constructor checks.

The complete ordered node-pair product extends row by row: each old left-node row retains all old right-node cells followed by its fresh-right cell; the final fresh-left row contains every old right node followed by the fresh self-pair. The equality retains the full generated order and does not assert that every generated pair is selected as an edge.

For an already successful held cut selection, extending the node carrier and appending edges returns exactly the old cut list followed by the added edges whose endpoint held-membership differs. Selection validity follows from the original nonempty and carrier-membership guard; no distinctness requirement is added to the held list. For an internally balanced pair of ingress/egress trace lists, appending one previously absent trace to each side preserves equal counts and separate distinctness, hence preserves the original balance predicate.

68. Held-partition updates and exact paired-balance conditions

Appending a carrier node to a valid held selection replaces each endpoint-membership test by membership in the old selection or in that appended singleton. Filtering the complete original edge list by the resulting unequal-membership test gives exactly the updated cut, including the original allowance for repeated held nodes. An edge whose endpoints differ from the appended node has an unchanged cut decision.

For an admitted old graph, a genuinely fresh node belongs to neither endpoint of any old edge. If that node is appended to both carrier and held selection while new edges are appended, the resulting cut is exactly the prior cut followed by the added edges crossing the enlarged held partition. The equation preserves original list order.

Appending one ingress trace and one egress trace yields internal balance if and only if the original trace counts agree, each original trace list is distinct, and neither appended trace already occurs on its respective side. This characterization includes empty original lists: their first trace pair is balanced. For an already balanced pair, the exact remaining requirements are freshness on both sides. No restriction on trace contents is added to the original operation.

69. Batch source-program correspondence

Native execution of all forty-four implemented source programs reproduces the pinned V3 registration fields, complete candidate censuses, all 23910 actual identifier-based decisions, closure and generality records, original control results and every embedded digest. The complete classification comparison across all 44 represented entries covers 23910 candidates; Exact Operations and Half-One include their complete source program adapters. All 3278 theorem proofs and 6394 audited definitions/theorems have zero axiom dependencies.

Fold Assembly combines the general finite-support, uniqueness, transition, return and proposal-classification proofs with exact execution of its original 192-candidate program. This completes the registered assembly entry after its Count and Fold dependencies. Completion is recorded separately from output parity alone. The Form adapters preserve tuple and list edge inputs; the protocol adapter preserves the original operand-returning behavior for literal host records and proves its Boolean specialization.

70. Complete Part Equivalence source correspondence

The registered common-refinement entry combines the general nested pair-cell, complete lifting, constructive bijection and uniform-refinement proofs with all 1024 original witness forms. Its complete native program reproduces every registered candidate and actual identifier-based decision, closure/generality certificate, control record and digest. The original tampered-artifact predicate counts accepted candidates in the complete enumeration and accepts the specified equal lifted traces; the unequal example is rejected. The three non-hash control outcomes have native proofs. Registration retains only the original One, Count and Part dependencies and introduces no axioms or free scientific parameters. Together these results complete the registered Part Equivalence entry.

71. Finite Form source input correspondence

The source validator requires a tuple at a Form root and accesses either tuple or list containers at the held edges. The formalisation retains both containers and their mixed combinations, arbitrary finite child forms and ignored trailing edge items. Its acceptance equivalence characterizes precisely the finite One/Fold productions over this representation. Original short-circuit order preserves false returns and index errors, including distinct tuple and list error messages. Canonicalization validates before recursively emitting the ordered trace, and every accepted represented input has a generated canonical Form. Canonical equality retains the established identity and equivalence laws. Fresh native execution reproduces all 288 Grammar and 192 Enforcement candidate decisions, complete evidence values and hashes. These results complete both registered entries in dependency order.

72. Completed mathematics source entries

Exact Arithmetic, Discrete Objects, Combinatorics, Algebra, Order and Lattice, Graph and Network, Geometry and Topology, Probability and Statistics, Optimization, Dynamical Systems, Logic and Proof, and Category/Type/Composition combine their retained source-mapped derivations, complete candidate grammars, original operational witnesses and controls with passing full-program comparisons. Their registered dependencies are complete. The current rebuilt audit establishes zero axiom dependencies across all 3278 theorem proofs and 6394 audited definitions/theorems. These twelve mathematics entries join fifteen foundation entries, twelve information-science entries and sixty-three computation entries, yielding 44 completed registered entries. The source-component and executable evidence records distinguish these completed entries from the remaining translated portions.

73. Completion of the represented foundation and mathematics entries

The initial 21 represented entries and Geometry/Topology are complete against their registered V3 source content. The six-entry verification batch covers Measurement Boundary, Exact Operations, Half-One, Fold Dynamics, Primitive Map Uniqueness and Derivation Trace, reproducing all 980 identifier-based candidate decisions and every original registration, census, closure, control and digest field. The complete operational and half-One programs retain the original 128-form enumerations and original dependency lists. Their program correspondence accompanies the previously established general positive-domain and first-Fold derivations.

The measurement protocol preserves Python conjunction semantics on its literal host records: truth testing controls evaluation, a false all-rows operand is returned unchanged, and an admitted source operand is returned unchanged. A nonempty source identifier therefore remains that identifier. Boolean inputs agree with the original Boolean protocol theorem. Constructive integer cases distinguish zero from positive and negative host integers without axioms. These host values are external protocol data, not added scientific parameters.

The current audit covers 3278 theorems and 6394 definitions/theorems with zero axiom dependencies. The census contains 2781 entries;2746 entries remain beyond these completed results.

74. Finite incidence and open-family derivations

The incidence predicate accepts exactly a nonempty distinct cell carrier and a distinct boundary relation whose endpoints belong to that carrier and whose pairs are not self-incidences. The adjacency predicate, which does not itself impose the incidence admission guard, accepts exactly two different cells with a shared retained face. General native proofs establish this equivalence, symmetry and irreflexivity.

Each proposed open is projected onto carrier order and an empty projection is represented by the original empty-One literal. The finite topology check retains the original canonical-set cardinality comparison, presence of empty One and the whole carrier, and exhaustive pairwise union/intersection closure. Its acceptance theorem states precisely these conditions. Structural induction then establishes membership of every finite generated join and intersection of admitted canonical opens. Off-carrier labels and collisions with the literal empty-One marker retain their original executable behavior.

Continuity constructs its image map with the last supplied row for each key, requires exactly the source point keys, and tests each target open's source-ordered inverse image against the supplied source open family. No extra topology validation or normalization of the supplied source family is inserted. Native proofs establish the exact key-support and inverse-image conditions and last-row precedence. The original incidence, adjacency, finite-topology and continuity witnesses hold. The path-search operation, returned-path soundness, general minimality and unreachable-case equivalence are established below.

75. Boundary-depth dimension

For a registered finite incidence complex, the dimension function returns one for a cell with no faces and one plus the maximum of the recursively derived face dimensions otherwise. The active descent stack rejects a return to a cell already on that stack. Independent branches share the parent stack, preserving the original push/pop behavior; an unrelated cycle outside the queried descent does not cause rejection. Termination follows constructively by removing each newly visited label from the finite support. This support introduces no depth bound or scientific parameter.

A native inductive derivation records every face in source order, its own complete derivation and the maximum rule at the parent. The successful computation is equivalent in both directions to this derivation for every admitted input and result. All successful dimensions are positive; the original carrier-admission and return-cycle error conditions are retained. Execution reproduces the original triangle dimension of three and edge dimension of two, together with leaf, invalid-carrier, reachable-cycle, unrelated-cycle and shared-descendant cases.

76. Ordered undirected path search

The path-distance procedure checks only membership of its two endpoints in the supplied carrier. Equal endpoints return the original empty-One value. For different endpoints, the queue stores both each held node and its full path. A newly visited node scans the original edge list in order, extending only to unvisited neighbors; the first target extension is returned. Pending extensions retain duplicate occurrences and source order. Edge labels outside the supplied carrier remain traversable, as in the source procedure.

The formal implementation has a constructive termination proof over unseen edge-endpoint support and queue length. Its extension decision is proved equivalent to the literal early-return edge scan. Every returned nonidentity path has a native inductive trace beginning at the origin and extended by original undirected incidences to the target. Endpoint rejection, empty-One identity, exhausted queue, visited-node skipping and fresh expansion are explicit native laws. Source-case execution preserves reversed-edge traversal, off-carrier intermediate labels, self-edges, duplicate queue entries and edge-order tie selection. Exhaustive checks on all 64 directed edge subsets over three labels and all 576 endpoint pairs agree with independent enumeration of simple paths for reachability and minimal length. The finite checks accompany the general shortest-path and unreachable-case theorems below.

77. General shortest-path and reachability derivations

The search queue preserves nondecreasing path lengths and a width of at most one edge. When a node is visited, every adjacent node that remains unvisited has a queued route bounded by any path to that visited node followed by the edge. This invariant, together with the retained origin route, supplies a frontier route no longer than any proposed trace to an unvisited endpoint. Since the target is returned upon discovery and is never queued, the final extension is no longer than any alternative trace to that target.

For every finite source edge list and distinct admitted endpoints, every undirected trace therefore yields a successful returned path of no greater length. Combined with returned-path soundness, this proves minimality of the actual result against all finite undirected alternatives. The procedure returns None if and only if no such trace exists. The proof retains the original carrier guard and does not require unique edges, globally admitted edge endpoints, or exclusion of loops. Equal endpoints retain the separate source identity result, empty One. These statements are general native proofs, with zero axiom dependencies; they do not depend on a finite regression bound.

78. Completed finite geometry and topology entry

The ten registered binary dimensions generate all 1,024 kernels in the original order. Native proofs establish product membership, exact cardinality, agreement of identifier acceptance with the original structural criteria, the unique admitted kernel, and rejection upon replacement of each required coordinate. All six source operational witnesses are evaluated by the translated operations; the original depth and path witnesses are additionally obtained from the general native derivations and source queue equations. The closure evidence retains the original five laws, induction base and successor, exclusions and control records. Connectedness and reversible incidence-preserving deformation retain their original named criteria and source explanations; the source entry introduces no separate executable procedure for those clauses.

Fresh native execution reproduces every original registration, census, decision, closure, control and digest field, including all 1,024 identifier-based decisions and refusal of an unknown identifier. Together with the general incidence, dimension, adjacency, shortest-path, topology and continuity derivations, these results complete the registered Geometry/Topology entry. The current completed census total is 35 of 2,781, with 2,746 entries remaining.

79. Exact events and common-refinement weights

An admitted event preserves its supplied order and is a duplicate-free held selection of a complete nonempty support. The native admission results establish support positivity and uniqueness, event uniqueness and inclusion, and the bound of held cardinality by support cardinality. Empty events retain the source empty-One marker. The original literal-marker comparison is preserved when a registered state itself has the label empty-One.

A nonmarker event weight is the reduced positive held-count/whole-count fraction. Native proofs establish exact cross-multiplication equality with the original counts, positive numerator, coprime numerator and denominator, and the through-One bound. Conditioning first validates the event and then the condition in the original order. Its common refinement selects exactly the support states belonging to both held sets; an empty refinement or marker condition returns empty One. Every fractional result is the exact refinement-count/condition-count ratio and is bounded by the whole. No fitted quantity, stochastic premise or additional scientific parameter enters these operations. The observation-class construction and complete probability program are included below.

80. Pair-cell factorization and exact modal summaries

The independence operation extracts marginal labels in their first-occurrence order and compares the supplied joint support with their complete ordered Cartesian product. The native product-filter identity establishes that retaining pair cells by both held-event predicates yields the Cartesian product of the retained marginal cells. Its cardinality is the product of the two retained marginal cardinalities. For duplicate-free held selections contained in their marginal supports, these cardinalities equal the supplied held counts; cross-multiplication therefore proves exact joint-weight factorization. The operational procedure retains the source rejection of incomplete or differently ordered products, its nonempty-event guard, raw event-list lengths and empty-joint division error.

The modal summary counts every occurrence in the complete trace, forms the greatest count over first-occurrence labels and retains every label attaining it in that order. Native proofs characterize membership exactly as trace membership with frequency at least that of every other trace label. Every nonempty trace has a modal label, the output contains no duplicate labels, and every tied alternative is retained. The empty trace retains the original rejection. These operations introduce no stochastic parameter.

81. Observation partitions and completed finite probability entry

An observation assigns each admitted support state its final dictionary value. Repeated mapping keys retain their last value, and admission requires equality of mapping-key and support sets. Native proofs establish that every support state has an image and that the image trace retains the support length. Distinct class labels are taken in first-occurrence order. Each class consists exactly of the support rows mapped to its label, retaining their original order and multiplicity. Every support state is covered, each emitted class is nonempty, and a state cannot belong to classes with different labels. Observation therefore closes distinctions between deterministic support states without adding a stochastic evolution law.

The complete nine-dimensional source product contains 512 candidate kernels. Native enumeration and criteria proofs establish the unique all-admitted kernel and rejection of each substituted coordinate. All six original operational witnesses evaluate successfully; their joint passing result is kernel checked. Native execution matches every source registration, census, decision, closure, control and digest field for all 512 identifier lookups, and preserves unknown-identifier refusal. The original induction, derivation exposition, correspondence terminology and empirical exclusions are retained. These results complete the V3 finite probability and descriptive-statistics entry. Inferential claims about natural measurements retain V3's requirement for preregistered blind validation through the empirical engine.

82. Exact finite optimization and retained alternatives

The optimization carrier is nonempty and canonical. Its feasibility relation retains the final verdict for every candidate key and must cover exactly the carrier's keys. The feasible selection preserves candidate order and contains precisely the candidates with true verdicts. A preference cell must join two distinct feasible forms. The undominated selection retains every feasible occurrence with no incoming preference cell; a native theorem supplies a retained feasible dominator for every excluded feasible form. No acyclicity or extra ordering assumption is imposed by the source procedure.

A unique optimum is returned exactly when the complete held list is a singleton. Every other result retains the original equivalence-class tag and full list, including ties, incomparable alternatives and an empty result from a preference cycle. Constraints require membership in every supplied allowed set. Pareto dominance requires every criterion to admit the directed comparison and at least one to

witness strict preference; self-comparisons cannot eliminate candidates. Native frontier membership and elimination-witness results establish complete selection and retention of incomparable forms, including the original empty-criteria behavior.

The approximation certificate accepts the original signed-integer domain and rejects a nonpositive optimum or candidate count below that optimum. Every accepted result is the reduced positive optimum-count/candidate-count part. Native ratio and through-One results establish exactness without rounding, negative penalties or a tuning parameter. Shared-interface decomposition retains the source's complete-factorization condition; no additional decomposition algorithm is introduced.

All 512 kernels in the original nine-dimensional grammar are retained, with native enumeration, criteria, uniqueness and coordinate-rejection proofs. All six original operational witness computations pass. Full native program execution matches every pinned registration, candidate, decision, closure, control and digest field and the unknown-identifier error. These results complete the finite optimization entry; complexity and particular objective-family laws retain their original placement in the algorithms and complexity branches.

83. Generated trajectories and predecessor records

Finite dynamics admission requires a nonempty canonical state list, distinct retained transition cells and both endpoints of every cell inside the state carrier. Trajectory validation has the separate original domain: it requires a nonempty trace and each adjacent ordered pair in the supplied transition relation. It introduces no additional carrier-membership guard. Native membership results establish these exact conditions, and composition at a shared canonical interface is equivalent to validity of both component trajectories.

Elapsed time retains the complete ordered adjacent-transition trace. Its length is one fewer than the number of retained states; every trace with at least two states has positive transition count. A singleton trajectory has empty-One duration and an empty trajectory is rejected. Fixed forms retain precisely the source rows with identical source and target, while predecessor records retain every source row mapped to the requested target, preserving order and multiplicity.

Record-based reversibility requires inclusion of every original transition in a duplicate-free retained ledger. Native results show that each source predecessor remains recoverable from that ledger and that omission of any original transition fails the check. Extra retained rows remain governed by the original inclusion test; no stronger equality condition is imposed. First-return selection, registered-perturbation reachability and the complete dynamical program are included below.

84. First returns and complete registered perturbation stability

The first-return operation scans terminal positions in source order and selects the earliest equal prior state. The returned trace is the exact inclusive slice between those positions. Native proofs establish that the prefix before the repeated terminal contains no duplicate states and that no matching state precedes the selected starting occurrence. No return is produced exactly when the entire trajectory is duplicate-free. Every returned slice has at least two state occurrences with equal endpoints; when the original trajectory is valid, the slice retains valid transitions throughout.

Registered perturbation stability uses the original directed breadth-first queue. The target is checked before the visited test; visiting appends the current state, and successors are appended in transition-row order with the source duplicate behavior retained. A constructive finite-support measure proves termination without an arbitrary search bound. The native equivalence proves successful search exactly when a finite retained transition derivation reaches the terminal. Stability holds precisely when this is true for every registered perturbation. Empty perturbation lists retain the

original vacuous result, and identity reachability requires no additional state-carrier admission.

All 1,024 kernels in the ten-dimensional source grammar are retained and verified. All seven original operational witnesses pass, with stability obtained from explicit transition derivations. Full native execution matches every original registration, candidate decision, closure, control and digest field, including unknown-identifier refusal. Together with the trajectory and predecessor results, these complete the exact finite Dynamical Systems entry. Continuum differential descriptions retain their original status as possible measured or finite-approximation correspondences, not premises of the entry.

85. Finite logic and registered proof traces

A proposition retains a nonempty distinguished form and one of the two original held orientations. A proof step retains nonempty ordered premises, its conclusion and a nonempty rule name. The native constructors preserve the exact source guards and error cases; round-trip results establish that their represented values retain every original field. Denial switches orientation without changing the form, is involutive and produces a distinct oriented proposition. Conjunction retains both constituents with the original joint tag. Disjunction retains a supplied generated branch and both ordered alternatives, rejecting a branch outside that pair.

The proof checker walks the supplied steps in order. Every premise must already be available, and the complete rule-name/premise-tuple/conclusion record must occur in the registry before the conclusion is appended. Native equivalence establishes that acceptance is exactly an inductive trace satisfying those conditions at every step. Relative soundness follows by induction: any property held by the initial premises and preserved by every applicable registered rule holds for all resulting available propositions. This is relative registered-rule preservation, with no additional logical axiom or authority-based admission.

Consistency checks that no admitted proposition has its held denial also admitted. Registered-grammar completeness separately requires distinct generated propositions and a held decision for each proposition or its denial. These checks retain their original empty and duplicate-list behavior and make no global completeness claim beyond the supplied grammar.

All 1,024 source kernels and all seven operational witnesses are retained and verified. Native program execution matches every pinned registration, census, decision, closure, control and digest field, including unknown-identifier refusal. These results complete the finite generated Logic and Proof entry. Computability, self-reference, undecidability and incompleteness retain their original placement in the later formal-computation branch.

86. Compositional paths and finite type constructors

The native arrow constructor retains name, source, target and the complete nonempty trace, with precisely the original nonempty-field admission rule. Identity constructs the original empty-One return record. Composition rejects unequal interfaces; every successful composite retains the first source, second target and both traces in their original order. Canonical paths remove exactly those items whose text begins with the registered empty-One-return prefix. Canonicalization distributes over trace concatenation. Consequently both successful groupings of three arrows have the same interfaces and canonical transition path, by constructive list association.

Type membership uses the complete source key-set comparison and the last registered verdict for each key. It preserves carrier order and repeated occurrences. The product is the full ordered pair support, with membership equivalent to membership in both factors. The sum retains the original left-held and right-held tags; its membership theorem identifies exactly the corresponding source

alternative. Empty and repeated carriers retain the original source behavior. These results formalise the path-association and finite type-constructor components of the registered compositional entry.

87. Identity, transport and route comparison

For every nonempty source object, the exact original identity constructor succeeds. Its empty-One return contributes no canonical transition. Successful left and right identity compositions retain the other arrow's source, target and canonical path. Composition of source-admitted arrows exists precisely when the first target equals the second source; original field admission and complete trace retention are preserved. These results derive the registered identity and interface conditions constructively.

The native transport predicate retains the exact source procedure: the last object and arrow mapping for each key is used, every supplied arrow must have its name and endpoints mapped, and its image endpoints must equal the mapped endpoints. Native equivalence gives these conditions for every supplied arrow. The predicate retains the source's empty-input behavior and does not introduce additional executable checks.

Naturality replays each supplied nonempty route in order, propagating the original first composition failure. The resulting trace is exactly the concatenation of the input arrow traces. Route comparison succeeds precisely when source, target and the complete canonical traces agree. Empty input retains the source indexing error. All 1,024 original kernels, seven operational witnesses and complete generated-program outputs are verified against the pinned V3 records, completing the registered finite compositional entry.

88. Symbols and observation-relative distinction

A source symbol retains its canonical form and held label as an ordered pair. Alphabet admission requires a nonempty distinct list of these pairs and nonempty fields, with the original guard order and without changing source labels. Observation validation requires a nonempty alphabet, equal row and alphabet counts, distinct observation source pairs, exact source-set coverage and nonempty observation labels.

The native lookup selects the label only when exactly one source row matches the full symbol pair. Constructive results establish that every member of a validated alphabet has exactly one nonempty observed label and that each returned label retains its original observation row. Two source-admitted labels produce a retained distinction exactly when those labels differ. The operation is irreflexive, including its original early return before observation lookup, and is symmetric on a validated alphabet. These results formalise the alphabet, observation-admission and pairwise-distinction components of the registered symbol entry.

89. Observation classes, complete distinction ledgers and extension

Observation classes retain the first occurrence order of the observed labels and the original alphabet order of every source fibre. Constructive results establish that their labels are distinct, their fibres are nonempty, every source symbol occurs in its corresponding fibre, and a symbol cannot belong to two fibres with unequal labels. This partitions the complete admitted alphabet without replacing or erasing microform identity.

The pair ledger generates every original positional pair with the earlier symbol first. Its separate retained and closed passes preserve the source order and original lookup-error behavior. On validated observations both passes succeed, their outputs are disjoint, and their union exhausts the generated pair list. A singleton alphabet has no pair distinction.

Appending a fresh symbol generates one new pair with each prior symbol in the original nested-loop order, while preserving prior pairs. Appending its one observation row supplies its unique nonempty label and preserves every old lookup and distinction; every member of the extended alphabet still has a nonempty observed label. Under coarsening, a previously closed distinction remains closed, and a previously retained distinction closes exactly when its image labels are merged. These results establish the source successor and coarsening clauses.

All 512 original symbol kernels and all six operational witnesses are retained and verified. Fresh native execution matches the pinned registration, census, decisions, closure, controls and digest fields, including unknown-identifier refusal. This completes the registered finite Symbol Distinction entry, preserving its original exclusion of linguistic meaning, occurrence frequencies and downstream information quantities.

90. Encoding classes and exact inverse decoding

Encoding admission retains the original nonempty, distinct source and code-support lists, complete equal-length source relation, distinct origin keys and code-support membership. Codewords are exact held-label lists; their original empty-word behavior is preserved. Source classes occur in first-use encoding order and retain source order and multiplicity. Native results characterize membership, distinct class keys, source provenance and nonempty used classes on admitted inputs.

The exact decoder exists if and only if every used codeword class contains a single source occurrence. Its ordered inverse relation preserves those codeword keys. Decoder lookup requires exactly one matching image and retains the original error on zero or multiple matches. Constructive proofs establish that applying the resulting decoder to every row of an admitted lossless encoding returns the original canonical source. The raw operations retain their original empty and malformed-input behavior independently of encoding admission.

91. Closed encoding distinctions and exact composition

The closed-distinction ledger enumerates source pairs in their original positional order and compares each source's first encoding image. On admitted encodings the computation is total. A pair occurs in the closed ledger precisely when it is one of the generated source pairs and both members belong to the same retained codeword class. The empty and singleton source cases retain empty ledgers; missing first images retain the original error behavior on unvalidated inputs.

Encoding composition requires distinct second-stage interface keys and an image for every first-stage codeword. Native existence is equivalent to these exact conditions. Every source occurrence remains in its original order, and an output relation holds precisely when a first-stage source-to-codeword relation joins a second-stage codeword-to-output relation. No source alphabet or injectivity condition beyond the original composition guard is added.

92. Encoding source extension and finite closure

For a fresh source, appending its one encoding pair leaves every previous source-to-code membership unchanged. Every codeword class retains its original source order and gains the fresh source at the end exactly when it is the newly assigned codeword. The used-codeword set gains precisely that word. A single source and one codeword have the singleton inverse. If an encoding has an exact decoder and the appended word is unused, all old singleton classes remain singleton and the new class is the singleton fresh source; therefore the extended exact decoder exists. If the appended word is already used, its nonempty class gains the fresh source and is no longer singleton; the exact decoder is absent and that merged class remains explicitly recorded.

These source-extension statements hold for arbitrary finite source lists satisfying the original admission and freshness conditions. They do not rely on the witness source size. All 1,024 original encoding kernels and all five operational witnesses are retained. Fresh native execution agrees with every pinned registration, census, decision, closure, control and digest field, including unknown-identifier refusal. The Encoding and Decoding entry is complete within its original finite representation boundary; code-length value, compression, channel reliability and correction remain subsequent source subjects.

93. Complete information support and position generation

The information-quantity support operation requires at least two distinct held labels and a nonempty canonical position trace. Its complete ordered Cartesian support contains exactly the words whose length equals the position-trace length and whose labels belong to the held alphabet. Native proofs establish completeness and duplicate freedom for every finite admitted input. Appending one position extends every prior word once with every held label in the original product order.

For every generated prefix and suffix, inserting any held label produces a member of the corresponding complete support. Inserting two distinct held labels with the same prefix and suffix produces distinct words. These statements establish the arbitrary finite independent-position support relation. Support composition retains the original left-outer/right-inner pair order and concatenates the two source words. Each output occurs precisely through such a source pair; raw duplicate or variable-length input supports retain their original behavior. No logarithmic value or additional information scale is introduced.

94. Exact information profiles and independent position traces

For every admitted support, the positional pair construction contains each unordered nonidentity pair once, with its earlier support member first. The native derivation establishes endpoint provenance, absence of self-pairs and reversed duplicates, complete pair coverage and unique records. Observation admission requires unique image keys and exact support-key coverage before the support-pair validation. Empty observation labels remain permitted. The profile retains the support trace, first-occurrence observation-class order, retained distinctions with unequal images, and closed distinctions with equal images. These ledgers are disjoint and exhaust the complete support pairs. Fine observation retains every pair and closes none; coarsening cannot reopen a closed distinction and can only remove retained distinctions.

The position-trace computation explicitly tests both first held labels for every generated prefix and suffix at each position. Constructive proofs show that the complete support contains both alternatives at every finite depth and that unequal held labels give distinct words. The admitted computation returns exactly one one-Fold-distinction record for every original position, in order. Appending a position preserves the preceding unit trace and appends its unit; composition concatenates the position traces. Together with complete word generation and pair coverage, these results establish the original finite successor clause without a depth table or logarithmic proof quantity.

All 512 quantity kernels and all five operational witnesses are retained. Fresh native execution matches the pinned registration, census, decisions, closure, controls and all digest fields, including unknown-identifier refusal. Information Quantity is complete within the original exact finite boundary; conventional logarithmic units remain post-seal correspondence summaries rather than SFT proof values.

95. Deterministic entropy partitions and observation refinement

The exact entropy partition requires nonempty duplicate-free microstate support, unique observation keys and exact support-key coverage. Class labels follow their first observed occurrence in the original support order; each fibre retains its microstates in that order. Native proofs establish complete coverage, disjoint classes, nonempty fibres and distinct members. Unresolved distinctions are the complete positional pair list within each class. Empty and singleton lists return structural empty One; the general empty-One result is equivalent to at most one input member. The complete closed-pair output flattens these class ledgers in class order.

The original refinement operation has a broader raw domain than partition admission: it tests only the two observation key sets against support and retains dictionary last-row behavior for repeated keys. Its native characterization states exactly that equality under the fine observation implies equality under the coarse observation for every support pair. The relation is reflexive on matching-key observations and transitive. For admitted exact partitions, refinement holds if and only if every fine class is contained in a coarse class. These results retain deterministic observation provenance without introducing a stochastic source or scalar entropy value.

96. Exact entropy parts and closed-pair monotonicity

Each entropy record retains its observation label, exact positive support part, complete microstate trace and structural unresolved-distinction value. The native construction preserves the partition's record order and uses the exact reduced fraction of the class cardinality over the complete support cardinality. A cross-product identity proves preservation of that rational value, and the part is positive and at most the whole. Literal empty-One microstate labels remain ordinary retained microstates; structural empty-One uncertainty is the separate unresolved-distinction result.

Refinement implies inclusion of each fine class's positional pairs in its containing coarse class. The complete closed-pair ledger has no repeated pair records, both within a class and across disjoint classes. Native finite-list counting therefore proves that a finer admitted observation has no more closed pairs than an admitted coarser observation it refines. The result applies to arbitrary finite admitted support and retains every pair's source coordinates.

97. Entropy successor closure and exact finite completion

For one deterministic microstate, the exact partition is one class whose native normalized support part is One and whose unresolved ledger is structural empty One. Adding a fresh state preserves every prior image and appends the new observation in source order. Its assigned class appends precisely that state; every other class retains its member trace. The pair successor preserves the original nested order and contains exactly the old class pairs together with one pair from each prior same-class member to the fresh state. Each returned class part satisfies the exact member-count/new-support-count ratio. The changed total is retained in the weights of all classes.

The general successor derivation, exact weighted class ledger and refinement containment/count results complete the original finite entropy statement. All 512 candidate kernels and all five operational witnesses are retained; fresh native program output matches the pinned census, actual lookup decisions, registration, closure, controls and digest fields. The retained 9805 independent input comparisons cover partition/closed pairs, raw unresolved pairs, refinement and normalized Fraction records, including the original invalid and duplicate input domains. Entropy and Uncertainty is complete at V3's declared finite boundary. Conventional scalar Shannon summaries remain subject to the separately declared source correspondence map.

98. Exact compression descriptions and reconstruction provenance

A dictionary description is admitted precisely when its dictionary and token trace are nonempty, dictionary keys are unique and every retained token has a dictionary entry. The executable source allows empty dictionary blocks and empty string labels; the translation retains these domains. Reconstruction concatenates each referenced block in token order. The native lookup theorem connects each selected block to its unique retained dictionary row, and the provenance theorem identifies the supplying token and block for every reconstructed cell.

Losslessness is equivalent to a nonempty source message and exact equality with the reconstructed message. An empty source message returns false without attempting reconstruction, preserving the source short circuit. Malformed descriptions retain the original reconstruction error, while a valid unequal reconstruction gives the original redundancy error. The redundancy trace retains a token/cell pair for each produced occurrence; projecting its cells recovers the exact source message, and its length equals the source length.

The resource record retains the dictionary rows in order, every token cell in order, the reconstructed trace and their complete resource concatenation. Native count identities retain every dictionary row and token occurrence. One retained token reconstructs exactly its registered block; appending any valid token appends precisely its block to the message and one cell to the resource trace. These results establish the source token-successor clause. Independent comparisons cover 1720 reconstruction/resource cases and 6880 lossless/redundancy cases, including duplicate keys, missing tokens, empty blocks, repeated tokens and empty-message behavior. Prefix parsing and the complete fixed/short support boundary remain distinct components of the Compression and Redundancy entry.

99. Prefix parsing and the complete-support shortening boundary

The prefix-family predicate rejects empty families, empty words, repeated words and any distinct pair for which the shorter word is the initial segment of the longer. A native common-initial-segment argument shows that two parses of the same stream must begin with the same admitted word. Cancelling that word and repeating over the finite traces proves unique parsing for arbitrary admitted concatenations. Explicit dictionary token boundaries are retained by the reconstruction representation.

Fixed support retains the complete ordered Cartesian product at the specified width. The shorter support concatenates all products at positive widths below that width, in increasing width order. Native membership and count proofs establish the complete original enumeration and its strict multiplicity-count inequality whenever the raw helper accepts. The pigeonhole argument rules out a pairwise-distinct shorter assignment to every generated occurrence. For set-injective coding of distinct messages, the distinct-label requirement is inherited from the registered Information Quantity complete_word_support domain, rather than added to Compression's executable guard. A native bridge proves that this canonical support is exactly the Compression product and establishes the no-injective-short-code theorem on that domain. The broader raw helper accepts repeated labels; its list-count result is preserved, without treating repeated equal words as distinct messages. The native raw helper therefore preserves repeated alphabet labels and their output multiplicities without an additional distinctness guard.

Compression and Redundancy retains all 1024 kernels and all five operational witnesses. Fresh native execution matches every original program field, including actual candidate lookups, controls and digests. The cumulative 11626 source-expression comparisons cover exact descriptions, errors, prefix families and complete fixed/short supports. These results complete the entry on its registered canonical support boundary, without a universal best compressor, fitted distribution or real-valued

asymptotic rate.

100. Finite channel admission and exact confusability

An admitted finite channel retains nonempty duplicate-free input and output carriers and a duplicate-free relation within those carriers. Every input has at least one image; unused output-carrier entries remain permitted. The native constructor preserves the original order of carrier, repeated-cell, endpoint and totality checks, including their distinct errors. Empty string labels remain allowed.

Output support filters the retained relation by the queried source and preserves every corresponding target in relation order. Native theorems establish exact cell membership, membership within the output carrier, nonemptiness for admitted inputs and absence of repeated targets. A query outside the input carrier retains the original error. Confusability is equivalent to the existence of a retained target shared by both inputs. The overlap is symmetric, and every admitted input is self-confusable because its support is nonempty. No probability or error threshold enters these predicates.

The distinguishable-code predicate retains its original nonempty, unique-input and input-coverage guards and tests every positional input pair. Its native characterization establishes that admitted codes have pairwise disjoint output supports. Every admitted single input forms a distinguishable singleton code. Independent source-expression comparisons pass 3913 constructor cases,600 output-support cases,2400 confusability cases and 6000 code cases, covering malformed and empty carriers, duplicate cells, missing transport, empty labels, repeated held inputs and invalid queries. Complete capacity selection, composition and the fresh-carrier successor are established below.

101. Complete finite capacity families and relation composition

Channel candidate generation enumerates each positive selection size in increasing order and, within that size, every input combination in source-index order. A native selection relation characterizes precisely these retained subsequences. The enumeration theorem establishes exact length, carrier membership and complete generation; distinct admitted inputs yield duplicate-free selections and candidates. Filtering retains exactly the original distinguishable codes. Every admitted input supplies a singleton candidate, so the admitted candidate family is nonempty.

The capacity computation takes the greatest candidate length and retains every candidate attaining it, in the original enumeration order. Native proofs establish the maximum bound, attainment and equivalence between returned membership and being a distinguishable canonical selection at least as large as every other such selection. No tie rule removes an alternative. Admitted capacity is nonempty and contains no repeated records. A one-input channel returns exactly its singleton code family.

Composition requires equality of the ordered interface carriers. It traverses the first relation and then each matching second relation row, keeping the first occurrence of each resulting endpoint cell. Native proofs establish that composite membership is equivalent to a retained two-edge path through the interface, and that admitted channels compose to a total admitted channel. If two inputs share an intermediate output, totality of the second channel supplies a common final output; composition therefore cannot restore that merged distinction.

Independent source-expression comparisons pass 186 combination enumerations,399 complete candidate/capacity families and 6272 compositions, covering all retained ties, repeated raw combination labels, relation-row permutations, first-occurrence duplicates and ordered-interface mismatch errors. The fresh-input successor and complete candidate program complete the Channel Capacity entry as established below.

102. Fresh-input closure of exact channel capacity

Appending a fresh input and its complete nonempty, duplicate-free support within the held output carrier preserves channel admission. Output support for every prior input is unchanged, while the new input has exactly the appended support in its original order. Every prior confusability decision remains unchanged. The new decision with each prior input is precisely the overlap of its retained support with the new support.

Every prior distinguishable code remains valid. Its extension by the fresh input is distinguishable exactly when each prior member has disjoint support from the fresh input. The native exhaustive successor theorem establishes that every generated code is an old code, the fresh singleton, or such a permitted extension of an old code, and conversely every one of those alternatives is generated. This closes arbitrary finite carrier growth while retaining the full candidate family used by the greatest-cardinality selection.

All 1024 channel kernels and all five original operational witnesses are retained. Fresh native execution matches every original registration, census, actual candidate lookup, closure, control and digest field. The cumulative 20505 source-expression comparisons include 735 fresh-input support/code checks alongside constructor, support, confusability, capacity and composition cases. Channel Capacity is complete at the source finite zero-error boundary; probabilistic error rates and asymptotic coding claims remain outside that original boundary.

103. Registered action relations and exact correction

An error relation retains nonempty duplicate-free source and received word carriers, duplicate-free source/image/action triples, and at least one image for every source. Every triple lies within the held carriers and has a nonempty action label. Empty words and unused received entries remain permitted. Distinct actions may connect the same source and image. The native constructor retains the original guard order and errors.

Image and predecessor extraction filters the original action relation, then removes repeated words in first-occurrence order. Native membership equivalences establish that a word is retained exactly when a corresponding action cell exists. Images of an admitted source are nonempty and lie in the received carrier; every retained predecessor lies in the source carrier. Both lists are duplicate-free. An unused received entry has an empty predecessor class, preserving the original domain.

Correction is unique exactly when the complete registered predecessor list is a singleton. The unique record retains that source; every other list, including the empty list, is returned intact as a closed source class. Native proofs establish that a uniquely corrected source has a registered action cell and that any registered predecessor of that received word equals the corrected source. No likelihood, distance rule or inferred predecessor enters the construction. Independent source-expression comparisons pass 7693 constructor cases,738 image extractions,738 predecessor extractions and 1476 correction decisions/records.

104. Complete position traces, detection and action succession

Word comparison requires equal generated position carriers and retains the original error for unequal widths. Complete ordered position pairs are filtered by label inequality, retaining each changed position and both its source and received labels. Positions are rendered with the original one-based position labels. Native indexed membership proves exact correspondence at every position of arbitrary finite words. The resulting trace is structural empty One exactly when the words are identical; otherwise it is the complete ordered changed-position trace.

The detection function retains identity as true. For nonidentical equal-width forms, detection is exactly received-word nonmembership in the valid codebook. No source-membership guard, probability threshold or distance measure is added. Unequal-width errors propagate unchanged.

The identity-action base admits a single source and yields its singleton predecessor class, unique correction and empty-One trace. At each finite successor, appended registered action cells extend the received support by their exact images. Native admission preservation retains the original carrier and cell conditions. Complete image and predecessor membership after extension is equivalent to prior membership or a new matching action cell. First-occurrence order equations retain the original traversal and deduplication behavior. These results cover arbitrary finite action additions, including shared images, without selecting among ambiguous predecessors.

Noise and Error retains all 1024 kernels, all five operational witnesses and both admitted witness relations. Fresh native execution matches every original registration, census, actual candidate lookup, closure, control, hash and unknown-identifier result. The cumulative 18867 source-expression comparisons include 1600 trace,6400 detection and 222 successor comparisons. The entry is complete at its finite structural boundary; stochastic frequencies, physical noise mechanisms and asymptotic reliability remain outside that original claim.

105. Complete coding support and held codebooks

Coding support enumerates the full ordered Cartesian product of at least two distinct labels over a nonempty position list. Its native membership characterization retains exactly every word of the held width whose labels belong to the alphabet. The cardinality is the alphabet size raised to that width, and distinct alphabet labels give duplicate-free support. The original position-list domain is preserved, including repeated position names.

A codebook returns the original held selection, in order, when it is nonempty, duplicate-free and contained in the supplied support. Native admission and membership proofs show that selecting from complete coding support gives every retained codeword the same position width and allowed labels. No additional guard is imposed on the supplied support itself.

Codeword difference retains the complete indexed changed-position trace and both labels at each change. Equality gives empty One; unequal widths retain Coding's original common-carrier error. The general position correspondence follows the established exact Noise and Error derivation.

The repetition constructor maps each supplied label to its constant word over the full position list. Its original helper accepts repeated labels, preserving output multiplicity and order. Under complete support's distinct-label condition, nonempty width makes the repetition map injective and its words distinct; each word belongs to the generated support. Independent source-expression comparisons pass 600 complete-support cases,600 repetition cases,1785 held-codebook cases and 1600 difference cases.

106. Complete single-label error images and exact decoding

Single-label image generation begins with the unchanged source word, then visits every retained position and every supplied label in order, skipping only labels equal to the held label at that position. Native membership is equivalent to identity or to a decomposition into an unchanged prefix, one different supplied replacement label and an unchanged suffix. This equivalence includes both completeness and exclusion of unregistered replacements. Every image has the source width, and generated changes are nonidentity. A source in complete coding support has all its images in that support. The raw helper retains repeated supplied labels and their repeated outputs.

The complete error family retains each source occurrence with its full image list. Detection holds exactly when each image is either its source or outside the codebook. Correction holds exactly when every positional pair of source image families has no common received word. Native equivalences establish these conditions over the full generated lists, including raw duplicate source occurrences.

Decoding filters source occurrences in their original order by exact received-image membership. A singleton gives the original unique-source record; any other length gives the entire closed source class. The native correction theorem proves that disjoint source families yield an empty or singleton filtered list. Consequently, every registered received image of a correctable code decodes to its source. Conversely, every unique decoded source is a retained codeword with that received image in its registered family. No distance or prior is used. Independent source-expression comparisons pass 1600 image lists,465 complete families,465 detection decisions,465 correction decisions and 2790 decoding records.

107. General coding action succession and retained source provenance

A registered error family assigns its complete finite image list to each retained codeword. The general native detection, correction and decoding loops are definitionally the original single-label loops when instantiated with those original images. The singleton-action base supplies one image per codeword; correction then holds exactly when every positional pair has different action images.

Appending an action's registered images preserves the old image list and its order. The detection successor theorem is an equivalence: extended detection holds exactly when both prior and added images satisfy the original codebook-membership rule. The correction successor theorem checks every old–old, old–new, new–old and new–new pair of image supports. Their joint disjointness is equivalent to correction of the extended family. The received source class extends by exactly the sources whose new image lists contain that form. These proofs apply to arbitrary finite registered actions and preserve raw list multiplicity.

The stated total source/codeword relation retains one codeword for each registered source. Native recovery proves that every received image of that codeword in a correctable family decodes to the held codeword while preserving the source row. The same recovery result follows after an action addition precisely when the complete successor disjointness conditions hold. No injective message assignment beyond the stated source-map constraint is imposed.

The original width-three repetition example is a held codebook and its complete single-label image families are disjoint. The one-position uncoded control fails correction; a width-two repetition family also fails, giving the exact finite separation witness for the retained width. Coding includes all 2048 original kernels and all six original operational witnesses. The cumulative 19745 source-expression comparisons include 9375 action-successor detection, correction and decoding outputs. Coding remains within the original finite classical boundary, without a stochastic error model, metric threshold, imported asymptotic rate or quantum-code claim.

Fresh native execution of the complete Coding program matches all 2048 original candidate lookups together with every registration, census, closure, control and digest field and the unknown-identifier rejection. These results accompany the native source derivations and complete the Coding entry on its original finite classical domain.

108. Exact observation incidence and conditional class restriction

Mutual and conditional information begins with nonempty duplicate-free support and an observation assigning exactly one label to every support form. The native validator retains the original support-first error order, observation-key uniqueness and exact support coverage. Admitted dictionary rows preserve insertion order, while class generation visits support order and retains each observed label on its first occurrence. Empty labels and literal empty-One source names remain in the original raw domain.

Native validation equivalence connects this admission to the existing exact observation partition. Each class contains exactly the support forms mapped to its label; classes are nonempty, cover the support and are disjoint for different labels. The complete marginal-label product retains every pair in left-major/right-minor order. Each joint member belongs precisely to the held support and both corresponding observation classes.

Conditional membership first restricts to a given class, then filters by the target observation. Native coverage and disjointness prove that these target restrictions partition every given class, retaining source order and exact provenance. Independent source-expression comparisons pass 637 validations,637 class outputs,5832 joint-member filters,5832 conditional-member filters and 648 complete ordered label products.

109. Exact joint fractions and marginal-product comparison

The joint ledger visits every pair of observed labels in left-major/right-minor order. Each row retains the exact support-ordered joint member list, its normalized part of the whole support, both normalized marginal parts, their normalized product and the original comparison label. An empty joint cell retains the original empty-One presentation and empty-One-below-product relation. Literal empty-One source labels remain admitted.

Native proofs establish positive marginal counts, exact count-ratio values of every present joint cell and both marginals, the product fraction value, and the four comparison cases through exact integer cross-products. Full label projection proves that every marginal-label pair is retained in source order; admitted ledger evaluation preserves left-then-right validation. Independent comparisons pass 1360 complete ledgers,144 guard-order cases and 252 rational comparison cases, bringing the observation and joint-ledger comparison total to 15342.

110. Conditional fractions, resolution and retained dependence

The conditional ledger enumerates every target label inside every given label in the original given-major order. Each row retains the full intersection and its exact normalized ratio to the given class. Native proofs establish the empty-cell criterion, positive intersection count and fraction value, complete ordered label-pair coverage and admitted evaluation. These rows use the previously established conditional class partition.

The executable resolution rule counts, within each given class, precisely the presented member lists unequal to the source empty-One sentinel and requires one such row together with a nonempty target dictionary. This exact raw comparison is retained even for a source form literally named empty-One. Native equivalences characterize that count rule without adding a domain restriction.

Factorization is exactly the conjunction of joint-cell presence and equality with the marginal product across every ledger row. Dependence retains, in original order, all and only the rows whose comparison is unequal, returning the source sentinel when none remain. Native all-cell and retained-cell equivalences accompany 1504 independent comparisons for each of the conditional

ledger, resolution, factorization and complete dependence ledger. The cumulative observation and ledger comparison total is 21358.

111. Fresh-source joint and conditional succession

For a fresh source form, each observation extends by its one assigned label. Native admission preservation keeps total observation coverage and duplicate-free support. The joint member list after extension is exactly the old list followed by the fresh source when both assigned labels match the cell, and exactly the old list otherwise. Thus precisely the designated joint cell receives the new source. Conditional intersections satisfy the same ordered successor equation with given and target labels in their original roles.

The exact joint fraction is recomputed against the extended support count. The conditional fraction is recomputed against the extended given-class count, using the retained intersection plus its possible new member. Native ratio equations establish these updates for arbitrary finite support. The singleton base has one joint member and one conditional member with normalized whole part. Marginal class and label succession use the established observation-extension derivation.

All five original operational witnesses pass native proof: half-parts conditioned on the first coordinate, exact factorization of the complete grid, empty grid dependence, resolution by the copied observation, and nonfactorization with retained dependence cells for the nontrivial copy. Independent fresh-source comparisons pass 1344 each of conditional ledgers, resolution, factorization and full dependence ledgers, bringing the cumulative source-expression total to 26734.

112. Complete mutual and conditional program correspondence

Fresh native execution of all 1024 original mutual and conditional information candidate lookups matches the pinned census, registration, closure, controls and every retained digest, including rejection of an unknown identifier. Together with the exact ledger, decision and arbitrary finite source-successor proofs, this completes the entry on its original deterministic finite domain. No logarithm, prior, stochastic cause or added scalar measure is introduced.

113. Exact conservation and loss of source distinctions

A transformation assigns exactly one image to each member of complete duplicate-free source support. The native validator preserves the original source-first and map-second guard order and errors. Image names remain unrestricted strings. Complete source pairs visit each source form followed by every later source form in the original order.

The transformation ledger classifies each pair by exact image equality. Different images retain the distinction; equal images close it. Native membership equivalences prove that these two ledgers disjointly exhaust the complete source-pair list. Empty ledgers have the original empty-One presentation. The full record retains source support, first-occurrence image support, both pair ledgers and exact predecessor fibres.

For an admitted transformation, a closed pair lies in a common image fibre. Conversely, every source pair whose two members lie in a common image fibre is closed. These statements retain the exact source provenance of loss. Independent comparisons pass 813 validations,121 complete ordered pair outputs and 813 full transformation records, including empty-string and literal empty-One labels, reversed mappings and invalid source/map cases.

114. Composition and downstream retention

Transformation composition first validates the source map, derives its middle support in source-order first occurrence, and validates the second map on that complete middle support. Its output retains one row per original source form in the original order. Native proofs establish exact successive lookup and preservation of transformation admission.

When two source forms have equal first images, their composed images are equal. Every closed upstream pair therefore remains closed. Since the retained and closed ledgers are disjoint and exhaustive, each retained composite pair must have been retained upstream. The native downstream-retention theorem proves this subset relation for arbitrary finite admitted compositions, and establishes that the original executable law returns true. No external record or newly created distinction is introduced. Independent comparisons pass 648 complete compositions and 648 downstream results, including the original validation errors and reordered maps; the cumulative comparison total is 3043.

115. Exact predecessor records and recovery

The reverse record visits the predecessor fibres in their original order. A singleton fibre returns its source label directly; every other fibre returns the original held-predecessor-labels record with its complete ordered member list. Native equivalences establish precisely when each form is produced. Every source with the recorded image equals the unique predecessor in a singleton record. For a held record, membership is exactly source membership together with the recorded image. Two distinct retained predecessors exclude a unique record.

The executable reversibility decision retains V3's comparison of the number of distinct admitted dictionary images with the number of source forms. Independent comparisons pass 813 such decisions and 813 complete reverse records, including original errors and unrestricted string labels. The cumulative transformation comparison total is 4669. The native count-decision equivalence with singleton fibres is established below.

116. Reversibility is equivalent to singleton fibres

For every transformation admitted by the original validator, the executable distinct-image count test returns true exactly when every observed predecessor fibre contains one source form. The native proof retains the original dictionary-value count. First-occurrence deduplication preserves list length exactly when the value list has no repetitions. Unique source keys establish exact correspondence between dictionary values and the images obtained by traversing the admitted source support. Their distinct-image counts are equal, regardless of dictionary row order.

Distinct mapped source values are equivalent to injectivity on the held source support. Injectivity makes each nonempty image fibre a singleton; conversely, singleton fibres identify any two sources with the same image. These implications prove both directions of V3's reversibility statement on arbitrary finite admitted supports, without a new guard or an assumed inverse. The established reverse-record recovery theorems then give direct recovery for singleton fibres and complete retained predecessor alternatives for merged fibres.

117. Source succession and conservation of pair accounting

The singleton transformation has one source, one held image, one singleton predecessor fibre and empty retained and closed pair ledgers. Native evaluation proves the complete original record. Adding a fresh source and one image row preserves the admitted transformation conditions. Every image fibre retains its old members and appends the new source exactly when its label matches the added image.

The complete pair list after extension retains the original source traversal order, adding one pair between each prior source and the fresh source. A retained successor pair is exactly a previously retained pair or a new old/fresh pair with unequal images. A closed successor pair is exactly a previously closed pair or a new old/fresh pair with equal images. These native equivalences establish exhaustive successor accounting for arbitrary finite support.

All five original operational witnesses pass native proof: identity conservation, exact prefix loss, the identity/prefix reversibility boundary, complete held predecessor records and downstream erasure. Fresh-source comparisons pass 1395 validations,15 complete ordered pair lists and 1395 full ledgers. The cumulative source-expression comparison total is 7474.

118. Complete conservation and transformation program correspondence

Fresh native execution matches all 1024 original candidate lookups, registration, census, closure, controls, hashes and unknown-identifier rejection. Together with exhaustive pair accounting, exact composition, downstream retention, reversibility equivalence, predecessor recovery and arbitrary finite source succession, this completes the entry on its original deterministic finite domain. No scalar conservation measure, assumed inverse or unrecorded source distinction is introduced.

119. Classical state, observation classes and deterministic realization

A classical information state retains one member of the complete nonempty duplicate-free support together with that support as provenance. A total observation partitions the same support by its labels in first-occurrence order. Each probability row retains its entire ordered microstate class and the reduced positive fraction of complete support occupied by that class. Native proofs identify the exact class membership, retained order and fraction count equation. Deterministic pushforward constructs corresponding image rows with complete predecessor classes and exact parts.

Observation first requires exactly one row for its observed label. A singleton member class returns its held classical state; every other retained class returns all its members. The native singleton equivalence and unresolved-member identity preserve this boundary. Deterministic realization visits the probability rows in order, pairing every original microstate with the deterministic-member tag while retaining the class label and its exact support part. Source-expression comparisons cover 6296 support, ledger, resolution and realization outputs and error cases, including empty labels and original validation order. Exact-whole closure and the source microstate-successor construction are established below.

120. Complete observation-class cardinality

For every total observation of a finite support, the sum of the full fibre cardinalities over the first-occurrence observed labels equals the support cardinality. The proof counts each source occurrence against distinct observed labels: exactly its own image contributes once. Induction over the support then reconstructs the complete count. No equal-class-size assumption is used.

The raw exact-whole helper retains its Fraction input domain, including signed and zero fractions. Integer cross-multiplication accumulates the exact numerator and denominator; the nonempty ledger is whole precisely when those values are equal. Generated probability and pushforward rows supply their original positive reduced fractions to this calculation.3614 independent source-expression comparisons pass, including 2801 raw fraction lists and 813 generated-ledger or original-error cases. The universal reduced-fraction bridge and exact-whole result are established below.

121. Exact-whole closure and microstate succession

Every generated probability ledger and deterministic pushforward ledger reconstructs the exact whole. The proof tracks the numerator and denominator of the original exact sum through each retained class fraction. Its invariant relates the accumulated numerator to the count of represented microstates over the fixed support count. Reduced class fractions satisfy this relation by their native count equations. Complete class coverage gives the full support count, and cancellation of that positive count establishes equality of the accumulated numerator and denominator. Thus the executable exact-whole test returns true for every admitted ledger.

The singleton base retains its sole classical state with complete provenance, one observation class and one exact whole part. Extending admitted support by one fresh microstate preserves admission. Its assigned class gains exactly that state in source order, every other class retains its members, and each reduced fraction satisfies the updated membership-to-support ratio. The successor ledger again reconstructs the exact whole. These statements apply at arbitrary finite depth and preserve deterministic membership throughout. Full candidate enumeration and complete program correspondence are verified below.

122. Classical and probabilistic operational witnesses

All six original operational witnesses pass native proof: held-state provenance, two equal half classes, unequal three-quarter and one-quarter parts, singleton versus unresolved-class resolution, deterministic prefix pushforward reconstructing the whole, and exposed deterministic-member traces. Complete 1024-candidate program correspondence is verified below.

123. Complete classical and probabilistic correspondence

All 1024 original candidates and six operational witnesses are retained. Fresh native execution matches original registration, complete census, actual identifier-based decisions, closure, controls, hashes and unknown-identifier rejection. Together with the exact class ledger, singleton-only resolution, deterministic realization, universal whole closure and arbitrary microstate succession, this completes the registered finite correspondence. Empirical frequencies retain their separately registered blind-data boundary.

124. Quantum-information branch support and provenance

The support correspondence generates every held-label word across a nonempty canonical position trace. Its original validator requires at least two distinct labels and distinct positions. Native correspondence with the information-quantity construction preserves the full Cartesian order and proves exhaustive word membership and duplicate freedom. Empty string labels remain allowed.

The superposition-equivalent record retains the complete branch family and pairs each word with its full position-label trace. Projecting these records recovers the original branch family in order; projecting each trace recovers both its original word and every canonical position. The amplitude field retains the original not-admitted-at-support-boundary marker.3200 independent comparisons cover

full support/provenance outputs and original errors. Information-unit traces, basis records, joint support and observation/reconstruction are established below; the canonical basis-name correspondence is established below.

125. Basis records and joint support

The basis correspondence retains each original word together with its source colon-delimited basis name. Projecting the retained words reconstructs complete input support exactly, and distinct words yield distinct word/name records. The name field alone is not injective over the original unrestricted string-label domain: labels a and a:a at two canonical positions generate the distinct words (a,a:a) and (a:a,a), both named basis:a:a:a. The accepted support, distinct words and identical names are established by native proofs. Consequently, retained-record correspondence and name-only uniqueness are separate propositions; the latter does not hold over the complete executable domain.

Joint support requires two nonempty duplicate-free word carriers and enumerates every left/right pair in source order, retaining both coordinates and their concatenation. Native membership equivalence establishes complete pair coverage; coordinate recovery preserves both source words even when concatenated values coincide.993 independent output comparisons cover exact basis and joint records, errors, empty words and delimiter-containing labels.

126. Independently realized information-unit traces

Each canonical position retains a four-field record: its position name, the one-Fold-information-distinction tag, the all-first-label baseline word and the word differing only at that position by the second held label. Native proofs place both words in the full generated support, identify the baseline with repeated first labels and establish distinctness of the alternatives for distinct labels. The admitted trace has exactly one row per position and preserves the entire position order at arbitrary finite depth.1600 independent complete-record and original-error comparisons pass, including empty strings and delimiter-containing labels.

127. Exact support observation and reconstruction

Observation retains every branch with the selected image in source order, together with its complete complement and a record of every original branch and image. An empty complement retains the original empty-One marker. Native membership proofs establish disjoint and exhaustive accounting of retained and closed alternatives. Projecting the complete reconstruction record recovers the original support in order. On canonical nonempty support, reconstruction accepts this record and returns exactly the pre-observation support.

The raw observation helper preserves its original domain, including repeated support words when the observation keys cover that support set exactly. Reconstruction separately rejects repeated branch keys, as in the source.4660 independent comparisons cover full observation records, reconstruction outputs and original errors, including empty words, empty labels, repeated support, reversed mapping order and literal empty-One branch labels.

128. Canonical position succession

Adding one fresh canonical position preserves support admission and extends every prior branch by every held label in the original Cartesian order. The extended support remains duplicate-free. Each branch provenance is exactly its previous position-label trace followed by the fresh position and chosen label; a complete information-unit trace exists for the extended support. The first-position construction retains all held labels, with the original two-label instance producing its two branches. All six original operational witnesses pass native proof on their fixed source inputs. Name-only bijection holds throughout the registered fixed-label Fold support, as established below.

129. Quantum support program correspondence

Fresh native execution reproduces all 1024 original candidate lookups and all registration, census, closure, control, hash and unknown-identifier outputs. All six operational witnesses hold for the original fixed held-a/held-b support. These program results retain the source declarations verbatim; they do not establish name-only injectivity for arbitrary accepted string labels. The delimiter collision refutes name-only injectivity over arbitrary accepted string labels. The registered Fold Assembly generator fixes its labels to held-a and held-b, and the quantum grammar refers to the two held Fold labels. Native proofs establish name-only injectivity and the unique inverse on this generated domain at every finite depth; the arbitrary-label collision is outside that registered grammar and remains faithfully represented in the raw helper behavior.

130. Canonical Fold basis bijection

The registered Fold Assembly generator supplies precisely held-a and held-b. Quantum support with these inherited labels contains exactly the words in Fold Assembly at the same finite depth. The original colon-joined basis names uniquely identify these words: the native proof establishes exact character decoding, cancels the basis prefix and recovers the held-label sequence. Hence all emitted names are distinct and every emitted name has one and only one source word. This result holds at arbitrary finite depth and preserves the original executable functions on their broader raw input domains. Together with the unit, provenance, pair-support, observation, reconstruction and succession proofs and the complete 1024-candidate program correspondence, it completes the registered quantum-information support entry.

131. Exact generated transition systems

A transition system retains the ordered generated state carrier, held action labels and source/action/target rows. Construction first checks state uniqueness, then action uniqueness and nonempty carriers; it then checks each row in order for carrier membership and repeated source/action coordinates. The relation may be partial. A requested step succeeds exactly when its matching target list is a singleton, preserving the original missing-transition boundary.

Execution first checks that the initial state belongs to the carrier, then retains that initial state and every successive generated target, propagating the first failed step. Native proofs identify successful execution exactly with an inductive chain of the original step results. Every admitted trace begins at its registered initial state, has one position for every action plus the initial position, and remains entirely within the state carrier. Constructor admission is equivalent to the exact original carrier and ordered-row conditions.3397 independent comparisons cover complete outputs and original errors, including duplicate states/actions, empty strings, missing transitions, repeated coordinates, escaped rows and failure order.

132. Retained execution composition and extension

Every successful execution admits an exact decomposition into its retained predecessor positions and terminal state. Two such paths with a common terminal/initial interface compose by concatenating their action traces and predecessor records, retaining the shared state exactly once. An empty action trace is the identity path; appending one generated action retains the complete previous trace and appends its generated target. These statements hold for arbitrary finite executions.

For every admitted system, a requested transition succeeds if and only if its exact source/action/target row belongs to the retained relation. Original constructor validation excludes any second target row at the same coordinate. Consequently, an admitted extension that retains the earlier state carrier and transition rows preserves every earlier successful execution with its exact trace. This applies to appended carriers and rows and to both components embedded in a common admitted assembly. Compatibility is decided by the original constructor; no additional transition or scheduling rule is imposed.

133. Generated words and exact coordinate rewrite

The generated-word operation retains the original empty-One marker and then every complete word layer up to the given finite position depth, preserving both layer order and Cartesian order. Native induction identifies the exact frontier at every depth and proves that one new position appends the entire next layer. The original unique-alphabet, unique-position and nonempty-alphabet conditions remain unchanged; the literal empty-One symbol remains allowed.

Exact rewriting validates the unique position trace, equal word/position lengths and selected held position. The output retains the full position trace, changes the selected coordinate to its replacement and preserves every other coordinate and total length.2952 independent full-output/error comparisons and all three original formal-computation witnesses pass. Fresh native execution matches every original 256-candidate lookup, registration, census, closure, control and digest field. These results complete Exact State and Transition Law on its registered generated-finite boundary; no language, machine model or resource bound is introduced.

134. Finite language derivation certificates

The language formation rules retain the empty construction frontier and append a held alphabet symbol at each finite successor. Native derivation trees are sound and complete for the exact generated word layer: a word belongs to that layer if and only if it has the corresponding finite derivation certificate. The certificate fixes its depth, admits only symbols in the generated alphabet and retains a recoverable predecessor at each successor. Concatenating two derivations produces the concatenated word at the summed construction depth.

For the complete cumulative language output, membership is exactly the original empty-One marker or a positive-depth derivation bounded by the registered position trace. The marker and the empty host frontier remain distinct, and literal marker-valued alphabet symbols retain their original behavior. The shared native operation proofs and all three claim-specific witnesses combine with full 256-candidate program/output/hash agreement to complete the registered Fold Formal-Language and Grammar Law. No completed infinite language or conventional grammar model is introduced.

135. Finite recognition with retained accepting classes

The finite-recognition kernel retains its generated states, input alphabet, original transition relation, initial state, accepting class and complete run. Its accepting-class statement is represented by a native predicate requiring the original successful execution and a terminal state held in that class. This predicate is proved equivalent to the retained-path representation; it does not introduce a new executable source method.

Accepted traces preserve the initial state, full trace length and admitted input alphabet. The identity trace accepts exactly when its state belongs to the accepting class; an empty class admits no accepted trace. Matching paths compose with every trace position retained, and an admitted carrier/transition extension preserves earlier accepted traces. A one-symbol successor retains its prior trace and uses the original generated target. All results hold at arbitrary finite depth. Full 256-candidate program, witness, control and hash correspondence completes the registered Fold Finite Automaton Law without an equivalence-to-all-models or universality assertion.

136. Retained held-position rewrite traces

The exact coordinate rewrite retains its original position validation and output behavior. Replacing the same held position twice has precisely the effect of the second replacement; repeating an identical replacement is idempotent. Replacements at distinct held positions commute. Native proofs establish these equations for arbitrary finite words with the original matching position trace. An admitted rewrite permits another replacement at the same position without changing its admission conditions.

Retained step records associate rule identity, source word, held replacement position, replacement symbol and successor word. Replaying a record uses the unchanged exact rewrite operation. Each valid record preserves word length and carries the complete selected/unchanged coordinate correspondence. Finite chains compose at their exact successor/source interface, preserve length and retain a replay proof for every recorded step. These typed records express the declared provenance fields; they do not add an executable operation to V3.

137. Registered rewrite boundaries and trace classes

The finite relation of registered rewrite applications retains complete records, including rule identity, source word, selected position, replacement and successor. Admitted applications belong to this retained relation, have both endpoints in the generated carrier and replay the original exact rewrite operation. Every admitted application has an exact source-symbol occurrence at its held position, a source-word decomposition around that occurrence and the replacement at that same coordinate. Missing registrations and endpoints outside the generated carrier are excluded by the declared boundary; the raw executable helper keeps its original guards.

A terminal word has no admitted outgoing application. Consequently its retained traces are empty and it cannot begin a cyclic or branching trace. A cyclic trace retains a nonempty return path, which can be repeated by exact composition. A branching trace retains two admitted applications from the same word with distinct successors; each yields its own complete one-step trace. Identity, serial composition and one-record succession preserve the retained relation. Extending the registered application relation and generated carrier preserves all previous traces. These predicates formalise the declared finite trace classes; no claim of universal termination or confluence is introduced. Full 256-candidate native program agreement and the original three operational witnesses complete the registered Rewriting entry.

138. Structural finite recursion with complete values

The recursion operation validates unique successor-position identities, then retains the supplied base and every value produced by its exact step relation. Empty bases, empty successor traces and empty string labels retain their original behavior. Native derivation chains agree exactly with the executable result and uniquely determine all intermediate values. The full trace has one value per successor plus the base.

At every finite depth, appending a generated successor appends exactly its step result and preserves the complete earlier trace. The terminal value supplies the base interface for subsequent recursion. Composition concatenates predecessor records with the second complete trace, retaining the shared interface once; its admission uses the original distinct-position check on the combined trace. These results establish structural-One base, arbitrary finite successor, exact intermediate-value provenance and terminal composition without a completed transfinite object.800 independent output/error comparisons and all 256 original native program comparisons pass, completing the registered Fold Recursion Law.

139. Fold binding and self-application calculus

The registered FORM-LAMBDA-CALCULUS entry derives the generated Fold kernel from admitted finite forms, exact relations, distinction ledgers and composition. Its complete product contains 256 combinations of eight structural requirements. Native exhaustive proofs establish that precisely the complete carrier, canonical held identity, source-bound exact relation, complete trace, explicit boundary, interface-preserving composition, structural successor and no-extra-premise combination survives. Generated term grammar, held binder identity, free and bound ledgers, capture-safe substitution, exact reduction and normal or cyclic trace retain their registered structural meaning.

The entry's source-bound operational kernel is the exact transition system, generated words and held-position rewrite. Successful execution reconstructs every result from its initial form and actual retained relation rows. The structural identity trace has no predecessor. Appending a permitted step preserves the entire earlier trace and adds its exact successor. Matching interfaces compose without discarding intermediate records, and admitted extension retains previous runs.

Normal traces have no successful outgoing registered step; their identity execution is retained and they cannot form a nonempty cycle. Cyclic traces retain a nonempty return path, replay through the exact source execution, and compose with themselves while retaining both copies of the trace. These native results hold for arbitrary finite paths. No conventional beta-reduction model is imported.

All 256 original program decisions, three operational witnesses, controls, registration, closure and hashes agree with the pinned entry. The registered Fold Binding and Self-Application Calculus Law is complete within its original boundary. Source: claims/SFT-COMP-FORM-LAMBDA-CALCULUS-001/execution.py and its six-file source manifest; current computation publication v1.5.1 section 14.

140. Separate FORMX source components

The later FORMX source defines variable, application and abstraction tuples, free-name membership, whole-tree renaming and substitution. Their native translation preserves the exact finite fresh-name search and exhaustion behavior. Structural induction establishes the application and abstraction execution equations at arbitrary term depth, including mapping composition and error propagation.2052 source-expression comparisons supplement these equations. These translated helper components belong to the separately registered FORMX entry; its completion is not claimed here.

141. Finite executable descriptions and halt traces

The Abstract-Machine entry retains generated description and store words, its configuration carrier, exact transition rows, initial configuration and complete execution trace. Its word boundary is precisely the original generated-word support, including the empty-One marker and all finite admitted words through the declared positions. Held-position replacement preserves the exact generated store coordinates. Machines outside the frozen description grammar remain outside the result.

Native induction and execution correspondence establish uniqueness of the complete trace and membership of every configuration in the admitted carrier. A retained halt trace records its terminal no-outgoing-step boundary separately from exhaustion of requested actions. It replays through the original source run, contains one predecessor configuration per action, admits no nonempty continuation, and composes with a preceding path at the held interface. The identity halt is its base. Existing exact successor and extension results preserve every prior trace record.

The complete 256-candidate structural derivation, original three witnesses, four controls and every native program field and digest agree with the pinned entry. These results complete the registered Fold Abstract-Machine Law within its original description boundary.

142. Circuit interfaces and complete evaluation traces

Finite connected local transformations retain the generated node/wire carrier and typed interfaces supplied by the admitted Graph and Category dependencies. The native evaluation relation records successive successful original Category compositions. Induction proves complete ordered concatenation of all local trace rows, preservation of the initial interface, rejection of interface mismatches, serial composition and arbitrary finite successor extension.

When component interfaces are registered graph nodes and their local transformations are registered wires, evaluation supplies an original complete graph path and its terminal output remains in the node carrier. The original cycle detector is equivalent to a complete retained return path. Its false result excludes every such return; declared cycles retain their full path evidence. No external gate model is imported.

The full 256-candidate structural derivation, original operational witnesses, controls and complete native program agree with the pinned entry. This completes Fold Circuit Syntax and Evaluation within its generated-finite boundary. Resource size and depth comparisons remain with the source Complexity entries.

143. Operational process observations and failure traces

An admitted operational process retains its initial generated configuration, exact enabled transition rows, requested choices and full configuration trace. Successful and failed observations remain the original source outputs. Native failure records are equivalent to the original execution error branches at arbitrary finite depth. Each record decomposes the requested choices into a successfully executed prefix, the failing action and its unexecuted suffix. The complete successful prefix replays exactly up to the failure configuration.

A recorded failure excludes a successful result on the same execution. Prepending a successful path preserves every prior choice/configuration and propagates the same original error. Terminal processes retain their explicit no-outgoing-step boundary; cyclic processes retain a nonempty return path that replays and composes without losing records. These results use the unchanged source transition system and its admitted finite trace operations. No process algebra, distributed timing or stochastic selection is introduced.

All 256 original grammar members and native lookup decisions,3 operational witnesses,4 controls, registration/closure records and hashes agree with pinned V3. These results complete the registered Fold Operational-Process Law within its original generated-finite boundary.

144. Process composition and exact decomposition

Every retained serial execution decomposes at any declared cut in its action trace. Native induction reconstructs the intermediate configuration and both complete component paths; their predecessor records concatenate to the original trace. Conversely, exact interface composition reconstructs that original path. Each component replays through the unchanged source execution kernel.

Independent parallel records retain both systems, their initial configurations, action choices and original observations separately. Component execution reconstructs the observed pair, and unchanged component records determine unchanged observations. The exact source Category interface rule retains both local traces and rejects mismatch. These records add no shared state, timing, communication or scheduler. Concurrency involving communication or shared causality remains downstream as in V3.

The original 256-candidate grammar, witnesses, controls and complete native program agree with the pinned source. These results complete the registered Fold Process Composition and Decomposition Law within its original boundary.

145. Registered step-to-trace simulation

The model-equivalence construction retains a translation between the two registered generated grammars. For a translation satisfying the source step-or-trace simulation condition, native induction constructs a complete target path for every finite source path. The translated terminal interface is preserved, and the original target execution replays the complete generated path when the translated initial configuration belongs to its carrier.

The overhead ledger retains one complete target-action segment for each source step. Flattening concatenates all generated target actions in order; appending a source step appends exactly its target segment. No constant overhead or one-target-step restriction is introduced. Identity simulation and composition of successive simulations preserve the exact path relation. Registered finite encoding and exact reverse decoding connect these traces to the held model descriptions.

146. Finite registered model translation and reconstruction

The description translation composes the original finite encoding lookup with the target description decoder. Validated tables and their exact decoders provide total lookup on each registered carrier. Shared used-code coverage supplies a target description for every source description and derives its exact reverse reconstruction. Uniqueness follows from the unchanged lookup and decoder operations.

For an admitted original system, every successful native execution reconstructs its complete path and terminal carrier membership. The registered step-or-trace simulation constructs the corresponding target execution and its complete action-segment ledger. The finite-table reconstruction then decodes the translated terminal back to the original terminal. Applying this construction in each direction establishes the registered bidirectional execution correspondence under the two supplied simulation conditions. Equivalence retains V3’s boundary of the two registered generated model grammars.

All 256 original candidates, native lookup decisions,3 witnesses,4 controls and complete program fields and hashes agree with pinned V3. Together with the native encoding, trace and reconstruction

results, this completes the registered Fold Computational-Model Equivalence Law.

147. Interpretation of frozen generated descriptions

A frozen description retains its complete generated transition system, initial configuration and action input. The original finite encoding and exact decoder recover that held request. A uniform interpretation adapter then applies the unchanged source execution kernel. For every admitted description, native proofs establish exact agreement with its machine execution, recover the complete path and terminal from every successful observation, and preserve the original errors. Failed decoding or absence from the frozen registry supplies no interpretation. Universality ranges over descriptions generated by the registered grammar at finite named depths.

Resource records retain the original numbered observation trace and occupied support. The initial observation is counted: a successful execution records one more observation than requested actions. Native proofs reconstruct every recorded observation and occupied label. The same original base, successor and composition operations preserve complete traces and terminal results under interpretation; no external machine model or fixed resource overhead is imposed.

All 256 original grammar members and full program outputs match pinned V3.36 additional fresh native resource comparisons cover guard precedence, repeated observations, Unicode and position labels. These results complete the registered Fold Universal-Computation Law.

148. Recognition, decision and unresolved execution

The generated description ledger retains every admitted description in order under its original numbered position. Its original duplicate and nonempty guards remain exact. Held acceptance and rejection labels complement each other; other labels are excluded by the original complement operation.

A decision retains a complete original execution path terminating at an accepting or rejecting label. An unresolved record retains a finite path whose current endpoint carries neither answer. Both replay through the unchanged execution kernel. An unresolved prefix supplies no answer and does not establish whether a later prefix will terminate. A generated one-state loop admits arbitrary finite unresolved prefixes without a terminating decision.

Total decision applies to every registered input, retaining its initial configuration and exact action input. Native proofs reconstruct its complete terminating observation and establish the successor decomposition of the domain obligation. Terminating decisions exclude continuation at their terminal boundary. These results preserve the source distinction between partial recognition and total decision without importing a conventional machine model or completed infinite trace.

The full 256 original computability grammar and complete native program agree with pinned V3.13 additional fresh native comparisons cover exact enumeration, guard order, held complements and excluded labels. These results complete the registered Fold Recognition and Decision Law.

149. Terminal witnesses and the held halting boundary

A halting witness retains the complete finite execution path and an explicit terminal boundary. Native reconstruction yields the unchanged source observation trace, including its terminal state. The original held complement operation exchanges acceptance and rejection and has no fixed point on those labels. An admitted generated stop process halts on its identity trace; an admitted generated self-loop has no terminating finite path. These processes realize the two complementary terminal behaviors within the original transition kernel.

The contradiction certificate retains the exact decoded self-description, selected registered process and prediction at that same code. A correct prediction and a process realizing its held complement cannot both hold. When total prediction includes a supported registered self-description and its complementary execution, instantiation at that description produces the contradiction. This result preserves the declared self-description support and does not supply a halting decision for all generated universal-machine descriptions.

The complete 256-candidate grammar and every native program output field and hash agree with pinned V3. These results complete the registered Fold Halting Boundary Law.

150. Effective finite enumeration and reconstruction

The original enumeration retains every admitted description in order under its canonical numbered position. Native positional reconstruction recovers that exact label and description at every position in the finite support; positions beyond the support supply no row. The original distinct-description guard excludes duplicates, and projection recovers the entire admitted description sequence.

The one-description base has its original first position. Arbitrary finite block extension retains every prior row and offsets the new block by exactly the earlier support length. An admitted single-description successor appends its next canonical row without changing any predecessor. These statements apply at arbitrary finite generated depths and install no completed infinite enumeration object.

Each retained description links through the original finite encoding and exact decoder to its complete registered machine request. Interpretation reproduces the original execution with the same initial configuration and action input. Enumeration itself supplies no additional termination or answer claim.

All 256 original candidates and every full native program field and hash agree with pinned V3. These results complete the registered Fold Effective-Enumeration Law.

151. Explicit reductions and answer reconstruction

A registered reduction retains its source and target domains, finite input encoder, target procedure and answer decoder. Original validated encoding tables and exact decoders establish total translation on the held source carrier. The target procedure retains the complete original request, execution trace and terminal boundary; decoding its answer uses the original finite lookup and decoder operations.

The registered bidirectional correctness trace links source answers to decoded target answers. Native forward reconstruction yields the full target run and source answer decoding, while reverse reconstruction recovers the source answer from that target result. Encoding alone does not introduce an answer-preservation premise.

Composition retains the intermediate input and both input translations. The final answer decodes through the second reduction and then the first, retaining both intermediate and source answers. Native proofs establish both directions of this correspondence and preserve both generated-domain witnesses. The identity law preserves both the exact input and the decoded answer; finite successor laws retain the original table-validation boundary.

All 256 original candidates and all complete native program fields and hashes agree with pinned V3. These results complete the registered Fold Computability-Reduction Law.

152. Finite relative computability classes

Registered problems and their certified reductions form an original generated finite graph. Its exact reachability operation supplies complete retained paths. Every reduction step on such a path retains its finite encoders, decoders and bidirectional correctness certificate. Native path composition supplies transitive closure without replacing the intermediate interfaces.

Mutual reachability is reflexive on registered problems, symmetric and transitive. A degree class retains exactly the mutually reachable members of the original ordered problem carrier. Each representative belongs to its class; mutually reducible representatives give the same members, and overlapping classes imply mutual reducibility. These statements require no quotient axiom.

Strict one-way reachability is transitive and excludes mutual equivalence. A finite incomparability certificate excludes complete paths in both directions. Strict order and incomparability are independent of the chosen equivalent representatives. The singleton One supplies the base class, and retained nodes and edges preserve every prior mutual reachability witness. Degrees remain finite generated relation classes, with no completed transfinite hierarchy.

All 256 original candidates and every complete native program field and hash agree with pinned V3. These results complete the registered Fold Degrees-of-Computability Law.

153. Universal-machine execution and recognition

The frozen description grammar retains exact machine requests: the original generated machine, its initial input and every requested action. Native finite encoding and decoding reconstruct each admitted request. Prefix-free description streams have a unique complete parse. Interpretation returns precisely the original machine execution, preserving its complete trace and original errors.

A terminal execution retains its output and excludes further transitions. Conversely, a successful interpreted trace with its explicit terminal boundary reconstructs the original halt witness. Accepted and rejected answers retain their exact labels. A continuing finite trace remains distinct from a terminal decision. The original generated loop admits arbitrary finite continuing prefixes while its registered input is not a total decision domain. These results preserve the stated recognizability boundary without an omniscient halting oracle.

The identity trace supplies the base. Serial execution retains the intermediate state and concatenated action and trace rows. Retained decoding and registry selections preserve all prior interpreted observations; every newly admitted encoded request is covered by the same finite grammar theorem. All 256 original candidates and complete program fields and hashes agree with pinned V3. These results complete the registered Fold Universal-Machine Execution Law.

154. Generated diagonal contradiction

The registered undecidability construction applies on languages supporting exact self-encoding and complementation. The native diagonal constructor applies the original complementary-verdict operation and realizes its result in a generated machine with its complete identity-halt trace. Both held verdicts are supported, and the original failure boundary is retained.

Self-application retains the same generated code, decoded description and selected request used to query the proposed decider. Totality supplies a held answer on that description. The exact diagonal constructor supplies its complement and the corresponding native interpreted execution. Correctness would equate the proposed answer with that observed complementary answer, contradicting the proved absence of a complement fixed point. Thus a total correct internal decider is excluded on the

explicitly registered self-encoding/complement domain.

The contradiction retains each decoding, selection, construction and execution witness. Exact retained decoding and registry selections preserve self-support under extension. The identity trace supplies the base; original path composition preserves intermediate interfaces and complete action/trace records. All 256 original candidates and complete native program fields and hashes agree with pinned V3, completing the Fold Undecidable-Problem Law.

155. Computational proof-system boundary

A checked proof record retains the original inference checker, each registered premise and rule, the complete accumulated proposition state, and its original consistency test. Native reconstruction yields every step of the accepted trace. A record containing both a proposition and its denied form fails the consistency condition. Original finite enumeration retains each positional proof name and the selected complete proof-step list.

For the registered self-encoding and complement domain, an answer-correct internal verifier has a specific missing decision: its decoded self-description cannot receive a held answer without reconstructing the native diagonal contradiction. Hence complete internal decision is excluded. The result also applies to answers backed by checked proof records when those records preserve the corresponding execution answers. This is the declared computational proof-system boundary; it does not import arithmetic incompleteness or an external truth oracle.

The identity proof preserves a consistent initial state. Complete traces compose at their actual accumulated proposition interface, with exact final-state reconstruction and the retained consistency condition. Original enumeration and self-description extension preserve prior records. All 256 original candidates and full native program fields and hashes agree with pinned V3, completing the Fold Incompleteness-Boundary Law.

156. Declared relative answer interfaces

An oracle interface retains its named query alphabet and declared query/answer relation. Each answer record retains the selected source identity and exact query and answer. Native provenance reconstruction proves that every record belongs to the selected declared relation; undeclared source identities and answer pairs are excluded. The relation is retained without imposing a new single-valuedness condition.

Finite ordered ledgers retain nested queries and subsequent records. Validity at a nested call is equivalent to validity of its named outer record, nested ledger and continuation. The base computation retains its original native execution path and held initial state alongside the complete external ledger. Replay reproduces that exact base execution and every declared external source; terminal recognition preserves the original accept/reject classification.

The identity path and empty external ledger supply the base. Serial composition preserves the intermediate state, concatenated actions and complete ordered answer record. Registry extensions retaining earlier interface selections preserve all prior nested certificates. These results retain the registered oracle boundary without unexplained computational power. All 256 original candidates and complete native program fields and hashes agree with pinned V3, completing the Fold Relative and Oracle-Computation Law.

157. Hypercomputation admissibility boundary

A claimed computation retains its generated operational transition system, complete native execution, literal resource record and declared external-answer ledger. Native reconstruction recovers the entire transition path, occupied support, observation count and provenance of every external answer. A differing observed output, failed execution or undeclared oracle source excludes the submitted record.

The claimed input domain is generated at arbitrary finite depth. An explicit certificate retains the base and successor obligations; native induction establishes verification across that domain. A refuted generated record excludes such a certificate. Accepted finite samples remain conditional: a refuted domain record cannot be among their verified members and prevents complete-domain verification. The original self-encoding/complement boundary remains applicable to internal total prediction.

These are admissibility conditions, preserving the source requirement for an executable law and depth-independent certificate. They introduce no unexplained computational power. Retained domain extensions preserve prior records. All 256 original candidates and complete native program fields and hashes agree with pinned V3, completing the Fold Hypercomputation Admissibility-Limit Law.

158. Exact generated input extent

The canonical encoding retains an exact codeword and decoder reconstruction for every held source input. Complete position cells retain occupied-position identities together with every encoded symbol. Their full symbol projection reconstructs the codeword. The original resource ledger retains its unique, nonempty occupied-support boundary; native reconstruction gives the full trace and exact position count.

Two complete records of the same codeword have equal size. A representation correspondence retains both actual encoding-table rows and both exact codeword lengths; unrelated representations are not silently assigned one invariant size. Every instance in a certified family retains its actual input and encoding record.

The singleton position supplies the base. Complete concatenated encodings have the exact sum of their recorded extents, and an admitted one-position extension increases the count by one. All 256 original Complexity candidates, the three original resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Input and Problem-Size Law.

159. Time and retained-state resources

The original resource ledger reconstructs the complete ordered observation trace and exact occupied support. Its time count includes the initial observation. A terminal resource record retains the full halt trace, its exact action-plus-initial count and the exclusion of further transitions. These are exact host reports over generated traces.

Every finite nonempty sequence of configuration ledgers has an attained peak space-count class. Native reconstruction connects each measured bound to its actual occupied support. Equal complete time records recover equal histories; the original generated loop exhibits different histories ending at the same label, demonstrating why the terminal label alone does not suffice for recovery.

Ordered ledger composition retains shifted transition labels. An admitted observation/support successor preserves prior rows and adds one to each count. Original serial path composition counts the shared intermediate observation once. All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Time and

Space Resource Law.

160. Circuit structural resources

The declared dependency order is exhausted by the original ordered-selection generator. Filtering those selections with the original graph-path check retains exactly the admissible ordered causal traces. Native maximum selection returns an actual longest trace whenever this domain is nonempty, and its length bounds every admitted trace in that dependency order. Each selected gate and adjacent wire is reconstructed from the registered graph.

Original graph admission preserves unique gate and wire identities and held endpoints. Gate size counts the canonical gate carrier, so changed wire incidence does not duplicate a shared component. Original circuit evaluation retains complete local traces and interfaces and establishes output support. The singleton gate supplies the base. Carrier extension, wire composition and dependency-order extension preserve their exact records and counts.

These statements close resource accounting within the declared circuit boundary. All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Circuit Size and Depth Law.

161. Communication and query resources

Each query and answer record retains its registered interface identity. The ordered sent and received projections have the same exact cardinality as the source query ledger. Pairing those projections reconstructs every original query/answer record, and native proofs recover its declared alphabet and answer-relation membership. Serial composition retains both transcripts and adds their exact query counts; the singleton and successor constructions preserve every prior record.

The result certificate binds the decoded protocol identity to its registered executable request, its declared distributed input, the original protocol execution, the named oracle ledger and the local resource support. Its native proof reconstructs the actual transition path and complete terminal observation trace, with the exact action count and unchanged local knowledge. Unknown protocol or oracle identities are excluded by the original registry lookup. The declared oracle relation is retained without imposing single-valuedness or introducing an answer-producing premise.

All 256 original candidates, three resource-ledger witnesses and all native program fields and hashes agree with pinned V3, completing the Fold Communication and Query Complexity Law.

162. Scheduled unresolved branch resources

Every registered branch retains its label and complete deterministic execution request. Native schedule evaluation applies the original transition process to every branch, preserving order, multiplicity, successful traces and failure records. Successful runs reconstruct their actual paths and terminal observations. The terminal-output classes cover the complete declared branch support and are pairwise disjoint; failures remain explicitly classified.

A registered schedule description retains its source identity and exact encoding/decoding relation. Its complete encoded-position ledger supplies the exact description cost. Ordered composition and successor extension preserve every earlier execution record. The external-source alternative retains the original sealed measurement boundary, including complete rows and identified source; randomness is not introduced as a causal premise.

All 256 original candidates, resource witnesses and complete program fields and hashes agree with pinned V3, completing the Fold Randomness-Resource Law.

163. Information retained for inverse execution

For each registered action and target configuration, the complete transition fibre contains exactly the held source configurations whose original transition reaches that target. Distinct merged predecessors cannot be represented by a unique predecessor record. Erasing their distinguishing labels prevents a single recovery function from recovering both.

Induction on the original execution path constructs a retained inverse ledger with one source/action/target row per transition. Every row remains an actual forward transition. Native inverse traversal checks the shared configurations and original transitions and reconstructs the initial configuration. Identity, serial composition and successor extension preserve all rows. The original resource ledger reconstructs the full inverse observation history, with one initial observation plus one per retained transition and unchanged occupied support.

This closes the classical information-retention cost within the registered scope. All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Reversibility-Cost Law.

164. Independent and causally layered work

The native ready-set construction retains exactly the declared tasks not yet completed whose incoming predecessors are all completed. This condition proves that simultaneous ready tasks have no dependency edge between them. The original ordered-selection generator exhausts every layer choice over that ready set. Every synchronization row is an original incoming graph edge, and its predecessor belongs to the completed interface.

The layer schedule reconstructs the entire ordered work trace, preserving every task and synchronization interface under serial composition and successor extension. Full source coverage is transferred to the work trace without imposing the graph-carrier order on execution. The complete original causal-path enumeration and native maximum produce an attained span bounding every admissible causal trace in the declared execution order. Original resource ledgers retain exact work and support counts; no physical processor count or timing constant is introduced.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Parallel-Complexity Law.

165. Finite resource comparison certificates

Constructive upper certificates retain the registered executable request, its complete native transition trace and original resource ledger. The resource measure selects the original observation count or occupied-support count. Native finite induction constructs an attained maximum over the complete declared family, retaining an actual verified member and proving that its cost bounds every member.

An adversarial lower witness retains two problem-family inputs with identical registered observations and different required answers. No response depending on that observation can correctly answer both. A lower certificate supplies such a witness for every below-bound member of its declared comparison family, excluding every such member from correctness. Problem-family extension and comparison-family composition retain the complete witnesses. Arbitrary asymptotic lower bounds remain separate claims with their own generated families.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Lower- and Upper-Bound Law.

166. Resource-preserving reductions and completeness

A verified reduction retains its complete source domain, total encoded input translation and bidirectional answer decoder. Every source input has a registered executable overhead record satisfying the frozen resource law. Native reconstruction recovers its original observation history, occupied support and exact counts. Across stages, every trace is retained and the sum of verified observation costs equals the length of the full concatenated trace.

Reduction chains preserve each original mapping, intermediate description and overhead witness under the same resource law. Hardness supplies a retained chain from every member of the frozen problem inventory; completeness additionally requires membership of the target. Composition transports hardness while retaining the full chain. Inventory extension requires the new certificate and preserves all prior certificates; the identity chain supplies the singleton base. Completeness remains relative to the frozen problem class and resource law.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Reduction and Completeness Law.

167. Complete instance-family resource profiles

Each generated support cell retains its instance assignment and original resource record. Canonical-support validation, exact cell coverage and source lookups preserve distribution provenance and the explicit multiplicity of repeated instances. The exact aggregate sums every cell cost. Original positive-fraction normalization divides that sum by the full support-cell count, and native proofs establish exact value preservation without floating arithmetic or fitted weights. Original event weighting retains the exact support parts and its admission boundary.

The greatest-resource class contains every tied maximum. Native finite proofs construct a verified greatest member, bound every record and bound the aggregate by the complete support count times that resource value. Composition and successor extension preserve all costs and cells. The empirical-distribution alternative retains its original complete-row, identified-source and sealed measurement obligations.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Average- and Worst-Case Law.

168. Exact approximation and retained objective distinction

The approximation construction retains the registered optimization carrier, preference relation and positive generated objective-cell traces. Native proofs derive the optimum bound from absence of a better incoming preference row and preserve equality of the objective values of tied optima. The exact approximation ledger uses the original ratio of optimum cells to candidate cells. Positive-fraction normalization preserves its value by exact cross multiplication.

The output certificate retains the original computation trace, time ledger and occupied support. Missing objective rows and invalid lower optima preserve the source failure boundary. Carrier extension preserves a retained optimum under the original absence of a better preference row, and the singleton construction supplies its base. No unregistered metric or irrational ratio is admitted. All 256 original candidates, resource witnesses, controls and native program fields and hashes agree with pinned V3, completing the Fold Approximation-Complexity Law.

169. Resource stratification by structural coordinates

A named structural coordinate is extracted from the retained fields of each declared input. Native proofs establish both source-row provenance and uniqueness of the extracted coordinate. Total extraction yields a complete family partition: every held input belongs to its extracted coordinate class, and classes with different coordinates cannot share an input. Missing coordinates fail the extraction condition. The construction introduces no fitted constant.

Every family member retains its original verified resource record, complete time trace, occupied support and exact counts. An answer-preserving reduction that preserves the extracted coordinate maps a source family into the corresponding target family while retaining its input encoding, answer decoding and verified resource overhead. Two-stage coordinate preservation and full-family composition, singleton and successor proofs retain all source rows. All 256 original candidates and complete native program records and hashes match pinned V3, completing the Fold Parameterized-Complexity Law.

170. Shortest exact descriptions at declared finite depth

The frozen alphabet generates every nonempty description through the declared depth. Filtering by the original decoding relation retains exactly those descriptions that reconstruct the target object. A native minimum construction returns a census member whose length bounds every other member. The minimal-description class retains every tied minimum. Each member decodes to the exact target and, through the registered interpreter interface, reconstructs the original computation request and its execution.

Translation retains its source encoding row and target decoder result. An actual translated description with an explicit overhead trace bounds the target minimum; concatenation preserves exact length accounting. Singleton and depth-extension proofs retain the complete finite construction. Descriptions beyond the declared depth are excluded. The original boundary remains: unrestricted shortest-description values are uncomputable, and finite censuses state their depth explicitly. The finite minimum procedure does not assert unrestricted completeness. All 256 original candidates and complete native program records and hashes match pinned V3.

171. Exact search on registered carriers

The translated search operation preserves the original admission order: repeated carrier identities are rejected before target membership is checked. A held target yields its original position label and exact identity. Native induction proves finite exhaustion, reconstructs the carrier as the unmatched prefix followed by the target and remaining suffix, and establishes that every earlier comparison differs from the target. Appending further carrier rows retains an already found position; the singleton base returns position-1. These search results combine with the exact ordering construction and complete native entry-program comparison presented below.

172. Exact ordering and complete recursion traces

The registered finite carrier generates its full arrangement census. Exact comparison rows select all arrangements satisfying the ordering invariant; native proofs establish that membership is precisely a full-carrier permutation satisfying every required comparison. Every duplicate-value class retains its original identities. The terminal certificate preserves the complete carrier, its length and the ordering invariant. Extension adds the new comparison rows while preserving the earlier invariant.

Structural recursion retains the initial value and the result of each exact registered successor step. Its trace contains one more value than successor positions, and every transition reconstructs from its prior value and position. Duplicate successor identities retain the original rejection. All three original search and recursion witnesses pass. The complete 256 algorithms-axis census and all native registration, decision, closure, control and hash outputs agree with pinned V3, completing the Fold Searching and Ordering Law.

173. Exact arithmetic execution certificates

The arithmetic execution certificate retains each original registered operation, ordered input identities, intermediate exact part and bound output identity. Native proofs establish both directions of correspondence between this stepwise certificate and the original trace recorder. The recorded operation sequence is reconstructed exactly, with one row per operation. Every retained part remains positive, bounded through the structural One and in coprime normal form. Original fraction normalization preserves exact value by cross multiplication.

Terminal replay follows from the complete recorded execution. Serial composition retains the shared intermediate part and both row sequences; successor extension records the exact next operation and its inputs. The structural One supplies the base execution. Original operation failures propagate unchanged. All 256 original candidates, operational witnesses and native program fields and hashes agree with pinned V3, completing the Fold Exact Arithmetic-Algorithm Law.

174. Finite sequence matches and position reconstruction

For the registered subsequence alternative, complete finite scanning retains every skip/take choice that reconstructs the target sequence. Each match carries the source positions of its held symbols. Native proofs establish both directions between generated match ledgers and scan traces, preserving distinct occurrence positions even when symbols repeat. The match ledger contains exactly one position per target symbol.

Every recorded position and symbol reconstructs an exact source prefix, held symbol and suffix, with the position equal to its original offset. All positions lie inside the declared finite source interval. Appending source rows preserves earlier matches and their positions; a single held symbol supplies the base. No ungenerated infinite stream is treated as complete. All 256 original candidates, operational witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold String and Sequence Algorithm Law.

175. Finite graph traversal certificates

The traversal certificate retains the complete frontier and visited ledger at each original search step. Empty-frontier termination, target detection, repeated-visit skipping and fresh-vertex expansion preserve the source branch order. A finite trace is constructed using the original decreasing unseen-support/frontier measure. Expansion appends every original outgoing unvisited target in edge order; no frontier cap or pruning changes the search.

Native replay yields the original reachability result. A successful rooted traversal provides the existing path witness, while exhausted-frontier failure excludes such a path. The same-node case retains original node-membership semantics. Existing path composition and graph extension proofs preserve the generated structure. Physical-network application retains the original empirical translation protocol. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Tree and Graph Algorithm Law.

176. Finite algebraic and geometric execution

An admitted closed operation table supplies every generated input pair with its exact carrier result. Native induction constructs a complete execution for every finite input trace, retaining every intermediate input/input/output row and proving that all intermediate and terminal identities remain in the carrier. Replay reconstructs the terminal result; row count and input projection preserve the complete input sequence. Serial composition and successor extension retain their exact shared identities and table rows, while table failures propagate unchanged.

For incidence geometry, each face coordinate is equivalent to its actual retained boundary row. The original cell-dimension algorithm is linked to its complete finite boundary-descent derivation, preserving admission and cycle rejection. Geometry remains finite and exact. All 256 original candidates, operational witnesses and complete native program records and hashes agree with pinned V3, completing the Fold Algebraic and Geometric Algorithm Law.

177. Exact subproblem reuse and reconstruction

The finite decomposition retains each subproblem identity and its complete ordered dependency list. Each dependency lookup reads an existing exact result, including repeated references to overlapping subproblems. A result is admitted only through its sole matching registered recurrence row. Executable table construction and its typed execution certificate imply one another, and each task adds one retained result. Its identity sequence is exactly the initial sequence followed by the schedule, preserving the source distinct-identity condition. Missing dependencies or unresolved recurrence rows halt the construction.

Every result constructed from the empty table has a finite derivation through the actual decomposition and recurrence rows. Reuse preserves the original dependency identity and value across later table extensions. Composition and successor preserve all earlier rows and their shared interface. A problem invariant follows by induction when its local recurrence preserves that invariant from the established dependency results. The decomposition and optimal-substructure conditions must be proved for each problem, as required by V3; no particular recurrence or optimum is supplied by the generic construction. All 256 original candidates, operational witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Dynamic-Programming Law.

178. Complete optimization traversal

The traversal retains every feasible candidate in source order and every exact incoming dominance row. Its terminal class is proved equal to the original complete undominated selection. Every eliminated feasible candidate has a retained, distinct feasible dominator; all undominated candidates survive. Ties and incomparable alternatives remain explicit, and uniqueness is asserted only for a singleton. Original feasibility and preference admission conditions are preserved without a score, metric, differentiability or convexity assumption.

The incumbent history contains its initial class and one state for each traversed candidate. Each prefix reconstructs its exact incumbent, and serial composition preserves that shared class. Extending both the candidate support and preference relation retains precisely the forms with neither an old nor a new dominator. An earlier optimum can disappear only through an added dominance witness. The singleton carrier supplies the base. All 256 original candidates, operational witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Optimization-Algorithm Law.

179. Scheduled algorithms and exact success support

Every registered schedule executes the same held deterministic kernel and initial form. The complete schedule ledger retains each identity and its execution trace or error, with one associated resource record. Exact outcomes define a covering, disjoint partition. Successful schedule identities are precisely the members of the registered target-outcome class; canonical schedule identities imply duplicate-free success support contained in the whole support.

The success part uses the original held-support/whole-support operation. Its positive result satisfies the exact count-ratio identity and lies through One; empty support selection retains the original empty-One marker. Resource reconstruction retains the full observation trace, its initial-observation-inclusive count and the supplied occupied-state support and count. Composition and successor preserve all previous records. Apparent randomness remains an unresolved or externally measured deterministic schedule, with the original measurement boundary unchanged. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Randomized-Algorithm Law.

180. Causally independent algorithm layers

Every task in an admitted ready layer reads the same prior retained result table. The dependency graph places each prerequisite before the layer and excludes dependencies between simultaneous tasks. Complete synchronization rows retain the corresponding prerequisite identities. Native layer execution and its typed trace imply one another, retaining each task identity and one exact recurrence result per task. Local invariants propagate from prior dependency results, and every generated output reconstructs through the original finite decomposition and recurrence rows.

Successive layers append their complete result rows to the shared table. The combined result identities equal the initial identities followed by the complete flattened layer trace. Complete task coverage therefore transfers directly to the work ledger, while the existing exhaustive path construction supplies the span certificate. Composition and successor preserve previous records and their shared interfaces. No physical speedup is claimed without an implementation experiment. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Parallel-Algorithm Law.

181. Global replay of retained local process events

A retained event identifies its process, prior local state, action and resulting local state. The replay checker resolves the registered process kernel and verifies the unchanged local transition before updating that process in the global state table. Native certificates and successful replay imply one another for every finite event trace. Process identities are preserved, and every process absent from the trace retains its original local state. Serial composition and successor extension reconstruct through the shared intermediate state. These results establish the local-state replay component; the message and causal/fault certificates below complete its connection to distributed communication.

182. Exact distributed messages and causal reconstruction

Separate send and receive ledgers retain each message identity, payload, process and event identity. Every record binds to the registered alphabet and an actual process event. Every declared receipt has a retained send with the same message identity and payload at an earlier event position. Sends without a declared receipt remain visible; delivery is not assumed. Canonical event identities and exact positions make the causal relation transitive and irreflexive, excluding causal return cycles.

An original local transition error prevents global replay from continuing to a result. A completed certificate combines the full local-state trace, canonical event identities and verified message ledger, reconstructing the terminal global state while preserving all process identities. Trace extension retains prior port bindings and causal decisions; ledger composition retains both complete record collections. System implementation and physical network claims remain outside the formal theorem. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Distributed-Algorithm Law.

183. Finite-prefix streaming and retained distinctions

Every generated input has a complete ordered prefix family, including its identity prefix. Native proofs establish both directions between prefix membership and exact input decomposition, giving one result record per prefix and a total of input length plus one records. Each successful result reconstructs its complete original execution path and terminal state summary. The original transition relation determines the next summary, and serial continuation uses the retained intermediate state.

The full ordered pair ledger retains exactly distinct supported inputs whose state summaries coincide. Inputs with different summaries are excluded from that loss ledger. Input extension preserves every earlier prefix, output and lost-distinction record. Each resource certificate retains the complete observation trace and supplied occupied support with their exact counts. Every result names a finite generated prefix boundary; no completed infinite stream is admitted. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Online and Streaming Algorithm Law.

184. Exact oriented numerical-error records

For equal nonempty finite supports of exact positive fractions, the numerical error ledger retains one ordered cell record per paired input. Equal values carry the structural One marker and the equal orientation; unequal values carry their exact positive difference and the held-below or held-above orientation. Native arithmetic proofs identify each normalized difference by its cross-multiplied rational value. Nonpositive values and unequal or empty supports retain the original rejection behavior. These results establish the error-ledger component of the numerical formalisation.

185. Complete numerical error-ledger reconstruction

Successful numerical error-ledger execution is equivalent to a complete ordered certificate pairing each reference value with its approximation and exact oriented row. The certificate proves positivity of both input supports and every error value, equal support lengths, and one row per pair without omission. Conversely, every equal-length nonempty pair of positive supports constructs a successful ledger. Serial composition and successor extension preserve earlier rows while advancing the original cell labels. Independent finite expression comparisons exercise all 1,849 pairs of supports of widths up to two over six exact signed/zero/positive test values; their output and rejection records agree with native Lean execution.

186. Retained numerical update observations

Every state in a generated update trace resolves to its registered numerical carrier and a complete exact error ledger. The observation checker and the retained certificate imply one another; the ordered projection reproduces the entire state trace, and every numerical value is traced to its registered carrier row. Connecting these records to the original transition execution gives exactly one observation for each action plus the initial state. Composition retains all intermediate records. An admitted successor step appends its exact observation while preserving the earlier trace. Missing carrier values or rejected error ledgers cannot produce an accepted observation trace.

187. Finite numerical boundary certificates

The numerical carrier uses exact positive parts on finite generated cells. Its retained state updates reconstruct every oriented error row. Output enclosure is the complete held-order selection between registered bounds: every selected form belongs to the carrier and has both endpoint relations. Original exhaustive lower and upper bound results enclose each held output. A terminal certificate retains original state-constructor and order admission, the actual finite execution, all intermediate observations, its explicit target and its enclosure. The final numerical value and complete error ledger reconstruct from that certificate.

Projecting each original transition to its source and target connects executed paths to the admitted dynamics relation. The registered stability check holds exactly when every registered perturbation has a retained path to the target. Complete perturbation path families establish that check, and the target retains both enclosure relations. Successor construction requires the exact next transition, numerical record and enclosure evidence, retaining all earlier observations. This formalises the declared finite certificate boundary; it does not assert that every numerical procedure converges or import an ungenerated infinite limit.

The complete 256-candidate numerical program, its original operational witnesses, controls and all recorded hashes agree with pinned V3 under fresh native execution. Together with the carrier, update and boundary derivations, these results complete the Fold Numerical-Algorithm Law.

188. Exact symbolic transformations within the frozen grammar

The generated expression grammar contains positive exact values, held names and binary joins. Native evaluation preserves original binding uniqueness, literal admission and left-to-right error behavior. A successful result is equivalent to a complete evaluation derivation. Bound names retain their original raw Fraction values; the scientific normalization certificate separately carries the positive-binding boundary. Exact signed arithmetic is represented constructively, including cancellation and normalized positive differences.

A held tree occurrence is identified by its generated left/right address and surrounding context. Native proofs establish both directions of that representation and reconstruct the complete expression. Equivalence of successful exact values is preserved inside every context. Each rewrite record retains a frozen registered rule, its held occurrence and its equivalence proof. All intermediate and final values are preserved along a complete trace; composition and successor retain earlier records. A normalization certificate includes the terminal absence of an admitted rewrite. Its final expression and exact positive scientific result reconstruct from the retained trace. No external algebraic rewrite theory or unconditional termination or confluence assertion is required by these results.

All 256 original symbolic kernels, witnesses, controls and complete program fields and hashes agree with pinned V3. Independent evaluation of the source expressions agrees with Lean in all 7007 cases

spanning 143 expressions and 49 environments, including invalid literals, missing and duplicate names, signed values and nested joins. These results complete the Fold Symbolic-Computation Law on its frozen grammar boundary.

189. Exact approximation-bound admission and source rows

The inherited approximation operation is accepted exactly when the optimum cell count is positive and no greater than the candidate cell count. This equivalence preserves the original signed-integer input domain and its rejection behavior. Each successful part reconstructs the exact optimum-to-candidate ratio by cross multiplication; its numerator is positive and bounded by its denominator. A retained bound record identifies both original objective rows. Extending those rows preserves the candidate, optimum and exact part without changing the objective or introducing a tolerance. These results establish the bound component of the approximation formalisation.

190. Approximation with complete objective and execution ledgers

The exact objective observation binds the feasible candidate and held optimum to their original cell counts. Approximation partitions every original objective distinction into retained or lost records; the partition is exhaustive and disjoint. Loss requires unequal original values and equal approximate values. Identity loses no original distinction, and an admitted downstream composition cannot restore a distinction it has closed. Preserving the original row images preserves every earlier decision under extension.

The same candidate and optimum counts generate the original exact oriented error row. A result certificate retains original feasibility and preference admission, complete objective rows, the actual state path, every registered numerical observation and the exact terminal lookup. Native reconstruction proves that its error belongs to that candidate and reference. The resource ledger reproduces the entire state trace, supplied occupied support and exact counts, including the initial observation. Identity, serial continuation and successor preserve the recorded interfaces; the successor may carry a different candidate with the same reference. These finite certificates introduce no tolerance, metric or conventional approximation algorithm. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Approximation-Algorithm Law.

191. Generated program syntax and complete occurrence records

The formation checker accepts exactly the generated value, name and binary join trees. Positive exact literal payloads and held names remain explicit. Rejection is equivalent to a constructive malformed derivation identifying an invalid literal, unsupported constructor or malformed child. Syntax membership imposes no binding environment: a well-formed name may remain unbound under execution.

The tree ledger retains the root and every child occurrence with its complete left/right address and exact subtree payload. Membership is equivalent to successful focus at that address, in both directions; one address identifies one subtree. Every occurrence of an accepted tree is formed and has a registered constructor. An invalid occurrence rejects the entire tree. The literal base has its identity record. Joining trees preserves both complete ledgers with their address prefixes, and generated contexts compose by retaining the held subtree. All 256 original candidates, the three exact semantics witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Program-Syntax Law.

192. Complete name occurrences and registered scope

Every name occurrence in the registered expression tree has a retained structural address, exact label and original environment lookup. Ledger membership is equivalent to a native occurrence derivation. The ledger contains exactly as many records as the tree contains names; free and bound records form an exhaustive disjoint partition. A bound occurrence reconstructs its exact environment row, and a free occurrence has no registered lookup. Preserving lookup results preserves the entire occurrence ledger.

Every successful evaluation resolves every name occurrence to its original value. Conversely, resolving every occurrence in a generated expression constructs its evaluation derivation. The original unique-binding admission remains part of executable evaluation. These results establish the name-occurrence and scope component of the binding formalisation without assigning a new binding grammar.

193. Binding-extension formation and exact substitution behavior

The SEMX extension supplies value, name, join, let and four-branch same term shapes. Its formation decision is equivalent to a native derivation retaining the original bound-name list; formed terms have no free name relative to that list. Free-name set membership preserves the original initializer and body scopes. Value payloads are carried unchanged. Whole-tree renaming retains the source operation, and identity renaming preserves the complete term.

The original fresh-name search examines u, v, w and z in order, excluding exactly the free names of the body and replacement and the substituted name. Every selected name has that provenance and those exclusions. Exhaustion holds exactly when all four candidates are unavailable.

The translated substitution preserves literal values, exact name matches, ordered join and same recursion, the held-binder stop in the body, substitution into the initializer and conflicting-binder renaming before descent. Accumulated name mappings are proved equivalent to the source whole-body rename operation for arbitrary generated terms. Fresh-search failure precedes initializer substitution as in the source. The original let-y/name-x witness reconstructs its u-bound result. These are exact behavioral correspondence results; a general capture-preservation result is not inferred from the witness.

194. Capture preservation at the evaluator substitution boundary

The original evaluator enters let substitution only after the initializer is a value. This source guard is equivalent to the value-constructor shape. On that exact call domain, structural value replacement agrees with the original substitution operation for every generated body and cannot fail through fresh-name exhaustion. Formation under the discharged binding produces formation in the remaining scope, including shadowing and initializer recursion.

The complete binder ledger retains every original structural address and binder name unchanged. Substitution removes the discharged free name and preserves every other free name. Thus all evaluator substitution calls preserve the stated binder identities and free-name distinctions. The standalone helper admits open replacements outside this evaluator call boundary; these results do not assert its unrestricted capture behavior.

195. Canonical replacement in the registered expression grammar

Binding identity is the exact name admitted by the original environment uniqueness check. Successful admission of a new target binding proves its identity is absent from the surrounding environment; conversely, a fresh identity constructs an admitted extension. The new lookup returns its recorded value, while every other binding is unchanged.

Structural replacement has a complete native derivation for each value, name and join. The derivation and computed result reconstruct one another, preserving original payloads and complete child trees. For an arbitrary open replacement evaluated in the surrounding environment, substitution reconstructs the original evaluation under the discharged target binding. Every resulting occurrence resolves to that exact surrounding environment row. No binder or presentation alias is introduced by replacement. Formation, identity replacement, sequential composition and generated join successor are preserved. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the registered Fold Binding and Substitution Law.

196. Complete source evaluation traces

The evaluation record retains each entered expression and exact environment, every visited recursive child, and its terminal value or original failure. The native execution derivation preserves duplicate-binding rejection before descent and left-before-right evaluation. A first-child failure stops the join; a second-child failure retains the first completed trace. Successful joins retain both complete traces and their exact sum.

Computation and native derivation reconstruct one another. Every recorded result equals the original evaluator result, and two valid executions of the same expression in the same environment have identical results and complete event lists. Each accepted evaluation has such a trace. The literal base retains its identity record; join composition and successor preserve every visited child record, its syntax and its binding environment. All 256 original candidates, operational witnesses, controls and full program fields and hashes agree with pinned V3, completing the Fold Evaluation Law.

197. Operational and compositional meaning correspondence

Constructor-indexed meaning records retain the original syntax, environment, exact result and ordered failure clauses. Soundness transforms each complete operational execution into its meaning record. Adequacy reconstructs a complete operational execution from every meaning record. The result map is equivalent to the original evaluator, and meaning is unique. Consequently no admitted execution and corresponding meaning can disagree on their result within the frozen grammar.

The positive literal supplies the base correspondence. Join assembly preserves both complete component traces and their exact sum. Equality of meaning observations is equivalent to equality of the original result, including failure. Identity, serial composition and every frozen expression context preserve this observation relation. Existing complete occurrence and execution ledgers preserve the original syntax and binding identities throughout assembly. All 256 candidates, source operational witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Operational and Denotational Correspondence Law.

198. Positive semantic-value typing and preservation

Within the registered value/name/join grammar, the typing derivation retains a complete premise tree for positive exact literals, exact positive name bindings and binary composition. The typing decision is equivalent to this derivation and implies generated syntax. Every typed expression evaluates to a positive exact value under the original unique-environment admission, and its complete operational trace reconstructs. Structural substitution preserves this typing relation; evaluating the substituted expression retains a positive result and complete trace.

Rejected typing checks have explicit mismatch derivations: invalid literal payloads, absent or nonpositive name bindings, unsupported constructors, or a mismatched child. Rejection and mismatch derivation are equivalent, and a typed expression cannot have such a mismatch. These judgments characterize the positive semantic-value type; the original evaluator and its raw signed environment behavior remain unchanged. They establish the positive typing and preservation component of the Type Theory formalisation.

199. Generated type support and exact interface derivations

Types retain the original source-bound predicate on a finite generated carrier. Extending the carrier while preserving earlier verdicts retains the entire prior ordered type support and adds exactly the admitted new rows. Product extension retains the old joint support and every new pair. Sum extension retains the original held left/right labels and their order. These are generated source inputs, with no additional scientific parameter.

An interface derivation records every original composition call, its two arrows, the intermediate result or its first rejection. Replay and reconstruction establish equivalence with the source path fold. Successful composition requires exact interface identity; unequal interfaces produce the original rejection. Serial assembly preserves complete component traces, and a rejected prefix remains rejected under extension. The reconstructed flattened trace contains every earlier and appended component record. Together with positive typing, substitution and evaluation preservation, these results formalise the registered structural-admissibility requirements. All 256 original TYPE candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Type-Theory Law.

200. Program equivalence on registered observations

For each input in the common finite support, the ledger retains both complete source executions and their registered observation labels. Ledger membership reconstructs the original input and both execution derivations. The equivalence certificate is true exactly when observations agree at every registered input. A failed certificate constructs an input with unequal observations.

Equal and unequal observation rows form exhaustive disjoint closed and retained distinction ledgers. Equivalence closes every observation row, while a retained distinction refutes equivalence. The complete underlying traces remain available in both cases. Identity, symmetry and serial equivalence composition preserve the declared observation relation. Support assembly and successor retain every earlier execution record; extending support requires agreement on its new inputs. A declared post-map preserves already equal observations. These results respect the original observation and input boundary. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Program-Equivalence Law.

201. Generated descent and terminal coverage

Every recursive call of the registered expression evaluator follows a strict child of its generated syntax tree. Structural induction constructs accessibility, and the retained node count decreases at each call. The call relation preserves the original duplicate-binding guard and admits the right child only after successful left evaluation. Every expression and input reconstructs a complete execution ending in its exact value or original failure.

For the original transition system, a retained descent certificate decreases on every transition. Each complete path has length bounded by its initial rank; native execution reconstructs both that bound and its exact state trace. Serial assembly and successor preserve the complete records and bound. A nonempty return cycle contradicts strict descent, and the registered loop provides an explicit counterexample to the existence of such a certificate. These results preserve the stated boundary: no total termination decider for the universal language is claimed. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Termination Law.

202. Correctness against the registered specification

The specification retains its exact precondition, postcondition on complete execution records, and partial or total termination declaration. Partial correctness establishes the postcondition for each retained execution of an admitted input. Total correctness additionally retains execution existence, supplied here by the finite registered evaluator. The complete-support certificate is equivalent to the declared correctness relation in either mode. Rejection reconstructs an admitted input whose postcondition fails.

Every input row retains its complete execution, specification decisions and termination declaration. Failure rows are exactly those carrying original runtime errors; their acceptability remains governed by the registered postcondition. Support assembly and successor preserve every earlier row. Specification consequence retains the same original execution under proved precondition and postcondition implications. Serial agreement connects original path certificates through their exact middle interface and retains both component traces. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Program-Correctness Law.

203. Generated formal specification relations

A specification retains its generated input and output carriers and exact allowed relation under the original relation-admission guard. Membership proves both carrier identities. The complete observation certificate is true exactly when every observed row is allowed; falsification retains every observed row excluded from that relation. Refinement is allowed-row inclusion, preserving verified observations through identity and composition. Removing an allowed row produces an explicit falsification at the new boundary.

Invariant checking covers every state of the retained source trace and follows from the admitted initial invariant and transition preservation. A finite liveness witness retains original input admission, a complete executable path and its visited registered goal. A terminal obligation retains the original halted execution and normal terminal state. Both reconstruct native execution, and serial extension preserves the earlier goal and component records. The generated relation supplies the postcondition of the existing correctness checker; every verified execution therefore reconstructs an allowed pair in the original input and output carriers. All 256 candidates, witnesses, controls and full native program fields and hashes agree with pinned V3, completing the Fold Formal-Specification Law.

204. Retained program transformation traces

A generated transformation retains the registered rule identity, source syntax, target syntax and exact contextual occurrence at every step. Semantic correspondence follows through arbitrary finite traces by contextual preservation and serial composition of the original evaluation relation. Every successful source evaluation reconstructs a complete native target execution with the same value. The proof retains every rule and both generated endpoints.

The recorded predecessor rows supply a reverse traversal of the complete transformation; a target without its record cannot recover distinct predecessors that share it. Resource evidence remains the original separate resource-ledger operation, retaining the initial observation, each rule event and occupied support. Semantic preservation alone asserts no resource improvement. The complete 256-member grammar, operational witnesses, controls and native program fields agree with pinned V3.

205. Semantics-preserving compilation relations

The generated compiler relation retains both frozen language grammar carriers and every allowed source-to-target row. Its finite coverage check is equivalent to every source form having a target. A complete compiler certificate checks that coverage and every relation row against the declared runtime interface. It therefore derives a total verified translation relation, with source and target grammar exclusion and rejection of failed semantic checks.

Each observation row reconstructs both complete native executions. The result correspondence composes through an exact intermediate syntax and unchanged runtime input and observation interfaces. Identity and input-support extension preserve the certificate and complete observation records. Overhead is recorded by the original separate source and target resource ledgers, retaining every execution event and occupied support. All 256 grammar candidates and full native program outputs agree with pinned V3.

206. Machine-checkable program evidence

Program evidence retains the original proof-step grammar, registered premises and rules, specification, complete input boundary and bound program identity. The checker verifies both source and program identities, the complete native inference trace and the actual occurrence of the declared conclusion. The registered interpretation supplies proved rule and premise soundness and the exact meaning of that conclusion for the retained program and specification. Native soundness then establishes correctness over the complete declared boundary.

Mismatched identities, invalid proof traces and missing conclusions cannot pass the checker. Identity proofs retain an already-held conclusion; serial proofs preserve every step through the exact intermediate proposition state. The complete 256-member grammar and native program comparisons agree with pinned V3.

207. Positive causal relation expansion

Causal expansion retains the original edges and adds exactly endpoints joined through a held intermediate event. A positive path contains at least one original edge; reflexive edges are not inserted by convention. Native proofs show that every finite expansion round preserves original edges and introduces only original positive paths. A closed result containing the original edges is their least transitive closure. Its self-edge exclusion is equivalent to exclusion of positive cycles. These results establish the relation-level expansion and cycle boundary; the complete Concurrency entry is verified below.

208. Native causal fixed-point execution

Every growing expansion round fills at least one previously missing pair cell of the finite event support. This strict decrease establishes native termination without a cutoff. The executable result retains its complete expansion derivation and no-growth certificate. For carrier-supported original edges, its relation contains exactly the positive original paths; the final self-edge test is therefore exactly the positive-cycle boundary. Executed examples reconstruct the three-edge closure of a three-event chain, the complete cyclic relation of a two-event cycle, and empty closures. The complete Concurrency entry is verified below.

209. Exact causal operation boundary and output

The native causal operation preserves the source guard order: repeated event identities are rejected first, followed by escaping edges and positive cycles. Repeated edges are retained as set input and produce no repeated output row. Successful output has exactly the original positive-path relation and is emitted in lexicographic string-pair order. The original three-event witness is established directly by kernel reduction. A further 1602 complete native output/error comparisons cover every three-event directed graph, reversed input order, duplicate edges, Unicode and empty labels, and guard precedence. The exact missing-pair bound supports structural recursion: at zero a growing round is contradictory, so no incomplete result can be returned.

210. Complete concurrency schedules and local observations

The complete event schedule support retains exactly the permutations of the declared event carrier that respect every held causal relation row, including the empty identity schedule. Each retained event binds its exact process identity, local before-state, action and after-state to the original event and process registries. Native proofs establish that every scheduled event is an enabled original transition and that every registered process reconstructs its complete native observation through its terminal state.

Serial assembly preserves intermediate local states and every event record. Two enabled independent events on distinct processes may occur in either order with the same local endpoints; this is structural coexistence without a physical simultaneity assertion. The original causal kernel, three operational witnesses, complete 256-member grammar, four controls and every native program field agree with pinned V3, completing the Fold Concurrency Law.

211. Computational causality and retained provenance

The event predecessor structure retains the original generated event carrier and the separate process-order and message-edge ledgers. A positive predecessor path is formed by an original row or by composing two such paths at their shared intermediate event. Native proofs equate the successful causal-closure output with precisely these retained source paths; every process-order and message row is preserved. The original event-uniqueness, endpoint and cycle guards remain unchanged.

Transitivity follows from path composition, while the admitted closure has neither self-predecessors nor reverse predecessor pairs. Incomparability is the absence of paths in both directions. Its complete generated pair support retains the event memberships without assuming a total global order. Empty edge ledgers have no predecessors. Assembly preserves both component ledgers and their interface paths, and every earlier causal output row remains in an admitted extension. These results concern computational predecessor structure; physical causal claims retain V3's later empirical-branch boundary. The original 256 candidates, operational witnesses, four controls and full native program outputs are verified against the pinned source, completing Fold Computational Causality.

212. Structural generator three: exact recurrence derivation

For every finite Fold depth, the recursively doubled support count equals the cardinality of the full generated Fold-word support. At positive depth this count exceeds One and has a unique positive predecessor. Taking the unit part with that predecessor as denominator gives the unit third at depth two and the unit seventh at depth three. Each equality is proved from the native support construction and exact part normalization.

The native Fold retains the complete first-return trace 1/3 → 2/3 → 1/3 and the complete trace 1/7 → 2/7 → 4/7 → 1/7. The intermediate states are proved distinct from the initial state, establishing first return with respective positive transition counts two and three. For every count strictly after two, three is no larger; every count strictly after three is therefore nonminimal. This establishes existence and minimal positive-successor order without a bounded denominator search or measurement premise. The executable first-return routine, complete ten-axis program and original controls are verified below.

213. Native first-return execution

The original Fold first-return operation is implemented with its complete transition trace. It advances the exact positive part, tests equality with the original part, and then tests previous visitation. A prior visit before return produces the original recurrence error. A proof carried by the native execution records precisely these branches and connects every successful result to the retained first-return predicate.

Reduced denominators cannot increase under Fold. Consequently every executed part belongs to a finite carrier determined by the original denominator, and each fresh transition strictly reduces the remaining carrier. This proves termination without an arbitrary iteration bound or an additional scientific parameter. The native binary and ternary outputs and generator-period result are kernel-proved. Public Fraction guards and positive integer/Boolean count guards retain the original admission and rejection behavior. Complete-output comparisons for all 820 numerator/denominator pairs through denominator 40, including repeated rational representations, returning and nonreturning parts, and four invalid inputs pass against independent exact-arithmetic reference traces. The finite test set supplements the general loop and first-return proofs.

214. Complete Structural Generator Three entry

The ten registered binary structural axes generate all 1,024 candidate forms. Native proofs establish complete enumeration, exact decision correspondence and the sole preserving form, with each rejected coordinate violating its original registered requirement. The three operational witnesses evaluate the native unit-part first-return routines and the derived generator-period operation. Their outputs reproduce the complete binary and ternary traces and the exact successor result three.

The full StructuralPhysicsProgram reproduces every registered candidate, identifier-based decision, complete closure record, original control field and associated hash against pinned V3. The full-entry result therefore includes the Fold-support construction, positive predecessor, first-return existence, minimal successor proof, original executable guards and all 1,024 forms. This completes the Structural Generator Three entry with no axiom dependencies or free scientific parameters.

215. Spatial Dimension Three

The complete ordered pair support of the two held Fold fibre roles contains four cells. The original stability constraint retains a positive independent-direction count strictly above the fibre boundary and strictly below pair-cell saturation. Native enumeration retains every positive count in this open window, preserving ascending order and excluding both endpoints. For every generated count, these two strict inequalities imply equality to three; the complete native result is the singleton[3].

The independently completed Fold-recurrence operation also returns three. The native spatial-dimension operation checks equality between the complete stability census and the singleton recurrence result, establishing the original cross-lock. Exact integer and Boolean guards are retained. All 1,024 original grammar forms, three operational witnesses, four controls and complete program fields/hashes are verified against pinned V3, completing Spatial Dimension Three. Measurement remains downstream of the sealed structural count.

216. Boundary Rank Two

Holding the source-normal direction in the forced three-direction carrier leaves exactly all nonnormal independent directions. Their count followed by the held normal reassembles the carrier, establishing the unique positive predecessor two. Native execution retains the original predecessor and reassembly check.

At any generated positive scale, the two independent coordinate supports form the complete ordered pair product. Membership is equivalent to membership of both coordinates; each pair occurs once and the cardinality is the product of the coordinate counts. Extension of either coordinate retains all previous pairs and adds its pair with every coordinate on the other direction. Equal scale r therefore yields r paired with r. Exact rational growth preserves this product by normalized multiplication, with the original strict above-One input boundary. The scale-two and scale-three witnesses yield four and nine.

All 2,048 original candidate forms, three operational witnesses, four controls and complete native program fields and hashes match pinned V3. This completes Boundary Rank Two without a measured exponent, conventional surface formula or continuum premise.

217. Inverse fine-structure cover construction

The native binary count is the positive predecessor of the independently completed generator-three recurrence. Positive powers retain the source repeated-multiplication operation and its exact count domain. For every admitted carrier, the binary-cover search returns a positive depth whose complete Fold support covers the carrier; every earlier positive depth has strictly smaller support than the carrier. Its finite execution budget follows from the carrier and increasing positive support and does not truncate the search. The support count is proved equal to the cardinality of the complete Fold-word enumeration at every depth.

For generator volume three cubed and its successor volume, the least depths are five and seven. These independently agree with the source binary/generator decompositions. Executing the original typed block construction yields binary two, generator three, down depth five, up depth seven, tower 128, boundary 9 and cover 250. These verified cover and block results supply the complete finite promotion ladder and exact rational assembly described below.

218. Terminal inverse fine-structure ratio

The complete three-direction promotion object begins with three down-depth factors. Each successor replaces one remaining interchangeable direction by an up-depth factor, preserving the total count of directions. Its remaining-direction count decreases strictly and reaches structural absence after three promotions. The generated stages are distinct and exhaustive, with complete rung supports 125,175,245,343. No further successor is admitted once all directions are promoted.

The native continued assembly reads a positive rung count from one through four. It forms the descending chain of each rung plus the returning One divided by the following chain, then adds the reciprocal chain to cover 250. Exact normalized positive-rational arithmetic retains the typed tower 128 plus boundary 9 multiplied by the covered-return dilation. All four finite orders are represented; the default is the terminal fourth order. The leading ratio is 34259/250 and the terminal ratio is 503846395469/3676744786. Both identities are kernel-proved with zero axiom dependencies.

The first rung, 125, supplies the base cover 250 = 2 x 125 and hence the separate leading-order result; the terminal fourth-order assembly adds the reciprocal continuation built from the remaining rungs 175, 245 and 343.

Writing the terminal effective cover as D, the exact assembly is:
D = 250 + 1 / (175 + 1 / (245 + 1 / 343)).
alpha^(-1) = 128 + 9(D + 1)/D = 503846395469/3676744786.

All 1,024 original candidate forms, three native operational witnesses, four controls and full program outputs and hashes agree with pinned V3. This completes the registered inverse fine-structure entry. Measurement remains excluded from the construction and can only enter its separately registered downstream comparison.

219. Charged-lepton invariant operations

The complete colour support retains every word of each generated depth exactly once. Its cardinality is three to that depth; the two held Fold-fibre copies of the fifth-power support therefore contain 486 cells. The original predecessor operation returns 485, whose successor reassembles that complete support, and no other positive predecessor has this property.

Exact positive take admits a positive part strictly below the held whole. Its normalized remainder preserves the rational difference, and adding the retained part reconstructs the whole at the common

denominator. The native charged-lepton operation pairs the two fibres with generator three and applies the original neighbouring channel census. Channels two, three and four yield 2/969,3/1454 and 4/1939 respectively; only three has the identity of the admitted generator carrier. The constructed invariant tuple is 1,1/6,1/485,3/1454, without forming an irrational root. These native operation and support results are connected to the complete symmetric carrier below; the registered charged-lepton cubic entry is verified.

220. Complete charged-lepton symmetric carrier

The three-root carrier is represented by formal position identities, retaining exact symmetric coefficients without producing numerical roots. The complete distributive expansion contains every selection of the three formal factors exactly once. Grouping by selected degree yields the three singleton terms, all three distinct pair products and the single triple product, together with the leading One. The recursive expansion and the generated ledger are proved equal. Appending a fresh generated term retains every earlier term and records the new term once.

The native generator fixes the carrier count at three. The exact construction assigns root sum 1, pair sum 1/6, leading product 1/485 and the carrier-identity sharpened product 3/1454. All 2,304 original grammar forms, three native witnesses, four controls and complete program outputs and hashes match pinned V3. This completes the charged-lepton cubic entry while preserving its exact-rational invariant boundary and exclusion of measured or fitted root values.

221. Terminal charged-lepton self-coupling

The terminal operation uses the native generator three, fine-structure cover depths five and seven, and reciprocal terminal inverse fine-structure ratio. It attaches one alpha factor to every formal root, divides the resulting cubic coupling by the complete 27-cell colour carrier, and weights it by 5+(7/3)alpha. The correction is exactly 37692619301370829019075816481640237155527 2/52200858161852702444325842762753846686284139832 01. Holding this positive correction from 3/1454 yields 15659709397871168801443815466454510956837000584115/75900047767333829354049775377044093081857139315742 54. Native kernel proofs establish that the correction is strictly smaller than the admitted product and the retained product remains positive.

This result preserves the V3 observational-derivation provenance: the executable relation takes no measured lepton value, while empirical admission additionally requires the two complete post-seal CODATA comparisons and their custody controls. The exact native operation, registered structural-program comparison and native empirical validation are complete, including both post-seal CODATA comparisons and their custody controls; the Charged-Lepton Terminal entry is complete within its original empirical and observational-derivation boundaries.

222. Terminal grammar and exact root brackets

The complete 1,024-form terminal grammar preserves all ten registered axes, original decisions and unique survivor. Fresh native program execution reproduces every original census, closure, control and digest, including empirical evidence mode and observational-derivation provenance. The three operational witnesses are computed from the native correction, cover depths and retained product.

Post-seal evaluation translates the exact signed cubic and all original bisection branches. The dyadic isolation procedure enumerates every cell at each visited depth and retains exact lower roots or opposite-sign cells according to the source rule. A constructive run certificate links the complete native search to the original stopping rule: exactly three positive, nondegenerate brackets. For the native terminal product the search starts at depth one, rejects earlier stopping points and succeeds at

depth seven with brackets(1/128,1/64), (3/16,25/128) and(101/128,51/64). No numerical irrational root is constructed. The rational refinement, source parsing, custody and all-row validation are connected below.

223. Exact terminal mass-ratio refinements

The squared-ratio operation retains the source positivity checks and forms its lower bound from the numerator lower endpoint divided by the denominator upper endpoint, and its upper bound from the numerator upper endpoint divided by the denominator lower endpoint, squaring both exactly. Refinement bisects every retained root bracket together and stops on the original disjointness or containment condition. A constructive run theorem proves that every successful native refinement follows those transitions and retains every bracket.

Starting from the native terminal product and its complete dyadic isolation, the muon-electron computation reaches its exact registered interval after 26 refinements and the muon-tau computation after 14. Kernel proofs establish the full runs, exact final rational endpoints, containment and overlap with both registered source intervals. Independent exact-fraction comparisons reproduce all 42 successive prediction intervals alongside division, positivity and indexing checks. These results use the exact registered source endpoints; native fixed-width source-row parsing and the post-seal custody/adverse-control gate are connected below.

224. Exact source rows and complete comparison record

The native decimal parser retains positive finite decimal records, exact exponent scaling and the original rejection conditions. Fixed-width selection retains the complete snapshot row census, requires exactly one row with each requested quantity name, and parses the central and uncertainty fields without floating arithmetic. The uncertainty must be strictly positive and below the central value. Both complete parsed interval triples are kernel-proved equal to the registered rational bounds and central values. Fresh native file execution confirms the complete row sequence and both parsed results from the pinned snapshot.

The native comparison record combines those parsed triples with the actual terminal construction and interval refinements. It retains both predicted intervals, both source intervals, both central values, each overlap outcome and their conjunction. The complete record is proved successful and independently reproduced field by field. The source-row and numerical comparison components feed the original post-seal held-label prediction, custody and adverse-control procedure.

225. Registered terminal prediction and empirical validation

The registered predictor reads the sealed derivation premise, constructs the physical-observation label, retains the premise/label pair, and emits the label. Native proofs establish the output, retention of the bound premise and all four trace positions for every admitted nonblank premise. Source data-only instruction fields are retained. Complete before/after protected-tree records are compared across prediction execution. The target vault commits both original external rows, permits a single release only after the matching envelope is sealed, and verifies the experiment, custodian, seal, support, target content and release identity.

Both released rows are compared without omission. The original unfavorable control changes the external-observation label and is rejected. The numeric comparison and label comparison must agree; a disagreement produces the original terminal validation failure. The native wrapper is connected to the kernel-proved complete source-row and refinement result. Fresh execution reproduces every original empirical-validation field, including the prediction trace, isolation certificate, custody

certificate and measurement-receipt identity. Host metadata in the source receipt is retained explicitly as metadata, independently of native Lean execution. The Charged-Lepton Terminal entry is complete within its original empirical and observational-derivation boundaries.

226. Electroweak Fold-fibre mixing

The native binary count generates the half-One balance image. At binary depth two, the complete support is the ordered sequence 1/4,1/2,3/4,1. Filtering that entire support by exact Fold return yields precisely 1/4 and 3/4. Both parts are proved to return to 1/2, they are distinct, and their positive sum is exactly One. A completeness theorem excludes every other depth-two preimage. The source construction therefore returns unified=1/2, sin_squared=1/4 and cos_squared=3/4.

The enumeration preserves the original rational-image and positive-depth guards and every generated support position. Native membership equivalence identifies each retained part exactly with membership in the original support together with the required Fold image. The complete 256-form grammar, unique preserving form, three operational witnesses and four controls reproduce all pinned V3 program fields and identities. Measurement remains a later sealed claim, outside this bare exact squared-channel construction.